# Senior Software Engineer interview preparation

3 MAIN SECTIONS • PRINT-READY EDITION

**Current round:** technical discussion and coding. **Available study time:** 20 focused hours. **Scope:** backend technologies. Full system design belongs to your next round.

This guide is tailored to the four resumes you provided. The practice questions are preparation recommendations, not verified American Express interview questions. Your specific team, job description, Java version, and coding platform are still unknown. Use Java for practice unless the interviewer specifies otherwise.

## 1. Start here

Your strongest interview material connects Java and APIs to real business problems: commercial user management, payments, reconciliation, IAM/MFA, certificate automation, Redis caching, and production reliability. Your main preparation task is to turn broad leadership statements into precise explanations of implementation, tests, failures, and personal contribution.

For each concept, follow this cycle: **understand it in simple English → explain it without notes → implement or trace a small example → answer a failure question.** Reading alone is not enough for a coding round.

The professional answers in this guide are speaking templates. Learn the meaning and structure rather than every word. Adapt first-person statements to your actual experience. A clear 40-second answer is better than a memorized two-minute speech.

### Resume comparison and priorities

| Resume | Useful preparation evidence | How to use it |
|---|---|---|
| Sree.pdf | Java/J2EE, Spring MVC/Data/Integration, Vert.x, SOAP/REST, DB2/Oracle/MySQL, Couchbase, EDI and batch migration | Use for technical history and integration examples. It has an older experience summary. |
| Sree-Resume.pdf | Java/Spring/Vert.x, Redis, IAM/MFA, certificate automation, deployment practices; Airflow and Spark in skills | Use for backend delivery and security stories. Confirm depth behind skills-only entries. |

| Resume | Useful preparation evidence | How to use it |
|---|---|---|
| sree_resume-1.pdf | Most explicit backend detail: Vert.x/Java services, Spring Boot, Redis, Jaeger, Splunk, DR, Kubernetes/AWS, authorization | Use as the main study map. Verify each claim before repeating it. |
| sree_resume-2.pdf | Product strategy, payments, migration, operational outcomes | Use for business context. Add implementation details when answering engineering questions. |

**Priority A - demonstrate confidently:** Java collections and correctness, coding patterns, SQL, concurrency, Spring transactions, Vert.x event loops, REST, authorization, and your own projects.

**Priority B - explain and troubleshoot:** Redis, JVM performance, testing, tracing, deployment rollback, payment retry safety, certificate rotation, Couchbase.

**Priority C - brief review only:** AWS/Kubernetes specifics, Node.js, Airflow/Spark, SOAP/EDI tooling, deep distributed-system architecture. Move a topic up only if it is central to the role or to a project you choose to discuss.

# 2. Your 20-hour plan

The hours below are focused study time. Add short breaks, meals, and a full night's sleep separately. Target hours 1-10 on Thursday and hours 11-20 on Friday; move blocks to match your actual availability. Finish demanding practice at least 60 minutes before the interview. Use the last half-hour of study for recall, not new material.

| Study hours | Duration | What to do | Completion checkpoint |
|---|---|---|---|
| 1 | 1 h | Baseline: 15 min resume/intro; 30 min Two Sum + one SQL query; 15 min review | Identify your three weakest areas; record a 60-second introduction |
| 2-3 | 2 h | Java fundamentals, collections, equality, exceptions, generics, streams, money | Explain cards 1-9; code grouping and sorting without reference |
| 4-6 | 3 h | Coding patterns: hashing, sliding window, intervals, binary search, stack, heap/ tree basics | Solve exercises 1-6; narrate complexity and edge cases |
| 7-8 | 2 h | Concurrency, JVM, asynchronous execution, Vert.x | Explain cards 10-17; diagnose a race and a blocked event loop |

| Study hours | Duration | What to do | Completion checkpoint |
|---|---|---|---|
| 9-10 | 2 h | Spring/Boot, transactions, persistence, REST and tests | Trace one request; explain rollback and self-invocation; write tests |
| 11-12 | 2 h | SQL, indexes, locking, Couchbase and Redis | Write four SQL queries; explain lost update and cache invalidation |
| 13 | 1 h | API security, IAM/MFA, certificate lifecycle, retry safety | Explain cards 29-35 and answer duplicate-payment scenario |
| 14 | 1 h | Observability, incident response, deployment and secondary skills | Diagnose one slow API; rehearse safe rollout and rollback |
| 15 | 1 h | Three project stories with technical follow-ups | Deliver each in 2 min; answer implementation, tradeoff, test and metric questions |
| 16-17 | 2 h | Two timed coding sessions, including tests and review | Complete a fresh solution, explain correctness, fix one weakness |
| 18 | 1 h | Full technical mock using the script below | Score yourself; write the three most important corrections |
| 19 | 1 h | Repair the weakest area; repeat a failed problem from a blank editor | Solve independently without memorizing the old solution |
| 20 | 1 h | 30 min final verbal drill + 30 min final recall sheet | State key answers simply and consistently; stop studying |

**If time shrinks to 10 hours:** Java/coding 4 h; concurrency/Vert.x 1 h; Spring/REST 1 h; SQL/cache 1 h; security and project stories 1 h; timed mock and correction 2 h. Skip Priority C and full system design.

**If your baseline is weak:** borrow up to 2 hours from secondary technologies and broad production review for Java coding. Keep SQL, transaction correctness, authorization, and your project stories.

## How to use each one-hour block

Spend about 15 minutes learning, 30 minutes coding or answering aloud, and 15 minutes checking mistakes. For a concept, pass only when you can give its definition, a small example, and a limitation without reading. For code, pass only when you can explain correctness and test edge cases.

### Speaking practice method

Record an answer once. Listen only for three things: Did I answer the question in the first sentence? Did I give a concrete example? Did I stop after the important tradeoff? Repeat the answer in fewer words. Do not try to remove your accent.

## 3. Timed mock interviews and scoring

### Two coding sessions for hours 16-17

**Session 1, 60 min:** 5 min clarify an unfamiliar variation of sliding window or intervals; 30 min solve; 10 min tests/complexity; 15 min review and rewrite the faulty part. Example variation: find the longest substring containing at most K distinct characters.

**Session 2, 60 min:** 5 min clarify payment aggregation; 25 min implement; 10 min SQL latest-per-account query; 10 min race-condition or transaction debugging; 10 min review. Include mixed currencies and duplicate records in the discussion.

### Full technical mock for hour 18

| Minutes | Prompt | What a strong answer demonstrates |
|---|---|---|
| 0-5 | Introduce yourself and describe a recent backend contribution | Clear personal ownership and relevant technical depth |
| 5-12 | HashMap equality; volatile versus atomic increment; one stream operation | Correct mechanism and a small example |
| 12-37 | Solve merge intervals or a fresh hashing problem | Clarification, correct code, invariant, tests, complexity |
| 37-45 | SQL: latest successful payment per account | Correct filtering, partitioning, tie handling and tenant scope |
| 45-53 | A Vert.x API slows under load; retries create duplicate requests | Evidence-based diagnosis, event-loop care, safe retry semantics |
| 53-58 | Explain one project decision and a failure you handled | Truthful ownership, tradeoff, validation and learning |
| 58-60 | Ask the interviewer two questions | Interest in the team's actual engineering work |

### Score yourself out of 20

Give 0-4 points each for **coding correctness, complexity/reasoning, edge-case tests, technical accuracy, and clear communication/ownership**. A 0 means missing; 2 means partly correct with significant prompting; 4 means independent and precise. This is a personal practice rubric, not American Express's hiring rubric.

Target 15+ with no zero, then repair the weakest category. A confident wrong explanation is a bigger problem than a short pause followed by a correct answer.

# Resume Analysis

2 MAIN SECTIONS • PRINT-READY EDITION

Resume inputs reviewed: Sree.pdf (4 pages), Sree-Resume.pdf (3 pages), sree_resume-2.pdf (3 pages), and sree_resume-1.pdf (3 pages), from the archive directory you supplied. Resume outcomes are treated as claims to help you prepare, not independently verified facts. The guide intentionally omits UI preparation and reserves full system-design interviews for the next stage.

## 1. Resume Comparison and Priorities

| Resume | Useful preparation evidence | How to use it |
|---|---|---|
| Sree.pdf | Java/J2EE, Spring MVC/Data/Integration, Vert.x, SOAP/REST, DB2/Oracle/MySQL, Couchbase, EDI and batch migration | Use for technical history and integration examples. It has an older experience summary. |
| Sree-Resume.pdf | Java/Spring/Vert.x, Redis, IAM/MFA, certificate automation, deployment practices; Airflow and Spark in skills | Use for backend delivery and security stories. Confirm depth behind skills-only entries. |
| sree_resume-1.pdf | Most explicit backend detail: Vert.x/Java services, Spring Boot, Redis, Jaeger, Splunk, DR, Kubernetes/AWS, authorization | Use as the main study map. Verify each claim before repeating it. |
| sree_resume-2.pdf | Product strategy, payments, migration, operational outcomes | Use for business context. Add implementation details when answering engineering questions. |

## 2. Resume Differences to Resolve Before the Interview

- The summaries say 16+ or 18 years, while the work history starts in 2006. Confirm your own timeline. You can simply say, "I have extensive experience building enterprise payment and servicing platforms."
- The $500K result is described as cost avoidance in one version and annual savings in others. These are different claims. Identify which is accurate, the time period, and your contribution.
- A 30% onboarding result describes elapsed time in one place and manual effort in another. Identify the actual metric.
- A 40% improvement refers to debugging time, MTTR, or infrastructure savings in different entries. Keep each project and measurement separate.

- A 99.99% availability claim needs an observation period, service boundary, and measurement method. A successful DR drill alone does not prove that availability.
- AWS certification is marked **planned**. Do not present it as earned.
- For every technology, label your experience privately: **implemented myself / led or reviewed / conceptual familiarity**. These are all valid, but they support different answers.

# Interview Answer Templates

2 MAIN SECTIONS • PRINT-READY EDITION

## 1. Professional English for the Interview

Your goal is clarity, not a particular accent or complicated vocabulary. Speak slightly slower than normal, use short sentences, and pause between the definition, example and tradeoff.

### 1.1. A Reusable Technical-Answer Structure

> "[Concept] means [simple definition]. It is useful when [problem]. In a service, I would [mechanism]. The main tradeoff is [cost or limitation]. I would verify it by [test or measurement]."

Example: "Idempotency means repeating a request does not repeat its business effect. It is useful when a client retries after a timeout. I would use a tenant-scoped key and durable uniqueness checks. The main challenge is handling concurrent retries and uncertain downstream outcomes. I would test duplicate submissions and failures at each step."

### 1.2. Useful Phrases During Coding

- "Let me confirm the input constraints and expected output."
- "Can the input contain duplicates, nulls, or negative values?"
- "I'll start with a correct approach, then discuss the optimization."
- "The invariant is that the current window contains no duplicates."
- "This uses O (n) additional memory to reduce the lookup time."
- "Let me test an empty input and a repeated-value case."
- "I found an issue in the boundary condition. I'll correct it and trace the example again."

### 1.3. Useful Phrases During Technical Discussion

- "The key distinction is visibility versus atomicity."
- "That depends on the consistency requirement and the failure mode."
- "In our implementation, my specific responsibility was..."
- "The alternative was simpler, but it did not meet..."
- "I have not used that feature directly. My understanding is..."

- "I would verify the version-specific behavior before relying on it."
- "Could you clarify whether you mean concurrent threads within one instance or requests across multiple instances?"
- "Let me take a moment to organize the answer."

Avoid: "We used microservices for scalability" with no mechanism; "everything is thread-safe"; "there are no failures"; "exactly once" without a defined boundary; or a metric you cannot explain. Replace broad claims with one concrete implementation and one limitation.

### 1.4. Speaking Practice Method

Record an answer once. Listen only for three things: Did I answer the question in the first sentence? Did I give a concrete example? Did I stop after the important tradeoff? Repeat the answer in fewer words. Do not try to remove your accent.

## 2. Project Stories You Should Prepare

Use the same five-part structure: problem → your responsibility → implementation → tradeoff/failure handling → evidence and learning. Spend most of the time on what you personally did. Say "we" for team outcomes and "I" for your contribution.

### 2.1. Story A: Java/Vert.x User-Management APIs and Redis

**Professional opening to adapt:** "One relevant project was our commercial user-management platform. My role included [your exact backend responsibility]. The main challenge was [specific problem]. We used Java and Vert.x for the services, and I contributed to [specific implementation]. For frequently accessed data, we introduced Redis caching and measured [verified metric]. The main tradeoff was [freshness, complexity, or another actual concern], which we handled through [actual approach]."

**Prepare the details:** Trace one endpoint from authentication to response; identify the source of truth; name the client libraries and data model you actually used; explain worker versus event-loop behavior; define cache keys, invalidation and TTL policy; show how a permission change becomes effective; explain tests and the performance measurement.

**Questions to rehearse:** Why Vert.x? What exactly was cached? Why not cache every response? What happens when Redis is down? What was the baseline for 35%? What did you code yourself? How did you test tenant isolation?

### 2.2. Story B: IAM/MFA and Message-Signing Certificate Automation

**Professional opening to adapt:** "In the reconciliation platform, I worked on [actual security scope]. A recurring operational risk was [specific issue]. I implemented or led [accurate responsibility], with validation and monitoring around [actual steps]. The result was [verified outcome]. My main learning was that automation is complete only when failures are visible and recovery is reliable."

**Prepare the details:** Identify the identity provider and integration protocol you really used; explain role and resource checks; map certificate inventory to renewal and deployment; explain secure key handling, overlap, verification and rollback. Keep authentication work and certificate work distinct if they were separate projects.

**Questions to rehearse:** What happens if renewal fails? How do you know the new certificate is being used? What did the 90% effort reduction measure? How are privileges revoked? How are recovery and MFA bypass attempts handled?

## 2.3. Story C: Payment Integration or Production Improvement

**Professional opening to adapt:** "On a commercial payments project, we needed to [actual business requirement]. I was responsible for [specific technical scope]. We changed [old flow] to [new flow], while preserving [key invariant]. I addressed [failure case] through [actual mechanism]. We validated the change with [tests and rollout evidence] and achieved [verified result]."

**Prepare the details:** Explain file versus real-time tradeoffs, input validation, duplicate detection, partial failures, reconciliation, monitoring, rollout and operational ownership. If you select an incident instead, explain symptom, evidence, mitigation, root cause, fix and prevention.

**Questions to rehearse:** What if the request times out after processing? How did you prevent duplicates? How did you reconcile old and new flows? What made rollback safe? Was the $500K recurring savings or avoided cost?

# Professional Introduction

0 MAIN SECTIONS • PRINT-READY EDITION

I'm a senior engineer with extensive experience in enterprise payments, reconciliation, and client-servicing platforms.

At American Express, I've worked on Java and Vert.x-based backend services, RESTful API development and integration, identity and access management, UI development and operational automation.

I've also led platform modernization and reliability improvements, focusing on scalability, performance, code quality, and production stability.

Beyond my technical experience, I bring strong end-to-end delivery experience. I've worked closely with product owners to understand and prioritize business requirements, balancing business priorities with technical dependencies and opportunities to leverage ongoing work across teams.

My involvement spans the entire delivery lifecycle, from translating business requirements into technical solutions, hands-on development and testing, to guiding and mentoring engineers, coordinating with dependent teams, planning production rollouts, PIV's, and post-rollout support.

This experience has given me a strong understanding of how enterprise systems behave under real business constraints and what it takes to deliver reliable solutions in production.

For this role, I bring a combination of backend engineering experience, solution design, technical leadership, and delivery ownership, with a strong focus on code quality, reliability, and practical problem-solving.

# Coding Interview Companion

12 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

1. Your eight-hour coding path ..... 2
2. A repeatable live-coding method ..... 2
3. Recognize the pattern before you code ..... 4
4. Worked problem A: Two Sum ..... 5
5. Worked problem B: longest substring with at most K distinct characters ..... 6
6. Worked problem C: merge intervals ..... 7
7. Java coding tools to know by memory ..... 8
8. Coding workbook and practice ladder ..... 9
9. SQL workbook ..... 12
10. Two timed mock sessions ..... 14
11. Professional English while coding ..... 15
12. Reference files and how to study ..... 16

Integrated coding and SQL workbook

This is the coding section of your preparation in one place. It fits **inside** the 20 hours in the main guide: hours 1, 4-6, 16-17, and 19 provide seven dedicated coding hours. Use one hour of the Java study blocks (hours 2-3) for the mixed drill below; the hour-18 mock also includes coding. Use Java unless the interviewer asks for another language. These exercises are tailored practice, not confirmed interview questions from American Express.

**Your goal:** solve a correct problem in a way another engineer can follow. A good interview answer includes assumptions, a working approach, a short correctness argument, complexity, and tests. You do not need to speak continuously while typing; use short updates at decision points.

# 1. Your eight-hour coding path

| Block | Time | Work | Pass condition |
|---|---|---|---|
| Baseline | 1 h | Resume introduction, Two Sum from a blank editor, one SQL query, and review | State constraints, code, trace, test, and complexity |
| Core patterns A | 1 h | Hash maps, grouping, sets; problems 1 and 2 | Pick a key and explain its equality rules |
| Core patterns B | 1 h | Sliding window, two pointers; problems 3 and 4 | State the invariant and move pointers correctly |
| Core patterns C | 1 h | Sorting/intervals and binary search; problems 5 and 6 | Handle ties, boundaries, and empty input |
| Mock 1 | 1 h | Unseen problem: longest substring with at most K distinct characters | Solve in 35 min; test and explain in 10 min |
| Mock 2 | 1 h | Payment totals by account and currency, then one SQL query | Preserve money and tenant rules; finish cleanly |
| Mixed drill | 1 h | Use part of Java study hours 2-3 for stack, heap, BFS, and a debugging prompt | Recognize correct data structure within 2 min |
| Repair | 1 h | Repeat weakest problem from a blank editor, then 15-min oral drill | Solve without looking; state two things learned |

If you are short of time, do problems 1-6 plus the two mocks. Problems 7-12 are valuable after the essentials. Keep breaks outside these focused hours. The final hour before the interview should be gentle recall rather than a new hard problem.

# 2. A repeatable live-coding method

## Step 1: Clarify the contract (2-3 minutes)

Ask what the input can contain and what the output must mean. Clarify null or empty input, duplicates, sortedness, negative values, integer overflow, case sensitivity, Unicode, and tie behavior **only when relevant**. If the interviewer leaves a choice open, state your assumption and proceed.

**Say:** "Let me confirm the output and a couple of edge cases. Then I'll walk through an example before coding."

Example for Two Sum: "Should I return indices or values? Can the same number occur twice? I assume I must use two different indices. What should I return if there is no pair?"

## Step 2: Trace a small example (2 minutes)

Choose an example that reveals the rule, not just the happy path. For Two Sum, `[3, 3]` with target 6 reveals whether two identical values at distinct indices are allowed. For interval merging, `[1,10], [2,3], [10,12]` reveals nesting and touching endpoints.

**Say:** "On this example, the expected result is ... because ..."

## Step 3: Choose a simple approach and improve it (2-4 minutes)

Name the straightforward solution first when it helps. State why a data structure improves it. Do not recite every possible algorithm.

**Say:** "A nested scan would take O ($n^2$) time. A map of values already seen lets me check each complement once, so I expect O (n) time and O (n) extra space."

## Step 4: State the invariant (1 minute)

An invariant is a rule that stays true after every loop iteration. It is the core of a correctness explanation. Say it before a complicated loop.

- Hash map: "The map contains only elements before the current index."
- Sliding window: "The current window contains at most K distinct characters."
- Binary search: "The answer, if it exists, remains in the search range, or I have already saved a candidate."
- Merge intervals: "The result contains merged intervals for everything processed so far."

## Step 5: Code in small, complete pieces (15-25 minutes)

Write a method signature, initialize the structures, write the main loop, return the result. Use clear names such as `left`, `right`, `counts`, `lastSeen`, and `merged`. Do not add a framework, helper classes, or streams when a small loop is clearer. In Java, choose `ArrayDeque` for stack/queue practice and a suitable `Map` or `Set` for lookup.

**Say briefly while coding:** "I am checking the complement before storing the current item so I cannot use the same index twice."

## Step 6: Trace and test (5-8 minutes)

Run the small example through the code. Then use at least one boundary case and one adversarial case. Check whether the input is mutated. If you find a bug, correct it openly and test again.

**Say:** "I'll test empty input, duplicate values, and the boundary where the answer is at the end. I found a boundary issue; I'll fix it and retrace."

## Step 7: Close the answer (1-2 minutes)

Give time and extra-space complexity, mention the dominant operation, and state one important limitation. If you used sorting, include sorting cost. If you created an output array, say whether your space figure includes output.

**Say:** "The scan is O (n) expected time because each hash lookup is expected constant time. The map uses O (n) additional space. A hash collision does not change correctness, although worst-case behavior can differ."

# 3. Recognize the pattern before you code

| Clue in prompt | Pattern | Main idea | Common mistake |
|---|---|---|---|
| Find pair or repeated key | Hash map/set | Record what was seen | Use same index twice; forget key equality |
| Longest/shortest contiguous range | Sliding window | Maintain a valid window | Move left backward; fail to shrink fully |
| Sorted array and first/last match | Binary search | Cut search space in half | Overflow in midpoint; wrong loop bounds |
| Overlapping time or number ranges | Sort + sweep | Merge with last interval | Replace end instead of taking max |
| Matching brackets or nested syntax | Stack | Last opened is first closed | Ignore extra closing bracket |
| Top K or next minimum/ maximum | Heap | Keep only useful candidates | Wrong heap ordering or tie behavior |
| Levels or shortest unweighted path | BFS queue | Visit distance layer by layer | Mark visited too late; revisit nodes |
| All paths or recursive structure | DFS | Explore one branch at a time | Missing base case; shared mutable state |
| Number of subarrays with target sum | Prefix sum + map | Count prior required prefix values | Initialize count of zero prefix incorrectly |
| Repeated subproblems with choices | Dynamic programming | Store answers to smaller states | State not defined before recurrence |

An interviewer may use a payment or account story around a standard pattern. Translate the business words into the data structure problem. Also preserve business rules: group money by currency, scope accounts by tenant, and do not treat a local cache as a durable deduplication mechanism.

## Complexity you should say without hesitation

| Operation | Typical time | Important qualifier |
|---|---|---|
| Traverse n items | O(n) | Nested traversals can be $O(n^2)$ |
| Sort n items | O(n log n) | Include sorting in a merge-interval solution |

| Operation | Typical time | Important qualifier |
|---|---|---|
| HashMap get/put | Expected O(1) | Hashing/equality costs and worst cases matter |
| TreeMap get/put | O(log n) | Maintains sorted keys |
| Binary search | O(log n) | Input must already be sorted appropriately |
| Heap offer/poll | O(log n) | Peek is O(1); build and final sorting have costs |
| BFS/DFS | O(V + E) | Plus visited storage for a graph |
| Sliding window | Usually O(n) | Each pointer moves at most n times |

For a loop with one pointer moving forward and another pointer that never moves backward, do not assume O($n^2$) simply because the loops are nested. Count the total pointer movements.

# 4. Worked problem A: Two Sum

**Contract:** Given `int[] values` and `int target`, return two distinct indices whose values add to target. Return an empty array if none exists. Null input is invalid. If more than one pair exists, the first found by scanning left to right is acceptable.

**Walkthrough:** Input `[2, 7, 11, 15]`, target 9. At 2, seek 7; it is not in the map, so store 2 → 0. At 7, seek 2; find index 0, so return `[0,1]`.

**Code:**

JAVA

```java
static int[] twoSum(int[] values, int target) {
    Objects.requireNonNull(values);
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < values.length; i++) {
        long need = (long) target - values[i];
        if (need >= Integer.MIN_VALUE && need <= Integer.MAX_VALUE) {
            Integer previous = seen.get((int) need);
            if (previous != null) return new int[]{previous, i};
        }
        seen.put(values[i], i);
    }
    return new int[0];
}
```

**Why it works:** Before index `i`, the map contains only earlier indices. A found complement therefore gives a different index. If no complement is found, adding the current value makes it available to later elements. Every possible earlier partner is checked when the later partner arrives.

**Tests:** `[3,3]`, target 6 → `[0,1]`; `[]` → `[]`; `[1,2]`, target 9 → `[]`; `[-5,2,7]`, target 2 → `[0,2]`; use an overflow case such as two `Integer.MAX_VALUE` values with target `-2` and expect no false match.

**Cost:** O (n) expected time; O (n) extra space. The `long` calculation prevents integer wraparound from creating a false complement.

**Professional explanation:** "I store earlier values in a map from value to index. At each position I first check whether the value needed to reach the target is already present. Checking before insertion ensures I use two distinct indices. I also calculate the complement in a wider type to avoid integer overflow."

## 5. Worked problem B: longest substring with at most K distinct characters

**Contract:** Given a nonnull Java `String` and integer `k >= 0`, return the length of its longest contiguous substring containing at most K distinct UTF-16 `char` values. This is a practice problem; clarify Unicode handling in a real interview.

**Walkthrough:** For `"eceba"`, `k = 2`, the best window is `"ece"` with length 3. When `b` enters, three distinct letters are present. Move `left` until the window again has at most two distinct letters.

**Code:**

JAVA

```java
static int longestAtMostKDistinct(String s, int k) {
    Objects.requireNonNull(s);
    if (k < 0) throw new IllegalArgumentException("k must be nonnegative");
    Map<Character, Integer> counts = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char added = s.charAt(right);
        counts.merge(added, 1, Integer::sum);
        while (counts.size() > k) {
            char removed = s.charAt(left++);
            int remaining = counts.get(removed) - 1;
            if (remaining == 0) counts.remove(removed);
            else counts.put(removed, remaining);
        }
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

**Why it works:** The counts describe exactly the current window. If adding a character makes the window invalid, moving `left` removes characters until it becomes valid. At each `right`, the resulting window is the longest valid window ending there. Taking the maximum across all right endpoints finds the answer.

**Tests:** `"eceba", 2 -> 3`; `"aa", 1 -> 2`; `"abc", 0 -> 0`; `"", 3 -> 0`; `"abc", 5 -> 3`; negative k → exception. For full Unicode code points, use a code-point based representation instead of `char` indexing.

**Cost:** O (n) time because each pointer moves forward at most n times; O (min (n, character set size)) extra space.

**Professional explanation:** "I maintain a window and a count for each character inside it. I expand the right side and shrink the left side only when we exceed K distinct characters. Both pointers move forward, so the total work is linear."

## 6. Worked problem C: merge intervals

**Contract:** Merge overlapping **closed** intervals `[start,end]`. Touching intervals such as `[1,2]` and `[2,3]` overlap. Reject malformed intervals. Return new arrays rather than modifying the input.

**Walkthrough:** Sort `[8,10], [1,3], [2,6], [15,18]` to `[1,3], [2,6], [8,10], [15,18]`. Merge the first two into `[1,6]`, then start new intervals for the rest.

**Core loop after validating and sorting:**

JAVA

```
List<int[]> merged = new ArrayList<>();
for(
int[] next :sorted){
        if(merged.

isEmpty() ||next[0]>merged.

get(merged.size() -1)[1]){
        merged.

add(next.clone());
        }else{
int[] current = merged.get(merged.size() - 1);
current[1]=Math.

max(current[1], next[1]);
    }
            }
```

**Why it works:** Sorting means no future interval can start earlier than the current one. The last merged interval either overlaps `next` or finishes before it. Taking the maximum end handles nested intervals such as `[1,10]` followed by `[2,3]`.

**Tests:** Empty input; single interval; unsorted input; nesting; touching endpoints; invalid start greater than end. State whether time intervals are closed or half-open, because that changes the overlap test.

**Cost:** O (n log n) time for sorting; O (n) additional space when copying input and constructing output.

**Professional explanation:** "I sort by start time, then scan once. When the next interval overlaps the last merged interval, I extend its end to the larger endpoint. Otherwise I begin a new merged interval. The sort dominates the running time."

The source guides refer to **InterviewPractice.java** for runnable implementations of the ten core exercises. Complete each exercise before consulting that answer key.

## 7. Java coding tools to know by memory

| Need | Java choice | One thing to remember |
|---|---|---|
| Fast lookup | `HashMap<K,V>`, `HashSet<T>` | Correct equals/hashCode for custom keys |
| Preserve insertion order | `LinkedHashMap<K,V>` | Insertion order is different from access order |
| Sorted keys | `TreeMap<K,V>` | O(log n) operations |
| Stack or queue | `ArrayDeque<T>` | `push/pop`, `add/remove`; no null elements |
| Top K / next item | `PriorityQueue<T>` | Smallest by comparator is at head |
| Text construction | `StringBuilder` | Build in a loop, convert once |
| Sorting objects | `Comparator.comparing(...)` | Use `Integer.compare`, not subtraction, for integer comparison |
| Exact decimal money | `BigDecimal` | Construct from a decimal string and specify rounding where needed |
| Immutable list result | `List.copyOf` on supported JDKs | Check supported Java version first |

**Handy Java skeleton:**

JAVA

```java
static int solve(int[] input) {
    Objects.requireNonNull(input);
    // State or data structure.
    // Loop invariant.
    for (int value : input) {
        // Update state.
    }
    return 0; // Replace with the required result.
}
```

In a coding interview, imports can usually be stated or written once: `import java.util.*;`. Avoid hiding the entire solution in a stream if you cannot explain its behavior and complexity. Mention if the solution relies on Java 8, 17, or 21 features.

### Java pitfalls that commonly break otherwise sound algorithms

- `==` on objects compares references; use `equals` for logical value equality.
- A custom map key needs compatible `equals` and `hashCode`, and its key fields should remain stable after insertion.
- `int` arithmetic can wrap. Use `long` when summing or subtracting potentially large integers.
- A comparator based on `a - b` can overflow. Use `Integer.compare(a,b)`.
- `ArrayList.remove(int)` removes by index; `remove(Object)` removes by value.
- `PriorityQueue` iteration is not sorted; repeated `poll()` gives priority order.
- `String.charAt` returns a UTF-16 code unit. Clarify whether the prompt needs Unicode code points or human-visible characters.
- `BigDecimal.equals` considers scale; `compareTo` compares numeric value. Keep currency in the grouping key.
- `Map.get(key) == null` cannot distinguish missing key from a stored null; use `containsKey` if null is allowed.
- A thread-safe collection does not make a sequence of business actions atomic.

## 8. Coding workbook and practice ladder

Use a blank editor. Before coding, state input assumptions, output, an example, a simple approach, and a better approach. Then write code, trace it, test boundaries, and state complexity. If stuck after 8 minutes, read only the hint. After reviewing a solution, repeat from a blank editor later.

The companion file **InterviewPractice.java** contains reference implementations for all ten exercises and executable checks. Try the questions before reading it. The examples use Java 8-compatible syntax; their comments define assumptions such as lowercase words, UTF-16 characters, interval boundaries, and already-deduplicated payment input. Run `javac InterviewPractice.java` followed by `java InterviewPractice` in a folder containing the file and a configured JDK.

### Core exercises: problems 1–10

| # | Exercise | Time | Expected approach and complexity | Essential tests |
|---|---|---|---|---|
| 1 | Two Sum: return two different indices whose values sum to a target | 20 min | Hash map of seen values; expected O(n) time/O(n) space | Duplicate values, negative values, no match, fewer than 2 items, integer overflow in complement |
| 2 | Group anagrams of lowercase English words | 25 min | Sorted key: O(total word-sorting cost), or 26-count signature: O(total characters + 26n) | Empty input, empty word, duplicates, clarify case and character set |
| 3 | Longest substring without repeated characters | 30 min | Sliding window with last-seen positions; O(n) time | Empty, all identical, repeated pattern, define UTF-16 units versus Unicode code points |

| # | Exercise | Time | Expected approach and complexity | Essential tests |
|---|---|---|---|---|
| 4 | Merge overlapping closed intervals | 30 min | Sort by start then scan; O(n log n) time | Nested intervals, touching endpoints, unsorted input, empty input, invalid intervals |
| 5 | First occurrence of a target in a sorted array | 20 min | Binary search with candidate answer; O(log n) time/O(1) space | Repeated target, absent target, empty array, first and last positions |
| 6 | Validate balanced brackets | 20 min | Stack using ArrayDeque; O(n) time/O(n) space | Empty string, wrong nesting, early close, leftover opens, non-bracket characters policy |
| 7 | Top K frequent payment categories | 30 min | Count map plus heap of size k; O(n + u log k), u unique categories | k=0, k larger than u, ties with deterministic rule |
| 8 | Binary-tree level-order traversal | 25 min | Breadth-first traversal with queue; O(n) time, O(width) auxiliary space; capture the queue size per level | Null root, single node, skewed tree, balanced tree |
| 9 | LRU cache get/put in O(1) | 35 min | Hash map + doubly linked list; or access-order LinkedHashMap if allowed | Capacity 1, update existing key, eviction order, invalid capacity |
| 10 | Aggregate successful payments by account and currency | 25 min | Loop or streams with BigDecimal; O(n) expected grouping time | Failed/pending excluded, multiple currencies, scale, null policy, duplicate event policy |

**Core pass:** exercises 1-6 and 10. Use exercises 7-9 for later practice or if you finish early. Do not spend three hours on advanced dynamic programming while basic Java implementation remains uncertain.

## Additional exercises: problems 11–14

| # | Additional exercise | Time | Checkpoint |
|---|---|---|---|
| 11 | Count subarrays whose sum equals K | 35 min | Prefix sum counts; works with negative values |
| 12 | Validate a binary search tree | 30 min | Propagate lower and upper bounds |
| 13 | First nonrepeating character | 20 min | Count, then scan in original order |
| 14 | Shortest path in an unweighted graph | 35 min | BFS; record visited when enqueueing |

## Hints and small proof checks for 11-14

**11. Count subarrays with sum K:** Let `prefix` be the sum before or at the current position. A prior prefix equal to `prefix - k` forms a subarray of sum k. Start the frequency map with `{0:1}` so subarrays starting at index 0 count. Increment the answer by the number of prior required prefixes, then record the current prefix. Expected O (n) time and O (n) space. Sliding window is unreliable when negative values are allowed.

**12. Validate BST:** Each node needs a range inherited from all ancestors, not just a comparison with its immediate parent. Recurse with exclusive lower and upper bounds; use `long` bounds for `int` node values. A left descendant greater than the root invalidates the tree even if it is less than its direct parent. O (n) time and O (h) stack space for height h.

**13. First nonrepeating character:** Count all character occurrences, then scan the original string and return the first whose count is one. Clarify what to return if none exists. O (n) time and O (alphabet) map space.

**14. Shortest unweighted path:** BFS visits nodes by increasing number of edges from the start. Mark a node visited when placing it in the queue, otherwise it may be enqueued many times. Store parent links if you need the path itself; store distances if you only need length. O (V + E) time and O (V) space. Directed versus undirected edges must be clear.

## Correctness explanations to practice

**Two Sum:** "Before each iteration, the map contains indices from earlier elements. I look for the required complement before inserting the current value, so I never use the same index twice."

**Sliding window:** "The current window contains no repeated character. When a repeat appears inside the window, I move the left boundary past its previous occurrence. The left boundary never moves backward."

**Merge intervals:** "After sorting by start time, the next interval either overlaps the last merged interval or starts a new one. I extend the end using the maximum end so nested intervals remain correct."

**Binary search:** "I preserve the range that can still contain the first occurrence. When I find the target, I save its position and continue searching left."

**LRU:** "The map gives direct lookup, and the list maintains recency. Every successful access moves that entry to the most recent position. Eviction removes the least recent entry from both structures."

## Exercise 10: payment aggregation contract

Assume each record contains accountId, currency, amount and status. Group only SUCCESS records by **account and currency**, because adding different currencies without conversion is meaningless. Decide whether input records are already deduplicated; do not silently assume transaction IDs are unique. Use BigDecimal addition and an explicit null policy. If the interviewer asks for parallel processing, explain associativity and thread-safe accumulation before changing the implementation.

## Payment-flavored coding prompt for a senior engineer

"Given payment events with tenant, account, currency, amount, status, transaction ID, and timestamp, return successful totals by tenant/account/currency. Duplicate transaction IDs must be counted once."

**Clarify:** Is transaction ID unique globally or only within a tenant? Can duplicate events have different amounts? Does a reversal have a separate ID or update the original? Is input ordered? What should happen with malformed records?

**Good in-memory approach:** Use a set keyed by tenant and transaction ID to accept only the first valid event, then a map keyed by tenant/account/currency with `BigDecimal` totals. This assumes the first event for a duplicate ID is authoritative. If conflicting duplicates are possible, reject or reconcile them under a stated policy. Expected O (n) map work and O (n) space. If this is a production service across instances, a durable unique constraint or provider-level idempotency mechanism is also needed.

**Say:** "I am keeping the tenant in both the deduplication key and the total key. I would not combine USD and EUR without a conversion rule. For an interview exercise, an in-memory set is enough; for a payment service, I would require durable coordination."

### Three useful code-review drills

**Drill A - race:** `if (!map.containsKey(key)) { map.put(key, value); }` is not atomic merely because map is concurrent. For an in-memory per-key insert, use putIfAbsent. For a cross-instance business guarantee, use durable coordination such as a unique database constraint.

**Drill B - event loop:** A Vert.x handler calls a synchronous JDBC query, then `future.join()`. Both can block. Explain asynchronous composition or bounded worker execution, and how you would test event-loop responsiveness under load.

**Drill C - transaction:** A method calls another @Transactional method through `this`, catches its exception, and returns success. Check proxy bypass, the actual transaction boundary, rollback rules and error propagation. Do not fix it by adding more annotations without understanding the call path.

## 9. SQL workbook

Assume `payments(id, tenant_id, account_id, currency, amount, status, created_at)` and `accounts(id, tenant_id, name)`. The exercises assume `(tenant_id, id)` identifies an account, amounts use a decimal type, and timestamps are comparable. Adapt timestamp and pagination syntax to the interview database.

### Q1. Successful totals by account and currency

Write the query first, then compare:

SQL

```sql
SELECT tenant_id,
       account_id,
       currency,
       COUNT(*)    AS payment_count,
       SUM(amount) AS total_amount
FROM payments
WHERE status = 'SUCCESS'
GROUP BY tenant_id, account_id, currency;
```

**Explain:** WHERE filters individual rows before grouping. HAVING filters grouped results. Do not combine currencies or omit tenant boundaries. Clarify how refunds and reversals appear in the data.

## Q2. Accounts with no successful payment

SQL

```sql
SELECT a.tenant_id, a.id, a.name
FROM accounts a
WHERE NOT EXISTS (SELECT 1
                  FROM payments p
                  WHERE p.tenant_id = a.tenant_id
                    AND p.account_id = a.id
                    AND p.status = 'SUCCESS');
```

**Explain:** NOT EXISTS expresses the absence of a matching row and avoids the NULL trap of NOT IN when the subquery can return NULL. An account with failed payments only should still appear.

## Q3. Latest payment per account

SQL

```sql
WITH ranked AS (SELECT p.*,
                       ROW_NUMBER() OVER (
           PARTITION BY tenant_id, account_id
           ORDER BY created_at DESC, id DESC
         ) AS rn
                FROM payments p)
SELECT id, tenant_id, account_id, currency, amount, status, created_at
FROM ranked
WHERE rn = 1;
```

**Explain:** The extra id ordering makes timestamp ties deterministic. Decide whether the requirement means latest payment of any status or latest successful payment; filter at the appropriate stage. ROW_NUMBER chooses one row; DENSE_RANK can keep ties.

## Q4. Detect duplicate business requests

Assume an additional `idempotency_key` column:

SQL

```sql
SELECT tenant_id, idempotency_key, COUNT(*) AS occurrences
FROM payments
WHERE idempotency_key IS NOT NULL
GROUP BY tenant_id, idempotency_key
HAVING COUNT(*) > 1;
```

**Explain:** This detects existing duplicates; it does not prevent concurrent insertion. Prevention requires an appropriate unique constraint and transaction handling. Null uniqueness behavior differs by database, so define whether keys are required.

### Q5. Prevent a lost update

Assume `account_balances(tenant_id, account_id, balance, version)`:

```sql
UPDATE account_balances
SET balance = balance - :amount,
    version = version + 1
WHERE tenant_id = :tenant
  AND account_id = :account
  AND version = :expected_version
  AND balance >= :amount;
```

**Explain:** Require a positive amount, check the affected-row count, and place related ledger writes in the same appropriate transaction. Zero updated rows can mean stale version, insufficient balance, or missing account; resolve deliberately. This statement alone is not a complete payment or transfer implementation.

## 10. Two timed mock sessions

### Mock A: algorithm, 45 minutes

**Prompt:** "Find the longest substring with at most K distinct characters." Use the worked example only **after** your first attempt.

| Minute | Action |
|---|---|
| 0-4 | Clarify K, null, empty string, and character model |
| 4-7 | Trace `"eceba"`, K=2 and `"abc"`, K=0 |
| 7-11 | State sliding-window invariant and complexity |
| 11-30 | Code without looking at a solution |
| 30-38 | Trace manually and test edge cases |
| 38-45 | Explain correctness, cost, and one limitation |

### Mock B: backend coding, 45 minutes

**Prompt:** "Aggregate successful payment records by account and currency, handling duplicate transaction IDs."

| Minute | Action |
| --- | --- |
| 0-5 | Clarify tenant, duplicate conflict, amounts, status and return format |
| 5-9 | Work through six records including a duplicate, failure and two currencies |
| 9-29 | Write the solution with clear key types |
| 29-37 | Test duplicate conflict, empty input, decimal precision and currency split |
| 37-45 | Explain correctness, complexity and production consistency boundary |

Use a real timer. Do not pause it to read the answer. After the mock, record what actually slowed you down: recognizing the pattern, Java syntax, edge cases, or explanation. Repair that specific weakness in study hour 19.

### A five-point scorecard

Score 0-2 for each item: **contract clarity, suitable algorithm, correct implementation, meaningful tests, and clear explanation**. A score of 2 means you did it independently and accurately; 1 means you needed a prompt or missed a small case; 0 means it was absent or incorrect. A total of 8/10 with no zero is a useful practice target, not a hiring score.

## 11. Professional English while coding

Use the structure **decision → reason → check**. Speak in short sentences. A pause to think is normal.

**Opening:** "I'll clarify the input and output, trace one example, and then choose a data structure."

**After choosing an approach:** "A map lets me look up the value I need while scanning once. I am trading extra memory for lower expected time."

**Before a complicated loop:** "The invariant is that the current window has no more than K distinct characters. If it becomes invalid, I move the left boundary until it is valid again."

**When you spot an error:** "I see that this fails for a duplicate at the current index. I'll change the order so I check before inserting, then retest that case."

**When uncertain:** "I have not used that exact API recently. I know the algorithm and can write the simple loop, then verify the method name if needed."

**Closing:** "This is O (n) time because each element enters and leaves the window at most once. The frequency map uses space proportional to the number of distinct characters in the window."

Avoid memorizing these as fixed scripts. Practice saying the same idea in your own clear words. When you do not know an edge-case policy, ask or state an assumption. If you cannot finish, explain the partial algorithm, the missing step, and how you would test it.

### Final 10-minute oral drill

1. Why does checking the map before insertion matter in Two Sum?
2. What is the invariant in a sliding-window solution?
3. Why is merge intervals O (n log n)?
4. When does binary search apply, and what is the first-occurrence change?
5. How do you test a heap comparator with ties?
6. Why can `int` arithmetic give the wrong answer?
7. What data structures give O (1) LRU operations?
8. Why must financial totals be grouped by currency?
9. What does an in-memory duplicate set fail to guarantee across services?
10. Which one problem will you repeat from a blank editor before the interview?

## 12. Reference files and how to study

The original companion refers to **InterviewPractice.java** as the runnable answer key with ten core implementations and 38 executable checks. It also refers to **interview-preparation.pdf** for the 20-hour schedule and broader backend concepts. Neither reference file was included in the two Markdown inputs to this merge. Try each exercise before comparing your approach, complexity, edge handling, and Java details with an answer key. This companion fits inside the existing coding study blocks; it does not add eight hours to the schedule.

Use the language in this guide to explain your own reasoning. The target is a correct solution you can defend and adapt when the interviewer changes the prompt.

# Design principles, patterns, and backend architecture

9 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

1. How to study this guide ..... 2
2. Design principles you should know ..... 2
3. Object-level design patterns ..... 12
4. Architecture patterns and when to choose them ..... 23
5. Worked design: submit a commercial payment ..... 34
6. Fast comparisons interviewers ask ..... 37
7. Practice questions and model answers ..... 40
8. Four worked demonstrations: prove the choice, not just the name ..... 42
9. Final one-page recall ..... 45

**Senior Software Engineer interview study guide | October 22, 2026**

**Expanded edition (October 23, 2026):** Each topic now includes a practice case, a reason for the choice, and evidence or a counterexample. All new scenarios are hypothetical.

This guide builds on your technical and coding preparation. It uses commercial payments, reconciliation, identity, and user-management examples because those appear in your resumes. The examples are **practice designs**, not descriptions of American Express systems. State what you actually implemented when discussing your own work.

**Code examples:** The code anchors are intentionally short teaching fragments, not complete applications. Some use illustrative types (such as `Money`, `Payment`, or repositories) and omit imports, wiring, error mapping, and security. Some callback/lifecycle sketches are marked pseudocode. They demonstrate responsibility and failure behavior; they are **not drop-in production payment code**. Follow each with its verification question.

**Print layout:** Long comparisons and failure matrices use vertical blocks rather than wide tables. All headings, text, and code target US Letter portrait PDF output.

# 1. How to study this guide

**Design principle** = a rule for making choices, such as keeping responsibilities clear. **Design pattern** = a reusable way to organize classes or objects for a recurring problem. **Architecture pattern** = a way to organize larger components and their communication. An implementation may use several patterns, but every pattern should solve a named problem.

In the upcoming technical round, prepare to explain a design choice while coding or discussing a project. For the later system-design round, practice drawing boundaries, data flow, failure paths, and tradeoffs. Do not spend this week memorizing a pattern catalog. Learn the problem, the mechanism, the cost, and when to avoid the pattern.

## Fit into the existing 20-hour plan

**During Java fundamentals — 35 minutes.** Study SOLID, composition, immutability, Strategy, Factory, Builder. Checkpoint: Refactor one conditional-heavy example aloud.

**During Spring and API study — 35 minutes.** Study layering, Adapter, Repository, dependency inversion, transactions. Checkpoint: Trace a request and name each responsibility.

**During project-story practice — 30 minutes.** Connect one real decision to its tradeoff and test. Checkpoint: Give a two-minute design story with personal ownership.

**During production review — 35 minutes.** Study outbox, retry, circuit breaker, bulkhead, idempotency. Checkpoint: Explain timeout and duplicate-event behavior.

**During a mock follow-up — 15 minutes.** Practice selecting and critiquing patterns. Checkpoint: Answer without buzzwords.

That is **2 hours 30 minutes** inside your 20-hour plan. For the later system-design round, revisit the architecture section more deeply. If the interviewer asks about a technology you used, give its actual implementation details before naming a pattern.

Use this answer structure: **"The problem was __. I chose __ because __. The cost was __. I handled failure by __. We verified it with __."** Speaking the tradeoff is what makes an answer sound senior.

# 2. Design principles you should know

## 1. Single responsibility principle (SRP) [High]

**Plain English:** A component should have one coherent reason to change. This does not mean it must have only one method. A payment class that validates requests, calculates fees, saves records, and sends email may mix unrelated changes.

**Professional answer:** "I group behavior around one responsibility and keep boundaries clear. For example, payment policy, persistence, and notification change for different reasons, so I separate them when that separation makes changes and tests safer."

**When to use and cost:** Split when changes repeatedly touch unrelated logic or tests become difficult. Too many tiny classes can make a simple flow hard to follow. Ask: "What business decision belongs here, and who owns it?"

**Demonstration — a change request:** A `PaymentService` both computes fees and sends approval emails. Finance asks for a new fee rule; Support asks for a different email template. Editing and retesting the same service for unrelated requests indicates two reasons to change. Move fee decisions to `FeePolicy` and notification delivery to `NotificationSender`; keep `SubmitPayment` responsible for coordinating the use case.

**Why SRP applies:** The split follows separate business owners and change cadence, not a rule that every method needs a class. **Evidence:** A fee-policy unit test no longer needs a mail client; a notification failure does not change the fee calculation. If neither behavior changes separately, the split might be premature.

**Code anchor (illustrative fragment):**

JAVA

```java
final class SubmitPayment {
    private final FeePolicy fees;
    private final PaymentStore store;

    void accept(Payment p) {
        Money fee = fees.calculate(p);
        store.save(p, fee);
    }
}
// Fee policy and persistence change separately.
```

**Verify it:** Change the fee formula: only the fee-policy tests should change. A mail client should never be required for this unit test.

## 2. Open/closed principle (OCP) [High]

**Plain English:** Add a new variation with limited changes to stable, existing code. If every new payment method forces edits to one large switch, the design may need an extension point.

**Professional answer:** "I use an extension point when variations are real and growing. A PaymentMethod interface can let a new method be added without repeatedly changing the orchestration. I still avoid abstractions for speculative future features."

**Trap:** A change in business policy may legitimately change existing code. OCP is a design direction, not a ban on editing a class.

**Demonstration — growing payment channels:** Initially `submit()` contains `if (type == CARD) ... else if (type == ACH) ...`. When a third supported channel arrives, model `PaymentHandler` with implementations for each channel and register supported handlers. The stable `SubmissionService` asks a resolver for the matching handler and invokes `submit()`.

**Why OCP applies:** New channel behavior is added mainly through a new implementation, rather than editing the core processing algorithm. **Evidence:** Existing channel tests pass unchanged and a new channel gets contract tests. The resolver or its configuration may still change; OCP does not promise literally zero edits.

**Code anchor (illustrative fragment):**

JAVA

```java
interface PaymentHandler {
    PaymentRail rail();

    void submit(Payment payment);
}

final class HandlerResolver {
    private final Map<PaymentRail, PaymentHandler> handlers;

    PaymentHandler forRail(PaymentRail rail) {
        return Objects.requireNonNull(handlers.get(rail));
    }
}
// Add another PaymentHandler implementation for a new rail.
```

**Verify it:** Register an additional rail and run the same handler contract tests without editing submission orchestration. Registration still needs a supported-rail entry.

## 3. Liskov substitution principle (LSP) [High]

**Plain English:** If B is used wherever A is expected, B should honor A's promises. A subclass that throws for a supported parent operation or silently changes its meaning breaks that promise.

**Professional answer:** "I design subtypes around behavioral contracts, not only matching method signatures. A replacement should preserve the expectations callers depend on, including error behavior and business invariants."

**Example:** A read-only account should not extend a mutable account contract with `withdraw()` that always throws. Separate capabilities or use composition. Test the same contract against each implementation.

**Demonstration — broken substitution:** A `MutableAccount.withdraw(amount)` contract promises to withdraw a valid amount or return a documented insufficient-funds result. `ReadOnlyAccount extends MutableAccount` instead throws `UnsupportedOperationException` for every withdrawal. A caller that was correct for `MutableAccount` now fails unexpectedly.

**Better design:** Give `AccountView` a `balance()` operation; make `WithdrawableAccount` a separate capability. A read-only account implements only `AccountView`. **Why LSP applies:** Each implementation can honor every operation it advertises. **Evidence:** Run shared contract tests against all `WithdrawableAccount` implementations, including boundary/error behavior.

**Code anchor (illustrative fragment):**

JAVA

```java
interface AccountReader {
    Balance balance(AccountId id);
}

interface AccountWriter {
    Receipt withdraw(AccountId id, Money amount);
}

final class ReadOnlyAccount implements AccountReader {
    public Balance balance(AccountId id) { /* ... */ }
}
// Read-only accounts never promise withdraw().
```

**Verify it:** Run the same read-contract tests for every AccountReader. Do not fake support for withdrawals by throwing UnsupportedOperationException.

## 4. Interface segregation principle (ISP) [Medium]

**Plain English:** A caller should depend on only the operations it needs. One giant financial-platform interface makes unrelated clients know too much.

**Professional answer:** "I prefer focused interfaces based on consumers. A reconciliation reader should not need payment-submission or user-administration methods. This makes dependencies and permissions easier to reason about."

**Cost:** Too many one-method interfaces can create noise. Split where consumers genuinely need different contracts.

**Demonstration — unnecessary authority:** A report job only needs `listSettledPayments()`, but is given a `FinancialPlatform` interface containing `submitPayment()`, `deleteUser()`, and `updateLimit()`. Even if unused, those operations enlarge the apparent dependency and may complicate permissions and mocks.

**Better design:** Inject `SettlementReader` into the report job; give payment submission a separate `PaymentSubmitter` interface. **Why ISP applies:** Each consumer sees its own legitimate operations. **Evidence:** The reporting test needs only a read fake and the report job cannot call a write operation through its dependency.

**Code anchor (illustrative fragment):**

JAVA

```
interface ReconciliationReader {
    List<Mismatch> findMismatches(BatchId batch);
}

interface PaymentSubmitter {
    Receipt submit(Payment payment);
}

final class ReportService {
    ReportService(ReconciliationReader reader) { /* ... */ }
}
// Reporting cannot accidentally submit payments.
```

**Verify it:** Compile the reporting module against only ReconciliationReader and confirm it cannot call payment submission methods.

## 5. Dependency inversion principle (DIP) [High]

**Plain English:** Business logic should depend on an appropriate contract rather than on a specific database, HTTP client, or vendor SDK. An adapter connects the contract to the outside system.

**Professional answer:** "I keep core business policy independent of infrastructure details. For example, a reconciliation service can depend on a `TransactionStore` contract while an adapter implements it with DB2 or another store. This allows focused tests and controlled replacement."

**Clarify:** Dependency injection supplies an implementation. Dependency inversion is the design of which component owns and depends on the interface. Constructor injection does not automatically make a design good.

**Demonstration — a vendor replacement:** Bad dependency: `ReconcileUseCase` imports `Db2JdbcClient` and contains DB2 SQL. Better dependency: it uses `TransactionStore.findUnmatched(batchId)`, a use-case-oriented contract; `Db2TransactionAdapter` owns SQL and row mapping. A test uses `InMemoryTransactionStore`.

**Why DIP applies:** The business-facing contract is controlled by the application, and the DB adapter depends on it—not the other way around. **Evidence:** The matching rule can be tested with no database; a storage migration changes the adapter but not reconciliation policy. Merely injecting `Db2JdbcClient` does **not** invert the dependency.

**Code anchor (illustrative fragment):**

JAVA

```
interface PaymentStore {
    Optional<Payment> find(PaymentId id);
}

final class PaymentInquiry {
    private final PaymentStore store;

    PaymentInquiry(PaymentStore store) {
        this.store = store;
```

```
    }
}

final class JdbcPaymentStore implements PaymentStore {
    public Optional<Payment> find(PaymentId id) { /* SQL */ }
}
```

**Verify it:** Replace JdbcPaymentStore with a fake in a use-case test. The domain/application code does not import JDBC.

## 6. Cohesion and coupling [High]

**Plain English:** Cohesion means related work stays together. Coupling means components rely on each other's details. Aim for cohesive components with narrow, understandable connections.

**Professional answer:** "I evaluate a boundary by asking whether its logic belongs together and how many implementation details leak across it. Clear contracts and data ownership reduce the number of changes needed for a new feature."

**Example:** If changing a database column requires edits in five unrelated services, a storage detail has leaked through boundaries. Low coupling does not mean no collaboration: services still need contracts.

**Demonstration — schema leakage:** The API response, fee algorithm, and audit logger all read `PAYMENT_FEE_RULE_CODE` directly from a database row. Renaming the column forces edits across unrelated modules. Introduce a repository mapping from the row to a domain concept such as `FeeRule`, then let other components consume the stable concept.

**Why it fits:** Mapping belongs with persistence (cohesion); the column name no longer leaks into pricing, transport, or audit (lower coupling). **Evidence:** A storage-column rename requires changing the repository mapping and its integration test, not the fee unit test or API contract.

**Code anchor (illustrative fragment):**

JAVA

```
// Persistence maps database vocabulary to domain vocabulary.
final class PaymentRowMapper {
    Payment toDomain(PaymentRow row) {
        return new Payment(row.id(),
                FeeRule.parse(row.feeRuleCode()));
    }
}
// FeeCalculator accepts Payment, not PaymentRow.
```

**Verify it:** Rename fee_rule_code in the schema and verify that the API and fee calculator stay unchanged.

## 7. Composition over inheritance [High]

**Plain English:** Composition builds behavior using collaborating objects. Inheritance models a true is-a relationship. Composition often makes policy selection easier without a deep class tree.

**Professional answer:** "I use composition for variable behavior, such as fee calculation or provider integration. I reserve inheritance for a stable substitutable relationship. This keeps each policy independently testable."

**Example:** `PaymentProcessor` uses `FeePolicy`. It does not need subclasses for every market, channel, and currency combination. Those axes can combine without multiplying subclasses.

**Demonstration — combinatorial variation:** An inheritance plan creates `DomesticCardProcessor`, `InternationalCardProcessor`, `DomesticAchProcessor`, and `InternationalAchProcessor`. Adding an approval-policy variation multiplies subclasses again. Prefer `PaymentProcessor(FeePolicy, PaymentRail, ApprovalPolicy)` and select each collaborator explicitly.

**Why it fits:** Region, rail, and authorization are independent axes. Composition avoids pretending every combination is a distinct subtype. **Evidence:** Add a new approval policy and test it against existing rails without duplicating their code. Avoid composition when it produces arbitrary combinations that the business actually forbids; validate compatibility at assembly time.

**Code anchor (illustrative fragment):**

JAVA

```java
final class PaymentProcessor {
    private final FeePolicy fees;
    private final ApprovalPolicy approvals;
    private final PaymentRailGateway rail;

    PaymentProcessor(FeePolicy fees,
                     ApprovalPolicy approvals, PaymentRailGateway rail) {
        this.fees = fees;
        this.approvals = approvals;
        this.rail = rail;
    }
}
```

**Verify it:** Change the approval policy without making new subclasses for every region and rail. Reject unsupported collaborator combinations at assembly time.

## 8. DRY, KISS, and YAGNI [High]

**Plain English:** DRY avoids repeating the same knowledge in many places. KISS favors the simplest design that meets the requirement. YAGNI says do not build a feature solely because it may be useful someday.

**Professional answer:** "I remove duplicated business rules, but I do not extract unrelated code just because two blocks look similar. I start with the simplest design that meets current behavior and add extension points when variation becomes real."

**Trap:** Two similar-looking transformations may follow different business policies. Forcing them into one shared helper can create many flags and make changes riskier. Duplicate syntax and duplicate knowledge are not always the same thing.

**Demonstration — three different mistakes:**

- **DRY:** Two services each hard-code the same legally required transaction rounding rule. Put the single policy definition in an owned domain component and test it once with boundary cases.
- **KISS:** A fixed two-case mapping is clearer as a small switch than a plugin framework with discovery, reflection, and configuration.
- **YAGNI:** Do not build a multi-region payment router before the business supports a second region or has a credible migration requirement.

**Why these differ:** DRY concerns duplicated knowledge, KISS concerns unnecessary complexity, and YAGNI concerns speculative requirements. **Evidence:** A real policy change needs one source update, the code remains readable, and no extension machinery exists without a current use case.

**Code anchor (illustrative fragment):**

JAVA

```java
// DRY: one owner for a real shared business rule.
Money roundFee(Money input) { /* agreed rule */ }

// KISS: for two stable options, a switch is enough.
boolean needsReview(RiskBand band) {
    return switch (band) {
        case HIGH -> true;
        case LOW -> false;
    };
}
// YAGNI: no plugin registry without a real second plugin.
```

**Verify it:** Test one rounding policy with boundary values. Avoid building extension machinery to replace a readable two-case switch.

## 9. Encapsulation and invariants [High]

**Plain English:** Encapsulation hides implementation details and protects rules that must always hold. An invariant is such a rule: an approved payment cannot have a negative amount, or an account change must be authorized.

**Professional answer:** "I keep state changes behind operations that enforce the business rules. I do not expose mutable internals and hope every caller validates them correctly. I also use database constraints for rules that must hold across concurrent application instances."

**Example:** A `Payment` may expose `approve()` with checks rather than a public `setStatus()`. Client-side validation and DTO validation do not replace domain or database protection.

**Demonstration — illegal state:** A publicly writable `payment.status` permits a caller to set `SETTLED` on a draft without authorization. Replace arbitrary setters with `authorize(actor)`, `markSettled(providerReference)`, and guarded transitions. Persist conditional updates to handle competing processors.

**Why it fits:** An invariant such as "settlement requires authorization and a provider reference" is enforced at the state-changing boundary. **Evidence:** Tests reject invalid transitions; a concurrent update test confirms only the expected prior version can advance. Object-level checks alone do not prevent database races; use version checks, locking, or constraints as appropriate.

**Code anchor (illustrative fragment):**

JAVA

```java
final class Payment {
    private Status status = Status.CREATED;

    void authorize() {
        if (status != Status.CREATED)
            throw new IllegalStateException("bad transition");
        status = Status.AUTHORIZED;
    }
    // No public setStatus(...).
}
```

**Verify it:** An attempt to authorize twice must fail. Add database concurrency protection when two processes can update the same row.

## 10. Design for testability and observability [High]

**Plain English:** A useful design makes correct behavior and failure behavior visible. Small contracts make tests easier; logs, metrics, and traces make production behavior explainable.

**Professional answer:** "I build seams around external dependencies and test business rules independently. I also make important state transitions and failure paths observable so an incident can be diagnosed from evidence rather than guesses."

**Example:** Test a retry rule with a fake provider, then integration-test real persistence and deduplication constraints. Track request ID, business ID, state, latency, and error category without logging secrets.

**Demonstration — an intermittent timeout:** Rather than performing a real provider call in a unit test, inject a `ClearingGateway` port whose fake returns success, a transient error, or an unknown outcome. Verify the correct business status for each case. In production, emit a trace with instruction ID, dependency name, latency, and a sanitized outcome category.

**Why it fits:** Tests prove the decision path, while telemetry helps explain the same path under live load. **Evidence:** A test reliably reproduces timeout behavior and an operator can correlate a pending instruction to its retries without exposing bank or identity secrets. Unit tests do not replace contract or integration tests.

**Code anchor (illustrative fragment):**

JAVA

```java
interface ClearingGateway {
    GatewayResult send(Payment p);
}
```

```
// In a unit test:
ClearingGateway timeout = p -> GatewayResult.UNKNOWN;
PaymentService service = new PaymentService(timeout);

assertEquals(Status.OUTCOME_UNKNOWN, service.submit(testPayment()));
```

**Verify it:** Inject a fake timeout and assert the business state. In production, log the instruction ID and sanitized error category, never account secrets.

## 11. Fail fast at boundaries, recover deliberately [Medium]

**Plain English:** Reject invalid input before doing expensive or irreversible work. For failures that may be temporary, recover only when the operation is safe to repeat.

**Professional answer:** "I validate preconditions early, return a clear error for invalid requests, and reserve retries for known transient failures. For a timed-out payment call, I first determine whether the operation completed before retrying."

**Trap:** "Fail fast" does not mean crash every service on a recoverable downstream problem. The boundary, recovery contract, and customer impact matter.

**Demonstration — validation versus uncertainty:** Reject a missing beneficiary or unsupported currency with a clear client error before storing an instruction. By contrast, if a remote provider times out after a valid submission, record `OUTCOME_UNKNOWN`; do not treat it as an invalid request or blindly re-send it.

**Why it fits:** Invalid input is deterministic; a remote timeout may conceal a successful side effect. **Evidence:** An invalid-input test proves no instruction/event was persisted, while a timeout test proves the system schedules a status query or reconciliation. A retry is allowed only if the target's deduplication contract makes repetition safe.

**Code anchor (illustrative fragment):**

JAVA

```
SubmitResult submit(Request request) {
    if (request.beneficiary() == null)
        throw new BadRequestException("beneficiary required");
    GatewayResult result = gateway.send(toPayment(request));
    if (result == GatewayResult.UNKNOWN)
        return SubmitResult.pendingReconciliation();
    return SubmitResult.from(result);
}
```

**Verify it:** The bad input path must make zero gateway calls. The unknown-outcome path must not blindly retry a potentially completed payment.

### 12. Separate policy from mechanism [Medium]

**Plain English:** Policy is the business decision; mechanism is how the system performs it. "Who may approve a payment?" is policy. "How do we read the role from a token?" is mechanism.

**Professional answer:** "I keep business decisions explicit and avoid burying them in transport or persistence code. A policy can then be reviewed by business owners and tested without making an HTTP call or using a real database."

**Example:** `ApprovalPolicy.canApprove(principal, payment)` should express the rule. A security adapter obtains the principal, and a repository loads the payment.

**Demonstration — approval access:** A controller inspects JWT claims and contains `if (role == MANAGER && amount < threshold) ...`. That mixes token parsing with a business approval rule. Instead, a security adapter resolves `Principal`, the application loads `Payment`, and `ApprovalPolicy.canApprove(principal, payment)` decides.

**Why it fits:** Business owners can review the approval policy independently of token format, and an identity-provider migration changes the adapter. **Evidence:** Table-driven tests vary role, tenant, limit, and payment state with no HTTP request. Authentication still belongs at the boundary; do not move token parsing into the domain.

**Code anchor (illustrative fragment):**

JAVA

```java
// Business rule: no JWT parsing here.
boolean canApprove(Principal actor, Payment p) {
    return actor.tenant().equals(p.tenant())
            && actor.approvalLimit().compareTo(p.amount()) >= 0;
}
// HTTP/JWT adapter builds Principal; policy uses it.
```

**Verify it:** Vary role, tenant, limit, and payment status in table-driven tests with no HTTP request.

## 3. Object-level design patterns

Patterns are useful vocabulary, not proof of good design. In this section, say **problem → shape → example → cost**. When an interviewer asks "Which pattern would you use?", clarify whether behavior varies, a vendor must be isolated, work is asynchronous, or state changes are constrained.

### Strategy [High]

**Plain English:** Put each interchangeable rule behind the same small contract. Select the rule based on context.

**Professional answer:** "I use Strategy when one behavior has several valid policies, such as fee calculation by product. The caller uses one contract and each policy is tested separately. I avoid it for a single simple branch."

**Shape:** `FeePolicy.calculate(payment)`. Implementations could be domestic and international fee policies. Be careful that variation may depend on more than country; make selection rules explicit. **Cost:** More classes and a policy-selection mechanism.

**Demonstration:** A service must calculate different fees for domestic and cross-border instructions. It asks `FeePolicy.calculate(instruction)`; `DomesticFeePolicy` and `CrossBorderFeePolicy` implement that contract, and a `FeePolicySelector` chooses according to a documented rule.

**Why Strategy, not State:** The rule changes with instruction context; the payment does not transition from one policy object to another through its lifecycle. **Proof:** Run the same fee contract tests on both policies, plus selector tests at region/currency boundaries. **Avoid it:** A stable one-line conditional may be clearer.

**Code anchor (illustrative fragment):**

JAVA

```java
interface FeePolicy {
    Money fee(Payment p);
}

final class DomesticFee implements FeePolicy {
    public Money fee(Payment p) {
        return Money.usd("1.50");
    }
}

final class CrossBorderFee implements FeePolicy {
    public Money fee(Payment p) {
        return p.amount().times("0.01");
    }
}
// A resolver chooses the rule; processor calls policy.fee(p).
```

**Verify it:** Change one fee rule without changing the processor. Test policy selection separately from fee computation.

## Factory Method or simple factory [High]

**Plain English:** Put object creation and selection in one place. A simple factory can choose a `PaymentHandler` based on payment type. Factory Method specifically lets subclasses decide which concrete object to create.

**Professional answer:** "I separate complex creation from the business flow. If selection rules are centralized, a factory can return the correct handler while the processing code works with the handler contract. For a trivial object, direct construction is clearer."

**Trap:** Do not call every constructor wrapper a Factory Method. An interviewer may ask you to distinguish the formal pattern from a simple factory.

**Demonstration:** A `HandlerFactory.forRail(rail)` returns an `AchHandler` or `CardHandler`. The submission use case asks for a handler without knowing its constructor dependencies. This is a **simple factory**, because a selector method—not overriding subclasses—chooses the result.

**Why a factory:** Handler creation has real selection rules or nontrivial configuration. **Proof:** Unsupported rails receive an explicit rejection, and mapping tests show the correct handler for every supported rail. **Avoid it:** Construct `new FeeCalculator()` directly if no selection or construction complexity exists.

**Code anchor (illustrative fragment):**

JAVA

```java
final class PaymentHandlerFactory { // Simple factory.
    PaymentHandler create(PaymentRail rail) {
        return switch (rail) {
            case ACH -> new AchHandler();
            case CARD -> new CardHandler();
        };
    }
}
// Factory Method specifically delegates creation to subclasses.
```

**Verify it:** Verify the returned concrete handler for each supported rail; avoid a factory when one constructor is sufficient.

## Builder [High]

**Plain English:** Construct a complex object step by step while making required data and validation clear.

**Professional answer:** "I use Builder when a request or configuration has many optional fields and construction should be readable. I validate required fields at build time and favor an immutable result."

**Example:** A payment instruction has mandatory tenant, amount, currency, and beneficiary, plus optional remittance details. **Cost:** Extra code; records or constructors may be enough for a small object.

**Demonstration:** A `PaymentInstruction` needs tenant, beneficiary, currency, and amount, with optional remittance text and delivery preference. A builder makes call sites readable: `builder.tenant(t).beneficiary(b).amount(a).currency(c).build()`. `build()` rejects missing mandatory fields and returns an immutable result.

**Why Builder, not Factory:** The question is how to assemble and validate one complex value, not which implementation to choose. **Proof:** Tests check missing required fields, invalid combinations, and that the resulting value cannot be changed through shared mutable references. A short Java record or constructor may be better when there are only a few fields.

**Code anchor (illustrative fragment):**

JAVA

```java
PaymentInstruction instruction = PaymentInstruction.builder()
        .tenantId(tenantId)
        .amount(amount)
        .currency("USD")
        .beneficiary(beneficiary)
```

```
        .reference("invoice-42") // optional
        .build(); // validates required fields
```

**Verify it:** Omit each mandatory field in turn: build () must reject the incomplete value. Keep the built object immutable.

## Adapter [High]

**Plain English:** Translate an external interface into the interface your application wants.

**Professional answer:** "I use Adapter to keep provider-specific APIs and data shapes outside business logic. Our core service calls a stable internal contract; the adapter handles provider request mapping, errors, and transport details."

**Example:** An old file-based payment interface and a new real-time service can implement the same internal `PaymentSubmissionPort` during migration. **Cost:** Translation logic and another layer; do not hide differences that change business semantics.

**Demonstration:** Provider A accepts `{customerRef, totalMinorUnits}` and returns status `QUEUED`; Provider B expects an XML payment file and returns a receipt code. Both are mapped to the internal `ClearingGateway.submit(Instruction)` contract and a carefully defined `SubmissionReceipt`.

**Why Adapter:** Translation isolates external syntax and transport from the core. **Proof:** Provider-specific contract tests cover units, currency, provider errors, and status mapping; core tests use a fake `ClearingGateway`. **Important:** If one provider's `QUEUED` means only receipt rather than confirmed execution, preserve that semantic difference rather than mapping it to `SETTLED`.

**Code anchor (illustrative fragment):**

JAVA

```java
interface ClearingGateway {
    GatewayResult submit(Payment p);
}

final class LegacyFileAdapter implements ClearingGateway {
    private final LegacyFileClient client;

    public GatewayResult submit(Payment p) {
        return mapResponse(client.writeLine(toLegacyLine(p)));
    }
}
```

**Verify it:** Test field and error mapping. The use case must not depend on the legacy file format.

## Facade [High]

**Plain English:** Give callers one simple entry point to a complicated subsystem. The facade coordinates existing capabilities.

**Professional answer:** "I use a facade when clients should not orchestrate several low-level components themselves. For example, a user-onboarding facade can coordinate identity creation, role assignment, and audit recording behind one application operation."

**Cost:** A facade can become a very large service if it absorbs all business rules. Keep each collaborator's responsibility visible. A facade simplifies access; an adapter changes an interface.

**Demonstration:** New-user onboarding requires `IdentityCreator`, `RoleAssigner`, and `AuditRecorder`. Instead of requiring a controller to invoke them in the right order, expose `OnboardingFacade.createUser(request)` and let it coordinate their interfaces.

**Why Facade:** Callers need one business-level operation; no external interface needs translation. **Proof:** An application test verifies the call order and failure handling; each collaborator has its own tests. **Avoid it:** Do not hide every permission rule and persistence detail in a giant facade. In contrast, an Adapter primarily translates one interface into another.

**Code anchor (illustrative fragment):**

JAVA

```java
final class OnboardUserFacade {
    UserId onboard(NewUser input) {
        UserId id = identity.create(input);
        roles.assignDefaultRole(id);
        audit.recordUserCreated(id);
        return id;
    }
}
// One use-case entry point; collaborators keep their rules.
```

**Verify it:** Confirm that onboarding calls the collaborators in the required order and does not duplicate their domain rules.

## Decorator [Medium]

**Plain English:** Wrap a component with another component that has the same contract and adds behavior.

**Professional answer:** "Decorator can add cross-cutting behavior such as metrics or caching around a repository or client without changing its core implementation. I use it when that behavior is composable and the wrapper's effect remains clear."

**Example:** `TimedPaymentGateway` measures a gateway call and delegates to `PaymentGateway`. **Cost:** Several wrappers can obscure call order and failure behavior. A cache wrapper needs a freshness policy.

**Demonstration:** `TimedGateway` implements `ClearingGateway`, contains another `ClearingGateway`, records elapsed time and outcome, then delegates `submit()`. A `TracingGateway` could wrap the same contract separately.

**Why Decorator:** The caller still sees the original contract, while behavior is composed around it. **Proof:** The delegate is called once, the return value/error behavior is preserved, and metrics are recorded on success and failure. **Cost:** Wrapper order matters. For example, timing outside a retry wrapper measures total retry duration, while timing inside measures each attempt.

**Code anchor (illustrative fragment):**

JAVA

```java
final class TimedGateway implements ClearingGateway {
    private final ClearingGateway delegate;

    public GatewayResult submit(Payment p) {
        long start = clock.nanoTime();
        try {
            return delegate.submit(p);
        } finally {
            metrics.record(clock.nanoTime() - start);
        }
    }
}
```

**Verify it:** Measure both successful and failed calls. Verify that decoration does not change the gateway result.

## Proxy [Medium]

**Plain English:** A proxy stands in front of another object and controls access to it. It may add transactions, security, lazy loading, or remote access.

**Professional answer:** "I expect proxy behavior at framework boundaries. In Spring, transaction advice commonly runs through a proxy, so a method calling another method on the same instance may bypass that advice. I test transaction behavior at the actual call boundary."

**Difference from Decorator:** Both wrap, but a proxy often controls access or lifecycle, whereas a decorator intentionally adds composable behavior. The structures can look similar; discuss intent and effects. See Spring transaction implementation.

**Demonstration:** `@Transactional` is placed on `PaymentWriter.commit()`, but another method on the same instance calls `this.commit()`. With normal Spring proxy-based transaction management, that internal call bypasses transactional interception. Call the transactional method through a separately injected Spring-managed bean (or design a correctly advised external use-case boundary).

**Why Proxy:** Spring intercepts calls that pass through the proxy; it is a runtime access boundary rather than a fee-rule extension. **Proof:** An integration test forces a database error and checks whether the expected writes roll back. Verify configuration and transaction manager behavior; annotations alone are not proof.

**Code anchor (illustrative fragment):**

JAVA

```
@Service
class PaymentService {
    void accept() {
        saveInTransaction();
    } // Self-call bypasses proxy.

    @Transactional
    public void saveInTransaction() { /* DB writes */ }
}
// Prefer an external call through a proxied bean, or a
// correctly placed public transactional use-case boundary.
```

**Verify it:** Integration-test rollback through a real Spring proxy, not merely a direct method call. Official Spring docs confirm proxy self-invocation behavior.

## Observer and publish/subscribe [Medium]

**Plain English:** One component announces an event and interested components react. In-process Observer means listeners in the same process. A message broker makes communication cross-process, with different durability and delivery concerns.

**Professional answer:** "I publish an event when other components need to react to a completed state change without being called synchronously. I distinguish a local callback from a durable event and plan for duplicates, ordering, and failed consumers."

**Example:** User-created event triggers audit or notifications. Do not send a durable event before its source transaction commits unless the design handles rollback.

**Demonstration:** After `UserCreated` is durably committed, an audit consumer records the change and a notification consumer sends a welcome message. Each subscribes to the fact without the user-creation use case having to call them synchronously.

**Why pub/sub:** Independent reactions can lag without blocking creation. **Proof:** Duplicate `UserCreated` delivery does not create two audit effects or duplicate customer-facing notifications (by consumer-side idempotency). **Boundary:** A local Java listener has a different durability guarantee than broker delivery; simply calling `publish()` before a DB commit can produce a false event if the transaction rolls back.

**Code anchor (illustrative fragment):**

JAVA

```
record PaymentAccepted(PaymentId id, UUID eventId) {
}
// After source state commits, publish a durable event.
subscriber.

on(PaymentAccepted event) {
    if (dedup.insertIfAbsent(event.eventId()))
        projection.update(event.id());
}
// Dedup and side effect must be atomic in real storage.
```

**Verify it:** Deliver the same event twice: the projection must record one effect. This code is broker-neutral pseudocode for the callback body.

## Command [Medium]

**Plain English:** Represent an operation as a value. This can separate receiving a request from executing, recording, retrying, or scheduling it.

**Professional answer:** "A command makes the requested action explicit, such as `ApprovePayment`. A handler validates and executes it. This can help with workflow, auditing, or async dispatch, but I do not add command objects to simple code without a need."

**Trap:** A command is an instruction or intent; an event states something that already happened. `ApprovePayment` and `PaymentApproved` are not interchangeable.

**Demonstration:** `ApprovePayment(paymentId, approverId, requestId)` describes a requested action. An `ApprovePaymentHandler` checks permissions and state, writes an approved transition, and may later emit `PaymentApproved(paymentId, approvedAt)`.

**Why Command:** The request can be validated, traced, queued, or retried as a distinct unit of work. **Proof:** Handling the same request identifier twice does not create duplicate approval effects; unauthorized commands are rejected. **Key distinction:** `ApprovePayment` can fail; `PaymentApproved` claims that approval already happened.

**Code anchor (illustrative fragment):**

JAVA

```java
record ApprovePayment(PaymentId id, UserId actor) {
}

final class ApprovalHandler {
    void handle(ApprovePayment command) {
        Payment p = store.load(command.id());
        policy.requireAllowed(command.actor(), p);
        p.approve();
        store.save(p);
    }
}
```

**Verify it:** Run unauthorized and duplicate command tests. A command asks for work; it does not claim that work already happened.

## State [High]

**Plain English:** Behavior changes depending on an object's state. Representing valid transitions prevents scattered status checks.

**Professional answer:** "I model payment status as explicit transitions and reject invalid changes. For a complex workflow, a State pattern or transition table can make the rules reviewable. For a small fixed workflow, an enum with a validated transition method may be simpler."

**Example:** `CREATED -> AUTHORIZED -> SETTLED`, with `FAILED` paths. Define whether reversal after settlement is a new compensating operation rather than simply moving status backward. **Cost:** More structure, especially if every status becomes a class.

**Demonstration:** An instruction has `CREATED`, `AUTHORIZED`, `SUBMITTED`, `SETTLED`, and `REJECTED` states. `settle()` is legal only after a confirmed provider outcome; `submit()` from `SETTLED` is invalid. Encode the transition table in a domain method or, if behavior is complex, separate State objects.

**Why State, not Strategy:** Behavior is constrained by the object's lifecycle, not a freely chosen policy. **Proof:** Test every permitted and forbidden state/action pair, including competing updates. **Avoid it:** Do not generate one class per enum value when a transition table expresses the rules clearly.

**Code anchor (illustrative fragment):**

JAVA

```java
enum Status {CREATED, AUTHORIZED, SETTLED}

boolean allowed(Status from, Status to) {
    return switch (from) {
        case CREATED -> to == Status.AUTHORIZED;
        case AUTHORIZED -> to == Status.SETTLED;
        case SETTLED -> false;
    };
}
```

**Verify it:** Test every allowed and forbidden edge. A fixed enum can be better than one State subclass per status.

## Template Method [Medium]

**Plain English:** A base class fixes the overall algorithm and lets subclasses fill certain steps.

**Professional answer:** "Template Method fits a stable process with a few controlled variations. For example, each import may share validate, transform, persist, and report steps. I prefer composition if the variations combine in many ways or inheritance makes the hierarchy brittle."

**Trap:** A subclass must preserve the base process's contract. Hidden calls from the base class to overridable methods can complicate debugging.

**Demonstration:** `BaseStatementImporter.importFile()` fixes `validate -> parse -> persist -> summarize`. `CsvImporter` and `FixedWidthImporter` override only parsing; neither can bypass the shared validation in the base algorithm.

**Why Template Method:** The process order is stable, with a few controlled algorithm steps. **Proof:** Tests assert that invalid files never reach `persist()`, regardless of concrete importer. **Limit:** If validation, transformation, and persistence vary independently, composed strategies are easier to test and combine than a deep importer hierarchy.

**Code anchor (illustrative fragment):**

JAVA

```java
abstract class ImportJob {
    public final void run(File file) {
        validate(file);
        persist(transform(file));
        report();
    }

    protected abstract Rows transform(File file);
}
// Shared sequence is fixed; subclasses vary one step.
```

**Verify it:** A subclass may change transformation but not reorder validation, persistence, and reporting.

## Chain of Responsibility [Medium]

**Plain English:** Pass a request through a sequence of handlers; each handler may process, reject, or pass it on.

**Professional answer:** "I use a chain when independent checks need an ordered pipeline, such as authentication, input limits, and audit enrichment. I document the order and whether a handler can stop processing."

**Cost:** Debugging becomes hard if order is implicit or failures disappear in the chain. Spring filters or interceptors are familiar examples, though their exact semantics differ.

**Demonstration:** An API request passes through `Authenticate -> TenantAuthorize -> ValidateSize -> RateLimit -> Controller`. Each step can allow the request to continue or reject it with an explicit error.

**Why Chain:** Multiple focused handlers must run in a known order. **Proof:** A test with an invalid tenant shows the request stops before the controller; logs identify which handler rejected it. **Risk:** Rate-limiting before authentication may require a different key and have different security properties; filter order is a design decision, not an accident.

**Code anchor (illustrative fragment):**

JAVA

```java
interface Check {
    void verify(Request request);
}

final class ValidationPipeline {
    private final List<Check> checks;

    void run(Request request) {
        for (Check check : checks) check.verify(request);
    }
}
// Throwing from a check stops later checks.
```

**Verify it:** Test order and early exit. Security checks must not be accidentally skipped because of chain configuration.

## Repository and Unit of Work [High]

**Plain English:** A Repository gives domain-oriented access to stored objects. A Unit of Work tracks related changes that should commit as one unit. In Spring/JPA, a persistence context and transaction often supply much of the Unit of Work behavior.

**Professional answer:** "I keep persistence queries behind meaningful repository operations and place the transaction around the business operation. I do not assume a repository method makes a multi-step workflow atomic; that depends on the transaction boundary and database guarantees."

**Cost:** A generic repository with dozens of vague methods can hide inefficient queries. A repository is not automatically an abstraction over every storage engine.

**Demonstration:** `ApprovePaymentUseCase` loads an instruction through `PaymentRepository`, changes its domain state, and appends an audit/outbox record inside **one local transaction**. A call to `save()` alone does not make the entire multi-step operation atomic.

**Why Repository + Unit of Work:** Repository hides storage-specific queries; a transaction coordinates the related state changes. **Proof:** Force an outbox insert failure and verify the approval update also rolls back. Add a concurrent-approval test with version checks or suitable locking. An HTTP call to an external provider is not automatically part of that transaction.

**Code anchor (illustrative fragment):**

JAVA

```java
@Service
final class SettlementService {
    @Transactional
    public void settle(PaymentId id) {
        Payment p = repository.findRequired(id);
        p.markSettled();
        repository.save(p);
        outbox.save(PaymentSettled.of(p));
    }
}
// The local DB transaction covers both writes.
```

**Verify it:** Force an outbox insert failure: the payment update must roll back. A broker publish is not inside this local transaction.

## Singleton: recognize it, use sparingly [Medium]

**Plain English:** A Singleton controls creation so there is one instance within a defined scope. In a distributed service, each process still has its own instance.

**Professional answer:** "I usually let the application container manage shared, stateless services. I do not use a global singleton for request state or payment deduplication because it cannot coordinate separate instances or survive restarts."

**Trap:** A Spring singleton bean is scoped to an application context. Shared mutable fields still need concurrency protection.

**Demonstration:** A stateless `FeeCatalog` can be a shared Spring bean. A mutable `processedIds = new HashSet<>()` stored in a singleton is **not** safe payment deduplication: concurrent requests race, each service instance has a separate set, and restarts erase it.

**Why it matters:** Instance scope is not a distributed consistency mechanism. **Proof:** Run the duplicate-request test across two app instances; expect a database uniqueness rule or other durable, coordinated store to decide the winner. Avoid storing per-request tenant or customer context in singleton fields.

**Code anchor (illustrative fragment):**

JAVA

```java
@Service // Spring singleton by default: per app context.
final class FeeCalculator {
    Money calculate(Payment payment) { /* stateless */ }
}
// BAD: a mutable field such as currentTenant
// would mix data from concurrent requests.
```

**Verify it:** Send concurrent requests for two tenants; a shared service must not retain tenant-specific mutable state.

# 4. Architecture patterns and when to choose them

## 1. Layered architecture [High]

**Plain English:** Group work by role, such as API, application service, business model, and persistence. A request moves through clear responsibilities.

**Professional answer:** "Layering helps keep transport, business rules, and data access understandable. In a Spring service, the controller handles HTTP, the application service coordinates a use case, and repositories persist state. I watch for business rules leaking into controllers or repositories."

**Use when:** A service has a straightforward workflow. **Cost:** Too many pass-through layers; an anemic domain model if every rule is pushed into one large service class.

**Demonstration:** `PaymentController` parses HTTP and returns an API response; `SubmitPaymentService` authenticates business intent and coordinates the use case; `Payment` enforces valid state transitions; `PaymentRepository` reads/writes data.

**Why layering:** It prevents controllers from becoming business engines and repositories from deciding payment policy. **Proof:** A fee-rule change can be tested without an HTTP server; a request-shape change does not alter domain transitions. **Tradeoff:** Avoid controller → service → service → repository pass-through chains that do not add a meaningful boundary.

**Code anchor (illustrative fragment):**

JAVA

```java
@RestController
class PaymentController {
    private final SubmitPayment useCase;

    @PostMapping("/payments")
    Receipt submit(@RequestBody PaymentRequest req) {
        return useCase.submit(req.toCommand());
    }
}
// Controller: HTTP; use case: workflow; repository: storage.
```

**Verify it:** A policy change should not require editing HTTP mapping or SQL. Avoid layers that do nothing beyond forwarding.

## 2. Hexagonal architecture / ports and adapters [High]

**Plain English:** Put core business logic in the center. Define ports for what it needs and adapters for HTTP, databases, queues, or vendor systems. The core should not need to know the vendor's API.

**Professional answer:** "I use ports and adapters when external integrations or storage may change, or when business rules need isolated tests. The application owns a business-oriented port, and an adapter translates to the technology. I do not add a separate interface and adapter for every trivial class."

**Example:** `SubmitPaymentUseCase` depends on `PaymentStore` and `ClearingGateway`. REST and batch entry points call the same use case. DB2 and provider adapters implement the ports. **Cost:** More boundary code and mapping. See AWS hexagonal architecture guidance.

**Demonstration:** Two entry adapters, REST and batch-file import, both invoke `SubmitPaymentUseCase`. The use case depends on `PaymentStore` and `ClearingGateway` ports. `JdbcPaymentAdapter` and `ProviderHttpAdapter` implement those ports; neither transport leaks into fee and authorization rules.

**Why hexagonal:** The application owns stable capabilities while technologies sit outside. **Proof:** The same contract tests run against a fake store and the actual JDBC adapter; changing REST to batch input leaves the core use case unchanged. **Tradeoff:** Boundaries and mapping require code; a small CRUD endpoint may not need all those ports.

**Code anchor (illustrative fragment):**

JAVA

```java
// Owned by the application core:
interface ClearingPort {
```

```
    GatewayResult clear(Payment p);
}

final class SubmitPayment {
    private final ClearingPort clearing;

    SubmitPayment(ClearingPort clearing) {
        this.clearing = clearing;
    }
}
// ProviderSdkAdapter implements ClearingPort at the edge.
```

**Verify it:** Instantiate the use case in a test with an in-memory port and no provider SDK. The dependency points inward.

## 3. Modular monolith [High]

**Plain English:** One deployable application can still have strong internal modules, such as identity, payment, and reconciliation. Each module owns its rules and communicates through clear interfaces.

**Professional answer:** "I consider a modular monolith when teams need clean boundaries but separate deployments and distributed failure handling are not justified. It is a useful starting point because local transactions and debugging are simpler. I keep module ownership explicit so future extraction remains possible."

**Cost:** Shared deployment and runtime; boundaries can erode if every module reads another's tables. A module is not isolated merely because it has a package name.

**Demonstration:** One Spring deployment contains `identity`, `payments`, and `reconciliation` modules. Reconciliation calls an exported `payments` query interface instead of reading payment tables directly; module tests reject forbidden imports.

**Why modular monolith:** Teams gain explicit ownership without network latency and distributed transactions for every interaction. **Proof:** A payment schema refactor does not require edits in the reconciliation module; module dependency rules are enforced in CI. **Tradeoff:** The application still deploys and fails as one process. Extract a service only when the independent deployment or scaling need is proven.

**Code anchor (illustrative fragment):**

JAVA

```
// identity module exposes an application-level interface:
interface IdentityLookup {
    Principal findAuthorized(UserId actor);
}

// payments module depends on IdentityLookup,
// not identity's private tables or persistence classes.
final class SubmitPayment {
    SubmitPayment(IdentityLookup identities) { /* ... */ }
}
```

**Verify it:** Add a module-boundary test: payments must not import identity.persistence.* or query its tables directly.

## 4. Microservices with owned data [High]

**Plain English:** Independent services own capabilities and, ideally, control their own data. They communicate through APIs or events. This helps independent change but creates network, consistency, testing, and operations costs.

**Professional answer:** "I choose service boundaries around business capabilities and team ownership, not around small classes. I account for API contracts, data ownership, observability, deployment, and partial failure. I do not split a cohesive workflow simply because microservices are popular."

**Caution:** Database-per-service means one service controls access to its data; it does not require a separate database server for every service. Cross-service reports need API composition, materialized views, or events. See database-per-service pattern.

**Demonstration:** The Payment service owns instructions; Reconciliation owns matching runs; Identity owns user access data. Reconciliation consumes `PaymentSettled` events or a supported Payment API rather than querying Payment tables directly.

**Why microservices:** Separate teams may need independent releases, capacity, and fault isolation. **Proof:** The reconciliation deploy leaves Payment unchanged; a reconciliation outage does not corrupt accepted payments, and lag is monitored. **Tradeoff:** Cross-service reports require projections/APIs, events may be delayed, and failure recovery is harder. A split that causes constant synchronous calls may be worse than a modular monolith.

**Code anchor (illustrative fragment):**

EXAMPLE

```
Payment Service --HTTP/event--> Identity Service
      |                              |
Payment-owned tables         Identity-owned tables

// No cross-service SQL join or shared-table writes.
// Payment may cache a permitted identity projection.
```

**Verify it:** Stop Identity Service: Payment must have a defined timeout or degraded response. Teams must own API versioning and data boundaries.

## 5. Synchronous request/response [High]

**Plain English:** A caller sends a request and waits for an answer. It is simple when the answer is needed now. The caller's response time and availability depend on the downstream path.

**Professional answer:** "I use a synchronous call when the caller needs an immediate decision. I set a deadline, bound concurrency, and define what a timeout means. I avoid a long chain of dependent calls when it makes latency and failure hard to control."

**Example:** Validate a payment request and return whether it was accepted for processing. Do not promise final settlement if that occurs later.

**Demonstration:** `POST /payment-instructions` synchronously authenticates, validates, and commits an instruction record. It returns an ID and `ACCEPTED` once local acceptance is durable; settlement occurs asynchronously.

**Why synchronous for acceptance:** The caller must know whether its request was durably recorded before leaving the screen. **Proof:** A successful response always maps to a retrievable instruction, and a commit failure never produces a successful acknowledgment. **Limit:** Provider settlement should not be implied by the acceptance HTTP status. Long dependency chains can consume the caller's latency budget.

**Code anchor (illustrative fragment):**

JAVA

```java
HttpRequest request = HttpRequest.newBuilder(uri)
        .timeout(Duration.ofSeconds(2))
        .POST(BodyPublishers.ofString(payload))
        .build();
HttpResponse<String> response =
        client.send(request, BodyHandlers.ofString());
// Timeout is a caller deadline, not proof of remote failure.
```

**Verify it:** Verify request deadlines and distinguish accepted-for-processing from fully settled. Client and transport settings may add other timeouts.

## 6. Event-driven and publish/subscribe architecture [High]

**Plain English:** A service emits a fact about a completed change; other services respond when they receive it. This reduces direct coupling but introduces delivery delay and duplicate or out-of-order events.

**Professional answer:** "I use events when downstream consumers can react independently and eventual consistency is acceptable. I define event ownership, schema evolution, idempotent consumers, ordering needs, and how failed events are replayed."

**Example:** `PaymentAccepted` can trigger audit and status projection. Do not put sensitive data in an event merely for convenience. A command tells one component to do something; an event tells interested components something happened.

**Demonstration:** After a payment settles, the Payment service emits `PaymentSettled` with event ID, instruction ID, schema version, and non-sensitive status metadata. A reconciliation projection and an audit consumer update independently.

**Why events:** Payment does not need to wait for the two reactions to finish. **Proof:** Replay the event and verify that idempotent consumers do not duplicate their results; a delayed event shows an explicit 'last updated' time on the projection. **Limit:** An event is not a synchronous query or a guarantee of immediate read consistency.

**Code anchor (illustrative fragment):**

JAVA

```java
record PaymentAccepted(PaymentId id, UUID eventId) {
}

void onPaymentAccepted(PaymentAccepted event) {
    if (!inbox.claimOnce(event.eventId())) return;
    auditProjection.apply(event.id());
}
// Claim and projection update need an atomic strategy.
```

**Verify it:** Replay an event and deliver it out of order. Decide whether ordering matters and how projection versioning handles it.

## 7. Transactional outbox [High]

**Plain English:** Saving business data and publishing an event are two writes. One can succeed while the other fails. An outbox saves the business change and event record in one local database transaction; a relay publishes the event afterward.

**Professional answer:** "I use an outbox when a committed state change must reliably produce an event. The local transaction writes both business and outbox rows. A relay may publish more than once, so consumers still need idempotency and monitoring."

**Cost:** Outbox storage, relay, lag, cleanup, ordering decisions. It does not make an external payment provider part of the local transaction. See AWS transactional outbox.

**Demonstration — dual-write failure:** If `savePayment()` commits but `broker.publish()` fails, downstream consumers never hear about it. Instead, within a **single database transaction**, insert the payment row and an outbox row containing `eventId`, type, and payload. A separate relay publishes pending records and marks or tracks delivery.

**Why outbox:** It closes the gap between committed local state and a missing event. **Proof:** Kill the process after DB commit and before publish: the relay still finds the event; publish twice and verify the consumer effect occurs once. **Limit:** The relay can redeliver, so it is **not** a blanket exactly-once guarantee. Monitor relay lag and retention.

**Code anchor (illustrative fragment):**

SQL

```sql
BEGIN;
INSERT INTO payment_instruction(id, status)
VALUES (:id, 'ACCEPTED');
INSERT INTO outbox(event_id, aggregate_id, payload)
VALUES (:event_id, :id, :payload);
COMMIT;
-- A separate relay publishes outbox rows later.
```

**Verify it:** Roll back either insert and confirm both rows disappear. Test duplicate relay publishing and durable consumer deduplication.

## 8. Saga [Medium; later design round]

**Plain English:** A workflow across several services is split into local steps. If a later step fails, the system performs a defined compensating action where possible. Compensation is a new business operation, not a magic database rollback.

**Professional answer:** "I consider a saga when one business process crosses independently owned services and a single database transaction cannot cover it. I define states, retry and compensation behavior, and what happens after an irreversible step. For a simple workflow within one database, a local transaction is safer and easier."

**Example:** Reserve funds, submit instruction, record confirmation. If submission fails before an irreversible action, release the reservation. If the external outcome is unknown, reconcile before resubmitting. **Cost:** State tracking, delayed consistency, failed compensation and operational complexity. See Microsoft saga pattern.

**Demonstration:** A cross-service onboarding workflow creates a customer identity, provisions an account, and activates a payment profile. If account provisioning fails, the orchestrator requests an identity cleanup where business rules allow. If profile activation has already caused an irreversible outside action, the workflow needs a new reversal or review step, not a fake rollback.

**Why saga:** Each service commits to its own store, and no safe single ACID transaction covers the workflow. **Proof:** Test failure at each step, including a failed compensation, and ensure the saga ends in a visible `NEEDS_REVIEW` state instead of silently disappearing. **Avoid it:** One service and one local database transaction need no saga.

**Code anchor (illustrative fragment):**

JAVA

```java
void handle(ReservationSucceeded e) {
    saga.markReserved(e.orderId());
    clearing.submit(e.orderId());
}

void handle(SubmissionRejected e) {
    reservation.requestRelease(e.orderId());
    saga.markCompensating(e.orderId());
}
// Compensation is another action; it can fail.
```

**Verify it:** Simulate compensation failure and an unknown external outcome. Do not assume a remote charge can be rolled back.

## 9. CQRS [Medium; later design round]

**Plain English:** Commands change state; queries read it. CQRS separates their models so each can fit its job. This can be as small as separate code paths in one database or as large as a separate read store.

**Professional answer:** "I use CQRS only when read and write needs differ enough to justify separate models. For example, payment changes may enforce strict invariants while reconciliation dashboards need a denormalized view. I would define how the view is updated and what users see during lag."

**Cost:** More code and consistency management. CQRS does not require event sourcing, a message broker, or microservices. See Microsoft CQRS pattern.

**Demonstration:** A payment write path enforces approval and settlement invariants using normalized transactional tables. A reconciliation dashboard reads a denormalized projection keyed by tenant/date/status to avoid expensive joins for every chart.

**Why CQRS:** The write model optimizes correctness; the read model optimizes a genuinely different query shape. **Proof:** Compare dashboard latency and write throughput, and test how the UI displays a projection that is several seconds behind. **Limit:** CQRS can use one database and does not imply event sourcing; if one simple query meets the need, separation adds needless lag and operational work.

**Code anchor (illustrative fragment):**

JAVA

```java
// Write model owns invariants and state transitions.
void approve(ApprovePayment cmd) { /* validate + persist */ }

// Read model is optimized for a screen.
record PaymentSummary(String id, String status, Money total) {
}

PaymentSummary dashboardRow(PaymentId id) { /* projection */ }
```

**Verify it:** Measure projection lag when using a separate read store. CQRS can also use one database and separate query models.

## 10. Event sourcing [Low for this round]

**Plain English:** Store a sequence of events as the source of truth and rebuild current state from them. An audit log beside a normal database is not automatically event sourcing.

**Professional answer:** "Event sourcing can provide a detailed history and support rebuilding projections, but it adds schema evolution, replay, ordering, and operational complexity. I would use it only for a clear requirement that outweighs those costs."

**Example:** Account ledger events may be valuable, but financial audit requirements alone do not force full event sourcing. Know snapshots and replay only at a conceptual level now.

**Demonstration:** Rather than storing only `account.balance = 500`, an event-sourced ledger stores an ordered history such as `AccountOpened`, `FundsCredited(700)`, and `FundsDebited(200)`; the current balance is derived by replaying the accepted events.

**Why event sourcing:** Reconstructing prior state and knowing which accepted changes produced it are primary requirements. **Proof:** Rebuild a projection from an empty state and compare it with the live one; test event-version upgrades and replay handling. **Cost:** Corrections become new events, projections lag, and historical schemas must remain interpretable. A table of audit records alongside current state is not event sourcing.

**Code anchor (illustrative fragment):**

JAVA

```java
record Deposited(Money amount) implements AccountEvent {
}

record Withdrawn(Money amount) implements AccountEvent {
}

Balance replay(List<AccountEvent> events) {
    Balance balance = Balance.zero();
    for (AccountEvent e : events) balance = apply(balance, e);
    return balance;
}
// Current balance derives from stored events.
```

**Verify it:** Replay event history and compare the final state. Audit logs next to mutable balances are not automatically event sourcing.

## 11. Cache-aside [High]

**Plain English:** Read the cache first. On a miss, read the database and fill the cache. When source data changes, invalidate or update the cached copy according to a freshness policy.

**Professional answer:** "I use cache-aside for repeated reads where stale data is acceptable within a defined bound. I set tenant-aware keys, TTLs, invalidation rules, and stampede protection, then measure hit rate and tail latency. I keep authoritative payment state and permissions on a deliberate consistency path."

**Cost:** Staleness, extra failure mode, memory, invalidation races. A 35% latency improvement is useful only when the baseline and workload are known. See Redis cache-aside guidance.

**Demonstration:** A public currency-metadata lookup reads `cache[tenant+currency]`; on a miss it reads the authoritative store and fills the cache. On a metadata update, invalidate the affected key. The key must include every dimension that changes the answer.

**Why cache-aside:** Repeated reads can be served faster when a defined staleness window is tolerable. **Proof:** Measure hit rate and p95 latency under a representative workload; test concurrent miss storms and invalidation races. **Limit:** Do not treat cached balance, mutable payment status, or authorization as authoritative without a separately justified consistency design.

**Code anchor (illustrative fragment):**

JAVA

```
PaymentView get(PaymentId id) {
    PaymentView cached = cache.get(id);
    if (cached != null) return cached;
    PaymentView fresh = repository.readView(id);
    cache.put(id, fresh, ttl);
    return fresh;
}
// Writes must invalidate/update this view deliberately.
```

**Verify it:** Test a cache miss, a hit, and staleness after a write. Use tenant-aware keys and exclude unsafe permission shortcuts.

## 12. Circuit breaker, retry, timeout, and bulkhead [High]

**Plain English:** A timeout limits waiting. A retry tries again when failure may be temporary and repetition is safe. A circuit breaker stops calls to a dependency that keeps failing. A bulkhead limits how much of the application's resources that dependency can consume.

**Professional answer:** "I combine bounded timeouts, selective retries, a circuit breaker, and separate resource limits according to the dependency's failure behavior. I also define a safe fallback or clear error. For payments, a timeout is an uncertain outcome, so idempotency and reconciliation must be part of retry design."

**Cost:** A retry storm can make an outage worse; a circuit breaker can reject a recovered service until probing succeeds; bulkheads use extra capacity. Tune from latency and error data. See Microsoft circuit breaker and bulkhead.

**Demonstration — provider slows down:** Set a per-attempt deadline so calls do not wait forever. Retry only safe transient errors with bounded attempts and jitter. If failure rates stay high, open a circuit breaker to fail fast. Give provider calls a separate small concurrency pool so an outage does not exhaust threads needed for status APIs.

**Why all four:** Timeout bounds a call; retry handles temporary failure; breaker avoids repeated work against an unhealthy dependency; bulkhead limits the blast radius. **Proof:** In a test outage, status queries still work, calls are capped, and request traffic does not grow into a retry storm. **Payment caution:** A timed-out submission has an **unknown** outcome until deduplication/reconciliation proves otherwise.

**Code anchor (illustrative fragment):**

JAVA

```
// Pseudocode: actual library API and ordering vary.
withBulkhead("clearing",() ->

withCircuitBreaker("clearing",() ->

retryOnlyIfSafe(() ->

callWithDeadline(provider::submit))));
// A timeout does not prove whether provider acted.
```

**Verify it:** Inject high latency and persistent failures. Verify resource isolation and bounded attempts; prevent duplicate charges.

## 13. Queue-based load leveling [Medium]

**Plain English:** Put work into a durable queue when producers can send faster than consumers can safely process. Consumers take work at a controlled rate.

**Professional answer:** "I use a queue when the caller does not require immediate completion and burst traffic would overload a dependency. I define capacity, acceptance response, retry, duplicate handling, dead-letter treatment, and backlog monitoring."

**Cost:** Delayed results, eventual consistency, operational queues, ordering limitations. A queue does not remove a bottleneck; it changes how you absorb and expose it. See Microsoft load-leveling pattern.

**Demonstration:** A partner sends a burst of 30,000 import records, while the clearing integration can process only a smaller controlled rate. The API durably enqueues validated work and returns a job ID; consumers drain the queue at the downstream-safe rate.

**Why queueing:** It absorbs bursts without requiring the slower dependency to match peak arrival rate. **Proof:** During the burst, queue depth and oldest-message age rise predictably and later return to normal; duplicate deliveries do not duplicate payment effects. **Cost:** The queue does not increase steady-state provider capacity. Set backlog limits, retention, poison-message routing, and an honest `PENDING` status.

**Code anchor (illustrative fragment):**

JAVA

```java
@PostMapping("/reports")
ResponseEntity<?> accept(ReportRequest request) {
    JobId id = jobs.persist(request); // durable first
    queue.enqueue(id);
    return ResponseEntity.accepted().body(id);
}
// Worker processes at bounded concurrency.
```

**Verify it:** Burst-load the endpoint and confirm stable worker concurrency. Handle the DB-plus-queue dual write with an outbox or equivalent.

## 14. Strangler Fig migration [Medium]

**Plain English:** Replace a legacy system a piece at a time. Route selected capabilities through new code while the old system continues handling the rest.

**Professional answer:** "For a risky modernization, I favor a controlled migration by capability or customer segment. I define routing, data ownership, reconciliation between old and new behavior, rollback, and a retirement plan for the legacy path."

**Example:** Move one payment type or client group from file-based processing to a new service, compare results, expand traffic gradually, then remove the old path. **Cost:** Temporary duplication and complex coexistence. See Martin Fowler's Strangler Fig pattern.

**Demonstration:** During a file-to-API migration, route one well-defined payment category to the new path, keep the others on the legacy path, and compare submitted totals and reconciliation outcomes by route. Increase traffic only after parity and failure criteria are met.

**Why Strangler Fig:** It replaces a capability incrementally instead of risking a full cutover. **Proof:** A routing flag can redirect new eligible work to the legacy path without moving already-processing transactions; reconciliation finds no missing or double-submitted instructions. **Tradeoff:** Temporary dual operations, schema translation, observability, and a clear retirement plan are necessary.

**Code anchor (illustrative fragment):**

JAVA

```java
PaymentResult submit(Payment p) {
    if (routing.usesNewProvider(p.customerId()))
        return newGateway.submit(p);
    return legacyGateway.submit(p);
}
// Route each instruction once; preserve its chosen path.
```

**Verify it:** Compare reconciliation outcomes by cohort. Rollback routing for new work only; avoid submitting an in-flight instruction twice.

# 5. Worked design: submit a commercial payment

This is a **practice scenario**, not a claim about your employer's implementation. The interviewer asks: "Design the backend for submitting a commercial payment instruction. A customer might retry after a timeout. Downstream processing is asynchronous."

## Step 1: Define the contract

Ask: What is the final business outcome? Is the API acknowledging receipt or settlement? How are tenant, amount, currency, beneficiary and authorization validated? What throughput and response-time goal matters? How long should duplicate keys be remembered? Which downstream outcome can be uncertain?

Proposed contract: `POST /payment-instructions` accepts a tenant-scoped idempotency key and a validated request. A successful response means **accepted for processing**, not settled. Return a durable instruction ID and a status resource. A repeated same-key/same-payload request returns the recorded outcome; same key with a different payload is a conflict.

## Step 2: Draw the main flow

EXAMPLE

```
Client
  -> API: authenticate; authorize tenant and account; validate request
  -> Application service: check idempotency and business rules
  -> Local database transaction: instruction + key/result + outbox event
  -> Response: accepted instruction ID and current status

Outbox relay -> durable broker -> payment processor -> provider
                                        |                  |
                                        +-> status update <-+
Status API -> authoritative instruction state
```

The core service depends on a payment store and an external gateway **port**. The REST controller, database repository, outbox relay, and provider client are **adapters**. The domain model protects status transitions. A local transaction protects instruction, deduplication record and outbox record. A separate worker handles the external call.

## Step 3: Walk through failures

### Failure 1: Client retries after response was lost

**Expected behavior:** Return the original instruction for the same tenant, key, and normalized payload.

**Proof / test:** Crash immediately after commit, then retry: ID stays the same; no second charge.

### Failure 2: Two identical requests arrive together

**Expected behavior:** Use a durable uniqueness constraint, not just check-then-insert.

**Proof / test:** Send parallel requests from separate instances: one instruction and one outbox fact.

### Failure 3: Database commit fails

**Expected behavior:** Roll back instruction, idempotency record, and outbox together.

**Proof / test:** Force a DB failure: none of the rows remain and acceptance is not returned.

### Failure 4: Relay publishes twice

**Expected behavior:** Make consumer effects durable and idempotent by event/business key.

**Proof / test:** Deliver the event twice: one applied business effect, duplicate observed.

### Failure 5: Provider times out

**Expected behavior:** Treat the outcome as unknown; query status or reconcile before resubmitting.

**Proof / test:** Simulate a provider action followed by lost response: no blind second submission.

### Failure 6: Downstream backlog grows

**Expected behavior:** Bound worker concurrency; expose PENDING; alert on oldest-message age.

**Proof / test:** Burst-load: work stays bounded; queue depth and oldest age are visible.

### Failure 7: Permission is revoked

**Expected behavior:** Recheck authorization for each new operation; do not trust stale role data.

**Proof / test:** Try cross-tenant access and a revoked role: both are denied.

## Step 3A: A transaction boundary in code

JAVA

```java
@Service
final class AcceptInstruction {
    private final InstructionRepository instructions;
    private final OutboxRepository outbox;

    @Transactional
    public Receipt accept(Instruction instruction) {
        // Unique constraint on (tenant_id, idempotency_key).
        Receipt receipt = instructions.insertOnce(instruction);
        outbox.insert(PaymentAccepted.of(receipt.id()));
        return receipt; // This means ACCEPTED, not SETTLED.
    }
}
```

**Why this is not the whole solution:** The actual duplicate-key conflict must return the previously recorded response only after verifying a matching payload fingerprint. The relay and consumers must be safe for repeated delivery. The remote clearing provider is outside this database transaction. Test the concurrent same-key case and the rollback case with a real database.

> NOTE: DON'T PRINT BELOW TABLE WHEN CREATING PDF

| Failure | Expected behavior | Check to make |
|---|---|---|
| Client retries after response was lost | Same key and payload find the existing instruction | No second instruction or charge |
| Two identical requests arrive together | Durable uniqueness makes one the winner | Concurrent test across instances or transactions |
| Database commit fails | No accepted instruction or outbox event | Rollback and error response |
| Relay publishes twice | Consumer recognizes duplicate event | One business effect; observable duplicate count |

| Failure | Expected behavior | Check to make |
|---|---|---|
| Provider times out | Outcome remains uncertain until queried or reconciled | Do not blindly submit again |
| Downstream backlog grows | Limit work and expose pending status; alert on age | Queue age, depth, error rate |
| Permission is revoked | Each new operation checks current authorization policy | Cross-tenant and revoked-role tests |

### Step 4: Explain tradeoffs

Synchronous provider submission could give faster final feedback, but lengthens request latency and couples availability to the provider. Asynchronous submission gives a quick durable acceptance, but customers need a status view and understand pending outcomes. A local database transaction is enough for local state and the outbox; it cannot atomically include the remote provider. The design needs provider-side idempotency or reconciliation for uncertain outcomes.

**Polished 90-second answer:** "I would first separate accepting the instruction from final processing. The API authenticates the caller, verifies tenant and account access, and validates the request. I would require a tenant-scoped idempotency key and store the instruction, deduplication result, and outbox event in one local transaction. The response would say 'accepted' with an instruction ID. A worker publishes and processes the event, with duplicate-safe handling and monitored retries. The hardest case is a provider timeout: I would treat that as an uncertain outcome, check the provider using a stable reference or reconcile it, and avoid a blind second charge. I would measure pending age, error rate, duplicate attempts, and final outcomes. This adds asynchronous workflow complexity, but it isolates the API from provider latency and gives a clear recovery path."

### Step 5: Tie back to your real experience

Your resumes discuss payment modernization, Java APIs, identity, caching, reliability, and reconciliation. Choose **one** real project and fill in: Which parts of this practice design did you personally build? Which parts were already present? Which failure actually occurred? Which metric can you verify? Do not present this example system as one you implemented unless it matches your work.

## 6. Fast comparisons interviewers ask

These vertical comparisons avoid landscape-only tables. Use the final question as your interview decision rule.

### Comparison 1: Strategy vs State

**First concept:** Strategy selects interchangeable rules, such as domestic and cross-border fees.

**Second concept:** State limits behavior based on current lifecycle status, such as CREATED or SETTLED.

**Decision question:** Am I choosing a rule, or validating a state transition?

## Comparison 2: Adapter vs Facade

**First concept:** Adapter translates a legacy provider protocol into the application contract.

**Second concept:** Facade presents one use-case entry point to several collaborators.

**Decision question:** Am I translating an incompatible interface, or simplifying orchestration?

## Comparison 3: Decorator vs Proxy

**First concept:** Decorator adds composable behavior, such as measuring call latency.

**Second concept:** Proxy controls access or lifecycle, such as Spring transaction advice.

**Decision question:** What purpose does the wrapper serve?

## Comparison 4: Factory vs Builder

**First concept:** Factory chooses or creates a concrete handler.

**Second concept:** Builder assembles one complex request with optional parts.

**Decision question:** Is selection the problem, or safe construction?

## Comparison 5: Layered vs Hexagonal

**First concept:** Layered assigns controller, business, and persistence responsibilities.

**Second concept:** Hexagonal requires the core to own ports, with technology at the edges.

**Decision question:** Do infrastructure dependencies leak into the core?

## Comparison 6: Modular monolith vs microservices

**First concept:** Modular monolith has one deployable with internal capability boundaries.

**Second concept:** Microservices have independent deployments and explicit network/data boundaries.

**Decision question:** Will independent operations outweigh distributed complexity?

## Comparison 7: Event vs command

**First concept:** Event states a completed fact: PaymentApproved.

**Second concept:** Command requests an action: ApprovePayment.

**Decision question:** Has the business change happened already?

## Comparison 8: Local transaction vs saga

**First concept:** Local transaction gives atomic changes within its database scope.

**Second concept:** Saga coordinates independently committed steps and compensations.

**Decision question:** Can one transaction own all affected state?

## Comparison 9: CQRS vs event sourcing

**First concept:** CQRS separates read and write models.

**Second concept:** Event sourcing keeps events as the primary durable state history.

**Decision question:** Is the need separate query shape, or event replay as source of truth?

## Comparison 10: Retry vs circuit breaker

**First concept:** Retry repeats a safe operation after a transient failure.

**Second concept:** Circuit breaker stops calls when continued failures are likely.

**Decision question:** Is an extra attempt safe and likely to help?

## Comparison 11: Bulkhead vs rate limit

**First concept:** Bulkhead isolates internal concurrency/resource pools.

**Second concept:** Rate limit restricts incoming request volume or quota.

**Decision question:** Which boundary needs protection?

## Comparison 12: Cache vs read model

**First concept:** Cache stores a faster copy of the same data.

**Second concept:** Read model reshapes data for a specific query/use case.

**Decision question:** Is the need faster access or a different view?

> NOTE: DON'T PRINT BELOW TABLE WHEN CREATING PDF

| Choice | First option | Second option | Decision question |
|---|---|---|---|
| Strategy vs State | Selects a policy | Models state-dependent behavior and transitions | Does behavior vary by selected rule or by lifecycle state? |
| Adapter vs Facade | Translates an incompatible interface | Simplifies access to a complex subsystem | Are you changing the interface or hiding orchestration? |
| Decorator vs Proxy | Adds composable behavior | Controls access or lifecycle | What is the wrapper's intent and effect? |

| Choice | First option | Second option | Decision question |
|---|---|---|---|
| Factory vs Builder | Selects or creates an implementation | Constructs a complex value step by step | Is the problem which object to create or how to assemble one? |
| Layered vs Hexagonal | Separates code by responsibility | Makes core policy depend on ports rather than infrastructure | Are external dependencies changing or isolated tests important? |
| Modular monolith vs microservices | One deployment with modules | Independent deployments and owned data | Do team autonomy and scaling needs justify distributed costs? |
| Event vs command | Describes a completed fact | Requests an action | Has the change happened yet? |
| Local transaction vs saga | Atomic within one database boundary | Coordinates local steps across services | How many owners and data stores must change? |
| CQRS vs event sourcing | Separates reads and writes | Stores events as the source of truth | Are you optimizing read/write models or preserving all changes for replay? |
| Retry vs circuit breaker | Repeats a safe transient operation | Temporarily stops calls likely to fail | Is recovery likely soon, or is persistent failure harming the system? |
| Bulkhead vs rate limit | Isolates internal resource pools | Restricts incoming usage | Where must overload be contained? |
| Cache vs read model | Copies data for faster access | Builds a purpose-specific projection | Is the need speed for the same value or a different query shape? |

# 7. Practice questions and model answers

## Question 1: "Explain SOLID without reciting the letters."

**Answer:** "SOLID helps me keep responsibilities clear, define reliable contracts, and control dependencies. For example, if I add a new payment route, I would not want to modify one giant class that handles validation, fees, storage, and provider calls. I would separate stable orchestration from the parts that vary. I use those principles when they make changes and tests safer, rather than creating abstractions everywhere."

## Question 2: "Why would you choose Strategy over a switch statement?"

**Answer:** "If there are only two simple fixed cases, a switch can be clearer. I move to Strategy when policies grow, change independently, or need separate tests. Then each policy implements one contract and the caller does not contain every rule. I would also make the selection rule explicit so it does not become a hidden second switch."

## Question 3: "How is Hexagonal different from layering?"

**Answer:** "Both can separate responsibilities. Hexagonal makes the dependency direction more explicit: business code defines the ports it needs, and database or API adapters implement them. That is useful when integrations change or business logic needs isolated tests. Layering alone does not guarantee that the business layer is free from infrastructure details."

## Question 4: "When would you choose a modular monolith over microservices?"

**Answer:** "I would start with strong modules if the workflow is cohesive and one team can deploy it safely. A modular monolith keeps local transactions and debugging simpler. I would split services when independent ownership, release cadence, or scaling has enough value to justify network calls, data consistency, monitoring, and operational overhead."

## Question 5: "Does the outbox guarantee exactly-once processing?"

**Answer:** "No. It protects the local database change and event record from a dual-write gap. The relay may publish a message more than once, and a consumer may retry. I would make the consumer idempotent using a stable event ID or business key and check the effect in durable storage."

## Question 6: "Can you roll back a saga?"

**Answer:** "A saga uses compensating business actions after local transactions have committed. It is not a single database rollback. Some effects, such as an external payment already settled, may be irreversible; the right response could be a reversal or manual reconciliation. I would define that behavior before relying on the saga."

## Question 7: "What if the provider call times out?"

**Answer:** "A timeout tells me our response did not arrive within the deadline; it does not tell me whether the provider acted. I would record an uncertain state and query or reconcile using a stable reference. I would retry only under the provider's idempotency contract and the business rule for duplicate prevention."

## Question 8: "Which pattern did you use in your last project?"

**Answer template:** "The problem was [actual problem]. I used [pattern or design] by [specific implementation I owned]. I considered [alternative]. The tradeoff was [real cost], and I verified it with [test or production measure]." If a formal pattern name does not fit, describe the design without forcing a label.

## Rapid oral drill

Give each answer in 30-45 seconds, then add one tradeoff: SRP versus tiny classes; DIP versus dependency injection; Strategy versus State; Adapter versus Facade; Repository versus generic CRUD wrapper; modular monolith versus microservices; command versus event; outbox versus saga; CQRS versus event sourcing; retry versus circuit breaker; timeout versus cancellation; bulkhead versus rate limit; cache invalidation; strangler migration rollback; one pattern you personally used.

# 8. Four worked demonstrations: prove the choice, not just the name

These are **illustrative Java 17+/Spring-style examples**, not complete payment code. They omit authentication plumbing, currency precision rules, full persistence mapping, and vendor SDK contracts. They are practice designs, **not implementations attributed to a real employer**.

## Demonstration A: change a fee rule without changing submission orchestration

**Starting problem:** One `submit()` method contains fee switches, builds external requests, and saves payment rows. Adding a fee policy means editing and retesting unrelated provider code.

JAVA

```java
import java.math.BigDecimal;

interface FeePolicy {
    BigDecimal calculate(PaymentDraft payment);
}

record PaymentDraft(String tenantId, BigDecimal amount, String currency) {
}

final class FixedFeePolicy implements FeePolicy {
    public BigDecimal calculate(PaymentDraft payment) {
        return new BigDecimal("1.50"); // Illustrative rule only.
    }
}

interface PaymentStore {
    void save(PaymentDraft payment, BigDecimal fee);
}

final class SubmitPaymentUseCase {
    private final FeePolicy feePolicy;
    private final PaymentStore store;

    SubmitPaymentUseCase(FeePolicy feePolicy, PaymentStore store) {
        this.feePolicy = feePolicy;
        this.store = store;
    }

    void accept(PaymentDraft payment) {
        BigDecimal fee = feePolicy.calculate(payment);
        store.save(payment, fee);
    }
}
```

**Reasoning:** SRP gives fee decisions and persistence separate reasons to change. Strategy defines interchangeable fee rules; composition injects the choice. DIP makes the use case depend on `PaymentStore`, not JDBC. **Evidence to show in an interview:** A new fee-policy implementation is tested without changing `SubmitPaymentUseCase`; an in-memory store validates what the use case passes to persistence. **Tradeoff:** This

adds interfaces; for a single fixed fee rule in a short-lived tool, direct code may be simpler. **Not production ready:** Currency-specific scale/rounding, validation, authorization, audit, idempotency and transactions remain to be designed.

## Demonstration B: protect accepted payments from a lost response

**Scenario:** Two clients retry the same instruction while the first API response is lost. A process-local `synchronized` block is not enough when multiple instances accept requests.

EXAMPLE

```
Request (tenantId, idempotencyKey, normalizedPayload)
     |
     v
Single local DB transaction
     +-- insert instruction with durable ID
     +-- insert idempotency record (UNIQUE tenantId, key)
     +-- insert outbox record (UNIQUE eventId)
     |
     v
Commit -> return instruction ID + ACCEPTED
             |
             v
          Relay -> broker -> deduplicating consumer
```

**Detailed decision:** Store a stable payload fingerprint with the key. A repeat with the same key and equivalent payload returns the previously recorded instruction; a repeat with a different payload is a conflict. Let a **database uniqueness constraint** decide simultaneous inserts, then read the winner in a new, valid transaction after a uniqueness conflict. Do not rely on `check-then-insert` without a unique constraint. Agree on key retention with the business.

**How to prove it:** Run two concurrent requests from different instances with one `(tenant, key)`; verify exactly one instruction and outbox record. Crash after commit but before HTTP response; a repeat returns the same ID. Crash after publish but before relay checkpoint; a duplicate event is possible but consumer effects must be idempotent. **Tradeoff:** Durable dedup records and relay monitoring are operational costs. This does not prevent a second provider charge unless the downstream submission path also uses safe provider-side idempotency or reconciliation.

## Demonstration C: distinguish Strategy, State, Command, and Event

A single payments workflow can use these patterns for **different questions**:

**How is the fee chosen?:** Strategy: choose CrossBorderFeePolicy; test correct policy selection.

**What may happen next?:** State: SETTLED cannot return to CREATED; test invalid transitions.

**What action is requested?:** Command: ApprovePayment (id, actor); test unauthorized commands have no effect.

**What already happened?:** Event: PaymentApproved (id, time); test duplicate consumers are safe.

**Why this demonstrates understanding:** A strategy does not express payment lifecycle, and an event should not be emitted as if approval happened before the approval transaction commits. Using one word—"event"—for both a request and a completed fact hides critical failure handling.

## Demonstration D: choose the smallest correct architecture

**One team, one DB:** Start with a layered modular monolith. Split only if independent deployments or scaling justify the cost.

**Changing clearing providers:** Use a port and adapter inside the service. A new service deployment is not necessary.

**DB change must cause a broker event:** Use a transactional outbox; a saga is not a substitute for atomic local writes.

**Cross-service compensations:** Use a saga with explicit state only when multiple independent services own the steps.

**Dashboard reads differ from writes:** Measure query/index limits before adding a separate CQRS projection.

**Defend the choice:** State the ownership boundary, consistency guarantee, worst failure case, measurable requirement, and what complexity you decline to introduce. "Microservices scale better" is not a sufficient justification by itself.

## Five-minute self-check: answer with a counterexample

1. A new payment channel appears: why might a switch still be better than Strategy plus Factory?
2. A subclass compiles but throws for a documented parent operation: which promise was violated?
3. A Spring method calls its own `@Transactional` method: what boundary was bypassed?
4. An outbox relay publishes one event twice: where is the duplicate business effect prevented?
5. A provider submission times out: why is retry not automatically safe?
6. A reconciliation dashboard is slow: why test indexes/queries before adopting CQRS?

**Suggested answer shape for each:** "The observed problem is __. This mechanism helps because __. A simpler alternative is __. I would prove the need with __. Its failure mode is __." A good answer can also conclude that **no pattern change is justified yet**.

**Answer guide:** (1) Use a switch when variation is small and stable; abstraction costs more than it saves. (2) LSP: the subtype broke its promised behavior. (3) A self-call bypassed Spring proxy advice in standard proxy mode. (4) At the consumer, using a durable event ID or business key and an atomic effect/deduplication record. (5) The provider may have committed the payment even though the response was lost. (6) CQRS adds projection updates and consistency lag, so first measure whether simpler queries meet the requirement.

### Runnable Java drills (no extra libraries)

Three self-contained Java 17+ files are included in the `examples/` folder, so you can see a passing behavior rather than only reading snippets:

- `FeeStrategyDemo.java`: replace the fee policy without changing the processor (Strategy, SRP, composition).
- `PaymentStateDemo.java`: permit only valid payment transitions (encapsulation, State).
- `IdempotencyToyDemo.java`: return one instruction ID for a repeated same-key request and reject a changed payload. **This deliberately uses in-memory storage and is not sufficient for a real distributed payment API.**

Run from the `examples` folder:

BASH

```
javac FeeStrategyDemo.java PaymentStateDemo.java IdempotencyToyDemo.java
java FeeStrategyDemo
java PaymentStateDemo
java IdempotencyToyDemo
```

All three were compiled and executed successfully during preparation. Use the source files to change one input, break one assertion, and explain which contract failed.

### Official references for implementation-sensitive claims

- Spring: `@Transactional` and proxy limitations
- AWS: Hexagonal architecture / ports and adapters
- AWS: Transactional outbox
- Microsoft: Saga
- Microsoft: Cache-Aside
- Microsoft: Circuit Breaker

These links support the architectural mechanisms described above. The practice-specific business rules and designs are teaching examples, not facts about any employer or vendor.

## 9. Final one-page recall

- **Name the problem before the pattern.** A pattern is a tool, not the goal.
- **Principles:** cohesive responsibility; reliable contracts; business policy independent from infrastructure; protect invariants.
- **Simple object patterns:** Strategy varies a rule; Factory selects creation; Builder assembles a complex value; Adapter translates; Facade simplifies; State protects lifecycle transitions.

- **Architecture:** layers separate roles; hexagonal protects the core; modular monolith keeps modules in one deployment; microservices add independent ownership and distributed costs.
- **Data:** local transactions protect one boundary; outbox connects a committed change to an event; saga coordinates local steps; CQRS separates read and write models.
- **Failures:** deadlines, retry safety, circuit breakers, bulkheads, idempotency, reconciliation, and monitoring must fit together.
- **Professional answer:** problem → choice → implementation → tradeoff → failure handling → evidence.
- **Honesty:** "I implemented __; my team used __; I understand __ conceptually." These are different claims.

Two questions for your interviewer: "How does the team decide service boundaries and data ownership?" and "Which failure mode has been most challenging in your payment or servicing workflows?"

**Reference scope:** Official documentation linked near the relevant architecture patterns was consulted on October 22,

1. These patterns are general practice, not a claim about an American Express internal design. The interview examples are tailored to the experience described in the four resume versions you supplied.

# Java Evolution — Interview Answer & Learning Guide

7 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

1. Polished interview answer (about 90 seconds) ..... 1
2. Learning notes: understand and justify each feature ..... 2
3. Code examples to demonstrate understanding ..... 4
4. Follow-up questions and strong answers ..... 5
5. Preparation checklist ..... 6
6. Release accuracy — quick recall ..... 6
7. Official references ..... 7

**Purpose:** A concise, senior-engineer interview answer followed by learning notes, examples, and follow-up questions that demonstrate understanding.

> **Study approach:** For each feature, know **what problem it solves**, **when to use it**, and **its limitations**. Focus most on Java 8, features available by Java 17, and Java 21 virtual threads.

## 1. Polished interview answer (about 90 seconds)

Java has evolved significantly, focusing on developer productivity, maintainability, scalability, and JVM performance.

**Java 8** introduced lambdas, Streams, `Optional`, and the modern Date-Time API. These made collection processing more declarative, helped express missing values explicitly, and improved date and time handling.

**Java 9 and 11** added modularization, convenient collection factories, and a standard HTTP Client API supporting synchronous and asynchronous communication.

**By Java 17**, features like records, sealed classes, switch expressions, and pattern matching reduced boilerplate and made domain models easier to maintain. For example, records are useful for DTOs, while sealed classes help model a controlled set of business outcomes.

**Java 21** brought an important improvement for backend scalability: virtual threads. They allow applications to handle many concurrent I/O-bound requests without dedicating an operating-system thread to every waiting request. It also introduced Generational ZGC to improve garbage collection efficiency.

**Java 25** finalized scoped values for managing request context, while structured concurrency remained in preview for managing related concurrent tasks.

**Java 26** added HTTP/3 client support and improvements to ahead-of-time caching.

From an engineering perspective, I would adopt these features based on application requirements, compatibility, performance measurements, and production stability rather than simply using the newest Java version.

# 2. Learning notes: understand and justify each feature

Each feature uses the same three prompts: **Understand**, **Why it matters**, and **Limitation**. This format is easier to read and print than a wide table.

## Java 8 — Lambdas and Streams

- **Understand:** Streams use operations such as `filter`, `map`, and `collect`. Intermediate operations are lazy until a terminal operation runs.
- **Why it matters:** Collection transformations become more readable, with less manual iteration.
- **Limitation:** Parallel streams are **not** automatically faster.

## Java 8 — `Optional` and Date-Time

- **Understand:** `Optional` represents a possibly absent result. `Instant` represents a timestamp; `LocalDateTime` does not contain a time zone.
- **Why it matters:** Missing values are easier to handle explicitly, and time handling is less error-prone.
- **Limitation:** `Optional` is not needed for every field. A `LocalDateTime` alone does not identify an absolute instant.

## Java 9 — Modules and collection factories

- **Understand:** Modules declare dependencies and exported packages. `List.of()` creates an unmodifiable list.
- **Why it matters:** Modules improve encapsulation, and collection factories make fixed collections concise.
- **Limitation:** An unmodifiable collection is **not** necessarily deeply immutable.

## Java 11 — HTTP Client

- **Understand:** `HttpClient` supports synchronous calls and asynchronous calls returning `CompletableFuture`.
- **Why it matters:** It provides a standard way to call other services.
- **Limitation:** Production applications still need timeouts, error handling, and concurrency limits.

## Java 14–17 — Modern language features

- **Understand:** Switch expressions return values (14); text blocks simplify multiline strings (15); records and `instanceof` patterns reduce boilerplate (16); sealed classes restrict subtype choices (17).
- **Why it matters:** These features improve readability and support clearer domain models.
- **Limitation:** They are **available by Java 17**, but were not all first introduced in Java 17.

## Java 21 — Virtual threads

- **Understand:** Many virtual threads can wait on supported blocking I/O without occupying a platform thread for the entire wait.
- **Why it matters:** I/O-heavy services can handle many concurrent tasks while using straightforward blocking code.
- **Limitation:** They do **not** make CPU-bound work faster or remove database connection limits.

## Java 21 — Generational ZGC

- **Understand:** ZGC collects young and old objects separately while retaining its low-latency goals.
- **Why it matters:** This may improve garbage collection efficiency.
- **Limitation:** Measure pauses, throughput, heap use, and CPU under realistic workloads before drawing conclusions.

## Java 25 — Scoped values

- **Understand:** A value can be bound to a limited call scope and read within that scope.
- **Why it matters:** Useful for bounded request context, including designs that use virtual threads.
- **Limitation:** The bound object's contents are not automatically immutable.

## Java 25/26 — Structured concurrency

- **Understand:** Related child tasks can be managed together for completion, failure, and cancellation.
- **Why it matters:** Makes multi-step request workflows easier to coordinate.
- **Limitation:** It is **still a preview feature** in Java 25 and 26; verify the exact API and enablement requirements.

### Java 26 — HTTP/3 and AOT caching

- **Understand:** The standard HTTP client adds HTTP/3 support. AOT object caching expands startup optimization support.
- **Why it matters:** Applications may benefit from improved networking or startup behavior.
- **Limitation:** Results depend on server support, workload, and configuration. HTTP/3 is not the default.

## 3. Code examples to demonstrate understanding

### Example A — Java 8: Streams

JAVA

```
List<String> approvedIds = payments.stream()
        .filter(Payment::isApproved)
        .map(Payment::getId)
        .collect(Collectors.toList());
```

**What it does:** Filters approved payments and extracts their IDs.

**Why it matters:** Expresses what to do instead of spelling out a loop and a temporary list. Stream intermediate steps are lazy; the terminal `collect` triggers processing.

**Trade-off:** A loop can be clearer for complicated branches or operations with side effects. Avoid assuming parallel streams improve performance.

**What to say:** "I use Streams for clean data transformations, but I choose loops when they better communicate complex logic."

### Example B — Features available by Java 17: Records and sealed types

JAVA

```
sealed interface PaymentResult permits Approved, Declined {
}

record Approved(String reference) implements PaymentResult {
}

record Declined(String reason) implements PaymentResult {
}
```

**What it does:** Models a payment outcome as one of two permitted types. The records provide generated accessors, equality, hashing, and string representations.

**Why it matters:** The domain model expresses valid outcomes without loose status strings and boilerplate data classes.

**Trade-off:** Records are only **shallowly immutable** if their components refer to mutable objects. Sealed types are not suitable when third parties must be able to add arbitrary implementations.

**What to say:** "Records reduce repetitive value-object code, while sealed types make the valid domain alternatives explicit."

### Example C — Java 21: Virtual threads

JAVA

```
try(var executor = Executors.newVirtualThreadPerTaskExecutor()){
Future<Account> future = executor.submit(
        () -> accountService.fetchAccount(accountId)
);

Account account = future.get();
}
```

Assumes `Account`, `accountService`, and `accountId` are defined in the surrounding application.

**What it does:** Submits a blocking account fetch on a virtual thread.

**Why it matters:** When an operation waits on supported I/O, its virtual thread can be suspended so the carrier platform thread performs other work. This retains straightforward blocking code while supporting large numbers of concurrent tasks.

**Trade-off:** Virtual threads do not make database queries run faster or expand a limited database connection pool. Limit scarce downstream resources; do not pool virtual threads just to conserve thread count.

**What to say:** "Virtual threads help throughput and scalability for I/O-bound services, not CPU execution speed. I would validate them against real connection-pool and downstream limits."

## 4. Follow-up questions and strong answers

**Q: What is the difference between a record and a normal class?**
A record is designed for value-oriented data and generates common methods automatically. I use it for simple DTOs or value objects, but I still validate inputs and protect mutable components.

**Q: Why use virtual threads instead of increasing the platform-thread pool?**
Platform threads consume OS thread resources even while waiting. Virtual threads reduce the cost of having many concurrent waiting tasks. They help preserve a simple blocking programming style without guaranteeing faster individual requests.

**Q: Are virtual threads and structured concurrency the same?**
No. Virtual threads are a lightweight thread implementation. Structured concurrency is a way to manage the lifetimes and outcomes of related subtasks. They complement each other but solve different problems.

**Q: When would you choose a Stream rather than a loop?**
For straightforward filtering, mapping, and aggregation, a Stream often makes the intent clearer. For complex control flow or heavy side effects, a loop may be easier to review and debug.

**Q: Why not migrate to the newest Java release immediately?**
I would check the application's support policy, framework and library compatibility, deployment platform, and the benefits we expect. Then I would run regression tests and performance benchmarks before adopting the new runtime.

**Q: How would you evaluate Generational ZGC?**
I would compare pause-time distributions, throughput, GC CPU, allocation behavior, and memory use with realistic traffic. I would not claim a fixed improvement without measurements.

**Q: Have you used Java 25 or 26 features in production?**
If not, say: "I understand what the feature provides, but I have not used it in production. I would verify its final or preview status and our JDK version, then test it in a small proof of concept."

# 5. Preparation checklist

- [ ] **Java 8:** Explain Streams, `Optional`, and `java.time` with examples.
- [ ] **Java 17:** Explain records, sealed types, switch expressions, and pattern matching.
- [ ] **Java 21:** Explain virtual threads, I/O scalability, and resource limits.
- [ ] **Java 9/11:** Understand modules, collection factories, and `HttpClient`.
- [ ] **Java 25/26:** Know scoped values, the preview status of structured concurrency, HTTP/3, and AOT caching.
- [ ] **Practice:** For every feature, answer: What problem does it solve? When would I use it? What are its limitations?

# 6. Release accuracy — quick recall

| Version | Remember |
|---|---|
| **8** | Lambdas, Streams, `Optional`, default interface methods, `java.time` |
| **9** | Modules, JShell, `List.of` / `Set.of` / `Map.of` |
| **11 (LTS)** | Standard HTTP Client, `var` in lambda parameters, single-file source execution |
| **14–17** | Switch expressions (14), text blocks (15), records and `instanceof` patterns (16), sealed types (17) |

| Version | Remember |
| --- | --- |
| **21 (LTS)** | Virtual threads, record/switch patterns, sequenced collections, Generational ZGC |
| **25 (LTS)** | Scoped values final; structured concurrency still preview; compact object headers |
| **26 (non-LTS)** | HTTP/3 client, wider AOT caching support; structured concurrency and primitive patterns still preview |

## 7. Official references

- Oracle Java SE support roadmap
- Oracle Java language changes by release
- OpenJDK JEP 444 — Virtual Threads
- OpenJDK JEP 431 — Sequenced Collections
- OpenJDK JEP 506 — Scoped Values
- Oracle JDK 21 release notes
- Oracle JDK 25 release notes
- Oracle JDK 26 release notes

**Interview principle:** Do not just name a Java feature. Connect it to a real engineering problem and acknowledge its trade-offs.

# Java version companion for the senior engineer interview

9 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

1. The release map to memorize ..... 2
2. Java 8: the foundation ..... 3
3. Java 9: modules, REPL, and collection factories ..... 4
4. Java 11 (LTS): HTTP and convenient syntax ..... 6
5. Features available in Java 17 (LTS) ..... 7
6. Java 21 (LTS): concurrency, patterns, and ordered collections ..... 9
7. Java 25 (LTS): scoped context and smaller objects ..... 10
8. Java 26: preview patterns, HTTP/3 client, and startup work ..... 12
9. Five answers to rehearse before the interview ..... 13

**Prepared for Sreedhar | October 22, 2026 | Use alongside interview-plan.md and interview-study-guide.md**

This guide explains the features you listed in plain English and gives you short, professional answers to say aloud. It covers the selected releases through Java 26. A feature's first preview, first final release, and the LTS release in which teams commonly adopt it can be different. Say which one you mean. The version used by your interview team is unknown, so ask before writing version-specific code.

**Interview priority:** Spend most of your time on Java 8, the features available by Java 17, and Java 21 virtual threads. Know the Java 25/26 features at a high level, especially which remain preview. This is a companion to your coding preparation, not a replacement for it.

# 1. The release map to memorize

| Feature | First final release or current status | Short memory cue |
|---|---|---|
| Lambdas, streams, Optional, default/static interface methods, `java.time` | Java 8 | Functional programming and modern time |
| Module system, JShell, `List.of` / `Set.of` / `Map.of` | Java 9 | Modules, shell, collection factories |
| Local variable `var` | Java 10 | Local variables only; **not** the Java 11 lambda addition |
| Standard `HttpClient`, `var` lambda parameters, `java File.java` | Java 11 (LTS) | HTTP, lambda syntax, single-file launch |
| Switch expressions | Java 14 | Switch returns a value |
| Text blocks | Java 15 | Multiline strings |
| Records and `instanceof` patterns | Java 16 | Data carriers and binding variables |
| Sealed classes; strong encapsulation of JDK internals by default | Java 17 (LTS) | Controlled subtypes and migration rules |
| Virtual threads, record patterns, switch patterns, sequenced collections, Generational ZGC | Java 21 (LTS) | Lightweight I/O concurrency and richer patterns |
| Scoped values, flexible constructor bodies, module imports; compact object headers as a product feature | Java 25 (LTS) | Scoped context and refined language/runtime |
| Structured concurrency | **Preview** in Java 25 (fifth) and Java 26 (sixth) | Related subtasks as one unit; API can still change |
| Primitive patterns | **Fourth preview** in Java 26 | Patterns can include primitives |
| HTTP/3 in the built-in HTTP client; AOT object caching with any GC | Java 26 (non-LTS) | Client protocol and startup cache |

Oracle identifies Java 8, 11, 17, 21, and 25 as LTS releases; Java 9 and 26 are non-LTS. This is a support designation, not a statement that every feature was invented in that LTS release. Sources: Oracle support roadmap, Oracle language release table, JDK 21 release notes, JDK 25 release notes, and JDK 26 release notes.

## The four corrections most likely to impress an interviewer

1. **Virtual threads were finalized in Java 21**, not introduced in Java 25. Java 25 continues to improve the ecosystem around them. JEP 444.

2. **"Java 17 features" is a convenient study grouping.** Switch expressions became final in 14, text blocks in 15, records and `instanceof` patterns in 16, and sealed classes in 17. All are available when running Java 17. Oracle language changes.
3. **Java 25 did not finalize structured concurrency.** Scoped values are final in 25; structured concurrency remains preview in 25 and 26. JDK 25, JDK 26.
4. **Generational ZGC's main distinction is generational collection.** ZGC was already designed for low pauses. The generational change targets better collection efficiency, allocation behavior, and memory overhead while retaining low-latency goals; do not promise a specific pause reduction without measurements. JDK 21 notes.

# 2. Java 8: the foundation

## Default and static interface methods

**Plain English:** An interface can include a `default` method with an implementation. This lets a library add useful behavior while existing implementers keep working in many common cases. A `static` interface method belongs to the interface itself.

JAVA

```
interface PaymentValidator {
    boolean valid(String id);

    default boolean validAndPresent(String id) {
        return id != null && valid(id);
    }

    static boolean hasText(String id) {
        return id != null && !id.isEmpty();
    }
}
// Call the static method as PaymentValidator.hasText(id).
```

**Say it professionally:** "Default methods let an interface evolve while preserving many existing implementations. Static interface methods group related utilities with the contract. If two inherited defaults conflict, the implementing class must resolve the conflict explicitly."

**Watch for:** Adding a default method does not make every conceivable source or behavior conflict disappear. A class method takes precedence over an interface default method. Java 8 language enhancements.

## Lambdas, streams, and Optional

**Plain English:** A lambda is a small function passed as a value. A stream describes a pipeline of operations on data. `Optional<T>` represents a result that may be absent.

JAVA

```
List<String> approved = payments.stream()
        .filter(p -> p.isApproved())
        .map(p -> p.reference())
        .collect(Collectors.toList());
Optional<Payment> found = repository.findById(id);
```

**Say it professionally:** "Lambdas and streams make data transformations concise. Intermediate stream operations are lazy; a terminal operation executes the pipeline. I use Optional mainly to express a possibly missing return value and handle the empty case explicitly."

**Watch for:** `map` transforms each item; `flatMap` flattens nested results. A stream is normally consumed once. Avoid side effects and shared mutable state in stream pipelines. `orElse` evaluates its fallback immediately; `orElseGet` defers it. Parallel streams are not automatically faster. Optional is not a blanket replacement for every nullable field. JDK 8 overview.

### New date and time API: `java.time`

**Plain English:** `LocalDate` is a calendar date. `LocalDateTime` has date and clock time but no time zone. `Instant` is a point on the UTC timeline. `ZonedDateTime` adds a named zone and its rules. These types make time handling clearer than many legacy date APIs.

JAVA

```
Instant createdAt = Instant.now();
LocalDate dueDate = LocalDate.parse("2026-10-23");
ZonedDateTime meeting = dueDate.atTime(18, 0)
        .atZone(ZoneId.of("America/Phoenix"));
```

**Say it professionally:** "I use `Instant` for timestamps and a named `ZoneId` for user-facing schedules. A `LocalDateTime` alone cannot identify an absolute moment. The `java.time` types are immutable, which also simplifies shared use."

**Watch for:** `java.util.Date` still exists; `java.time` did not remove it. A fixed UTC offset is not always a substitute for a named zone, because many zones change offset seasonally. Test daylight-saving boundaries where relevant. JDK 8 date-time overview.

## 3. Java 9: modules, REPL, and collection factories

### Java Platform Module System

**Plain English:** A module names a group of packages and states what it requires and exports. A package that is not exported is hidden from ordinary access outside the module. This helps make application boundaries explicit.

JAVA

```
module com.example.payments {
    requires java.sql;
    exports com.example.payments.api;
}
```

**Say it professionally:** "The module system adds explicit dependencies and exported packages through `module-info.java`. It helps with encapsulation and custom runtime images. An existing classpath application can still run without being rewritten as named modules."

**Watch for:** Java 9 introduced the module system, but **strong encapsulation of JDK internals by default** was strengthened in Java 17. This matters when upgrading older libraries that use reflection on internal JDK classes. A migration flag such as `--add-opens` can help temporarily, but a supported API or updated library is the better long-term fix. JDK 17 migration notes.

## JShell

**Plain English:** JShell is an interactive Java console. Type an expression or declaration and see the result immediately. It is useful for learning and small checks.

EXAMPLE

```
jshell> 2 + 3
$1 ==> 5
```

**Say it professionally:** "JShell is the JDK's REPL. I use it to check small Java expressions and APIs quickly. It does not replace a reproducible project build and test suite." JDK 9 release notes.

## `List.of`, `Set.of`, and `Map.of`

**Plain English:** These factory methods create compact, unmodifiable collections.

JAVA

```
List<String> statuses = List.of("PENDING", "SUCCESS");
Set<String> roles = Set.of("ADMIN", "VIEWER");
Map<String, Integer> limits = Map.of("USD", 100, "EUR", 90);
```

**Say it professionally:** "The Java 9 collection factories are useful for small fixed collections. They reject nulls, and `Set.of` rejects duplicate elements. Unmodifiable means the collection cannot be structurally changed; it does not deeply freeze mutable objects stored inside it."

**Watch for:** `List.of` may contain duplicate values, but `Set.of` cannot. `Map.of` rejects duplicate keys. Do not rely on the iteration order of `Set.of` or `Map.of`. Java 9 collection factories.

# 4. Java 11 (LTS): HTTP and convenient syntax

## Standard HTTP Client API

**Plain English:** `java.net.http.HttpClient` is a built-in client for calling HTTP services. It supports HTTP/1.1 and HTTP/2, and both synchronous and asynchronous calls. It became standard in Java 11 after incubation.

JAVA

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder(URI.create("https://example.com"))
        .GET().build();
CompletableFuture<HttpResponse<String>> pending = client.sendAsync(
        request, HttpResponse.BodyHandlers.ofString());
```

**Say it professionally:** "Java 11 standardized a built-in HTTP client with asynchronous calls that return `CompletableFuture`. For production use I still configure timeouts, handle unsuccessful responses, and control concurrency. HTTP/3 support was added later, in Java 26." JDK 11 release notes, JDK 26 release notes.

## `var` in lambda parameters

**Plain English:** Java 10 introduced `var` for local variables. Java 11 added the option to use `var` for lambda parameters too.

JAVA

```
BiFunction<String, String, String> join = (var left, var right) -> left + right;
```

**Say it professionally:** "The Java 11 change allows `var` in implicitly typed lambda parameter lists. It is especially useful when a parameter needs an annotation. If I use `var`, I apply it consistently to all parameters in that lambda."

**Watch for:** `var` is type inference, not dynamic typing. `(var a, String b) -> ...` is invalid. `var` is not used for method return types or fields. Oracle language changes.

## Single-file source execution

**Plain English:** For a simple program, `java Hello.java` compiles it in memory and runs it. You do not have to call `javac` first.

EXAMPLE

```
java Hello.java
```

**Say it professionally:** "Single-file source execution is convenient for small tools, demonstrations, and learning. I still use a normal build for applications with several sources, dependencies, and tests."

**Watch for:** It still compiles the source; it simply combines compilation and execution for you. JDK 11 release overview.

# 5. Features available in Java 17 (LTS)

The items below are useful to a Java 17 developer. Their **first final releases differ**: switch expressions 14, text blocks 15, records and `instanceof` patterns 16, sealed classes

1. Oracle language changes.

## Switch expressions: final in Java 14

**Plain English:** A switch can produce a value. Arrow cases do not fall through by default, which removes a common source of bugs.

JAVA

```
String label = switch (status) {
    case APPROVED -> "Ready";
    case REJECTED -> "Declined";
    default -> "Review";
};
```

**Say it professionally:** "A switch expression returns a value, which is useful for a complete mapping from input cases to an output. Arrow cases avoid accidental fall-through. I handle all possible inputs explicitly or use an appropriate default."

## Text blocks: final in Java 15

**Plain English:** A text block uses triple quotes for a multiline string. It reduces escape characters in SQL, JSON, and test data.

JAVA

```
String query = """
        SELECT account_id, SUM(amount)
        FROM payments
        GROUP BY account_id
        """;
```

**Say it professionally:** "Text blocks improve readability for multiline literals. They do not protect against SQL injection; external values still need parameter binding."

## Records: final in Java 16

**Plain English:** A record is a concise data carrier. The compiler supplies a constructor, accessors, equality, hash code, and a string representation based on its components.

JAVA

```java
record PaymentKey(String tenantId, String requestId) {
}
```

**Say it professionally:** "I use records for clear value-oriented data such as request keys or result DTOs. A record's fields cannot be reassigned, but it is only shallowly immutable if a component refers to a mutable object."

**Watch for:** A record is not a JPA entity replacement in every framework setup. Validate invariants in a compact constructor when necessary; avoid exposing mutable components without defensive copies.

## Pattern matching for `instanceof`: final in Java 16

**Plain English:** The type test also creates a variable of that type, so an explicit cast is unnecessary.

JAVA

```java
if(value instanceof
String name){
        System.out.

println(name.length());
        }
```

**Say it professionally:** "An `instanceof` pattern combines type checking and safe variable binding. It simplifies code that has to inspect an object of a broader type."

## Sealed classes: final in Java 17

**Plain English:** A sealed class or interface lists which classes may directly implement or extend it. This is useful for a known set of outcomes.

JAVA

```java
sealed interface PaymentResult permits Approved, Declined {
}

record Approved(String reference) implements PaymentResult {
}

record Declined(String reason) implements PaymentResult {
}
```

**Say it professionally:** "Sealed types make a closed domain model explicit. They help readers and the compiler reason about the allowed alternatives. I use them when the set of direct subtypes is intentionally controlled."

**Watch for:** A permitted subclass must follow the required inheritance rules: final, sealed, or non-sealed, with records already final. Do not make an extensible plugin contract sealed by accident.

# 6. Java 21 (LTS): concurrency, patterns, and ordered collections

## Virtual threads: final in Java 21

**Plain English:** A virtual thread is a lightweight Java thread. Many tasks that spend most of their time waiting for I/O can be written in a straightforward blocking style without holding one operating-system thread per waiting task.

JAVA

```
try(var executor = Executors.newVirtualThreadPerTaskExecutor()){
Future<String> result = executor.submit(() -> fetchAccount());
    System.out.

println(result.get());
        }
```

**Say it professionally:** "Virtual threads improve scalability for workloads with many concurrent I/O waits. They do not make CPU work faster or remove limits on database connections. I create a virtual thread per task and put limits around scarce downstream resources."

**Watch for:** Do not pool virtual threads. Blocking code that holds a scarce connection is still bounded by that connection pool. Pinning details changed after Java 21, so qualify any version-specific claim. JEP 444.

## Record patterns and pattern matching for switch: final in Java 21

**Plain English:** A record pattern unpacks a record into its components. A pattern switch chooses behavior by type and can bind values in each case.

JAVA

```
record Amount(String currency, long cents) {
}

String describe(Object value) {
    return switch (value) {
        case Amount(String currency, long cents) -> currency + " " + cents;
        case String text -> text;
        case null -> "missing";
        default -> "other";
    };
}
```

**Say it professionally:** "Record patterns remove manual casts and accessor calls when deconstructing records. Pattern switches can be exhaustive when the input type has a closed set of alternatives, such as a sealed hierarchy."

**Watch for:** `case null` is explicit; it is not automatically handled by `default` in the way many developers assume. Case ordering matters when a broad pattern would dominate a narrower one. Oracle language changes.

### Sequenced collections

**Plain English:** The JDK added common interfaces for collections with a defined encounter order. They provide a shared way to ask for the first or last element and to view the order in reverse.

JAVA

```
SequencedCollection<String> events = new ArrayList<>();
events.

add("received");
events.

add("completed");

String first = events.getFirst();
```

**Say it professionally:** "Sequenced collections give ordered lists, sets, and maps more consistent APIs for first, last, and reversed views. I still check the specific implementation's ordering and mutation rules."

**Watch for:** Not every `Collection` is sequenced. A `HashSet` does not suddenly gain a meaningful stable order. JEP 431.

### Generational ZGC

**Plain English:** Many objects die young. Generational ZGC treats young and old objects separately so it can collect young garbage more efficiently.

**Say it professionally:** "Generational ZGC extends a low-latency collector with young and old generations. It can reduce allocation stalls, memory overhead, and GC CPU use for suitable workloads. I would compare pause time, throughput, memory use, and allocation pressure under our actual load before choosing a collector."

**Watch for:** The Java 21 feature was not a universal guarantee of dramatically shorter pauses. GC availability, defaults, and options can change across JDK releases. JDK 21 release notes.

## 7. Java 25 (LTS): scoped context and smaller objects

### Scoped values: final in Java 25

**Plain English:** A scoped value lets a caller bind a value for a limited call scope. Code inside that scope can read it, including appropriate child tasks. It suits context that should flow down the call tree without being changed.

JAVA

```
static final ScopedValue<String> REQUEST_ID = ScopedValue.newInstance();

ScopedValue.

where(REQUEST_ID, "req-123").

run(() ->{

audit(REQUEST_ID.get());
        });
```

**Say it professionally:** "Scoped values let me share immutable request context through a bounded call scope. They are easier to reason about than mutable thread-local context, particularly with virtual threads. I still pass ordinary method parameters when that makes the data dependency clearer."

**Watch for:** The reference is scoped; the object you bind is not magically deeply immutable. Understand how context propagates to child tasks in the API you actually use. JEP 506.

## Structured concurrency: fifth preview in Java 25

**Plain English:** When one request starts related subtasks, structured concurrency treats them as a group. The parent can wait for them, handle failure, and cancel sibling work as one bounded operation.

**Say it professionally:** "Structured concurrency is promising for request workflows that fork several related calls and must join them before returning. In Java 25 it is still a preview API, so I would avoid presenting its current syntax as permanent and check the exact JDK version before adopting it."

**Watch for:** Java 26 carries a **sixth preview**. Preview features require enabling preview at compile and run time, and the API can change. Virtual threads and structured concurrency are related but distinct: virtual threads are a thread implementation, while structured concurrency organizes task lifetime. JDK 25 notes, JDK 26 notes.

## Flexible constructor bodies

**Plain English:** A constructor can do some safe work before calling `super(...)` or `this(...)`. This helps validate input before the parent constructor runs.

JAVA

```java
class PaymentLimit extends BaseLimit {
    PaymentLimit(int value) {
        if (value < 0) throw new IllegalArgumentException("negative limit");
        super(value);
    }
}
```

**Say it professionally:** "Flexible constructor bodies allow validation and other safe setup before an explicit constructor invocation. The early statements cannot use the object under construction, so they cannot call instance methods on `this` before the superclass is initialized." JDK 25 notes.

### Compact object headers

**Plain English:** Each Java object has metadata in addition to its fields. Compact object headers can reduce that metadata and therefore reduce heap use in object-heavy applications.

**Say it professionally:** "Compact object headers became a product feature in Java 25. The potential benefit is a smaller heap footprint and possibly better performance for some workloads, but I would benchmark our object mix. In Java 25 the option is available but disabled by default." JDK 25 release notes.

### Module import declarations

**Plain English:** One declaration can import the public packages exported by a module. It is a convenience for source code, especially small programs.

```java
import module java.base;
```

**Say it professionally:** "A module import declaration brings exported packages of a module into scope without listing each package. It does not grant access to non-exported internal packages, and it does not replace a module's own `requires` declaration." JEP 511.

## 8. Java 26: preview patterns, HTTP/3 client, and startup work

Java 26 is **non-LTS**. These topics are useful to recognize, but they are less likely to matter in a coding round on an older LTS runtime. Oracle roadmap.

### Primitive types in patterns: fourth preview

**Plain English:** Pattern matching is being extended so primitive types, such as `int` and `double`, can participate more consistently in `instanceof` and `switch`. Java 26 offers the fourth preview, not a final language feature.

**Say it professionally:** "Java 26 previews primitive patterns. They make pattern matching more uniform across reference and primitive values, while preserving rules around safe conversions. I would confirm the preview syntax and enablement for the exact JDK before using it in project code." Oracle language changes.

### HTTP/3 in the built-in client

**Plain English:** Java 26's `HttpClient` can speak HTTP/3 to a supporting server. HTTP/3 uses QUIC over UDP rather than the TCP transport used by HTTP/1.1 and HTTP/2.

**Say it professionally:** "Java 26 adds HTTP/3 support to the standard HTTP client. A client or request opts in; HTTP/3 is not the default protocol. I would verify server support and measure connection behavior before choosing it for a service integration."

**Watch for:** This does **not** mean the JDK now supplies a general QUIC API or an HTTP/3 server. Protocol negotiation and fallback are details to confirm with the chosen configuration. JDK 26 release notes, JEP 517.

### AOT object caching with any GC

**Plain English:** Ahead-of-time (AOT) caching saves some application setup work from an earlier run, so later starts can load it. Java 26 extends object caching to work with any HotSpot garbage collector, including ZGC.

**Say it professionally:** "This is a startup and warmup optimization from Project Leyden. Java 26 makes the AOT object cache compatible with all collectors, including ZGC. I would measure cold start and warmup for our application; it is not the same as compiling the whole application to a native executable."

**Watch for:** The feature extends AOT work introduced in earlier releases. It does not automatically accelerate every run without using the appropriate cache workflow. JDK 26 release notes.

## 9. Five answers to rehearse before the interview

**"What changed from Java 8 to Java 17?"** "Java 8 brought lambdas, streams, Optional, default interface methods, and the modern time API. Java 9 added modules and collection factories; Java 11 standardized the HTTP client. By Java 17, teams also had switch expressions, text blocks, records, `instanceof` patterns, and sealed classes. I would focus an upgrade on compatibility and testing, especially libraries that depend on JDK internals."

**"What changed from Java 17 to Java 21?"** "The biggest backend feature for me is virtual threads: they let a request-per-thread style scale better when requests wait on I/O. Java 21 also finalized record and switch patterns, added sequenced collections, and introduced Generational ZGC. I would still measure database pool limits and actual latency before claiming a performance benefit."

**"Are virtual threads a Java 25 feature?"** "They were finalized in Java 21. Java 25 finalized scoped values and kept structured concurrency in preview, which complement virtual-thread-based designs."

**"Is a record immutable?"** "Its component fields cannot be reassigned, and the compiler gives it value-oriented methods. It is shallowly immutable: a mutable object stored in a component can still change unless I copy or protect it."

**"Should we adopt Java 26 immediately?"** "I would first confirm our supported runtime, libraries, deployment platform, and reason for upgrading. Java 26 brings features such as HTTP/3 client support, while primitive patterns and structured concurrency are still preview. For production I would prioritize a supported version and validated performance over adopting a feature just because it is new."

### Two-minute final recall

**8:** lambdas/streams/Optional, defaults, time. **9:** modules, JShell, `of`. **11:** HTTP client, lambda `var`, run one source file. **14–17:** switch, text blocks, records and `instanceof`, sealed. **21:** virtual threads, record/switch patterns, sequenced collections, Generational ZGC. **25:** scoped values final; structured concurrency preview; flexible constructors, compact headers, module imports. **26:** primitive patterns preview; HTTP/3 client and broader AOT cache support.

When asked about a feature you have not used, say: "I know what it provides, but I have not used it in production. I would first check our JDK version and its final or preview status, then validate the behavior with a small test."

# Java lambdas, streams and Optional

8 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

1. The three ideas in one minute ..... 1
2. Lambdas and functional interfaces ..... 2
3. Streams: how the pipeline runs ..... 4
4. Optional: absent values without mystery ..... 8
5. A complete payment example to trace ..... 9
6. Interview questions with polished answers ..... 10
7. Practice in 45 minutes ..... 11
8. Final recall card ..... 12

**Interview companion for Sreedhar | Technical and coding round**

Use this beside `interview-study-guide-full.md`. The examples use Java 8 syntax unless a newer release is named. Read one section, run or trace its example, then explain it aloud without looking. For a coding exercise, first write a clear loop; use a stream when it makes the transformation easier to read.

## 1. The three ideas in one minute

**Lambda:** A short way to supply behavior where Java expects a functional interface. Example: `n -> n > 0` tells `filter` which numbers to keep.

**Stream:** A pipeline that processes elements from a source. It can filter, transform, group or aggregate values. It does not store the data itself. Intermediate operations are lazy; a terminal operation asks for a result.

**Optional:** A container that says a result may be present or absent. It is especially useful as a return type for a lookup that may find nothing. It does not automatically eliminate all nulls.

**30-second professional answer:** "A lambda supplies a small piece of behavior through a functional interface. Streams let me express a sequence of transformations and a final result. Optional makes an absent return value explicit. I choose these tools when they make the business logic clearer, and I still pay attention to nulls, side effects, ordering and performance."

**Simple picture:** `source -> filter -> map -> collect`. The source supplies data; the middle stages describe what to do; the terminal stage produces the result. The lambda tells each stage how to handle one element.

# 2. Lambdas and functional interfaces

## 2.1 What a lambda really is

A lambda is an implementation of the single abstract method of a **functional interface**. Java infers the parameter and return types from the interface expected at that position. `@FunctionalInterface` asks the compiler to reject accidental extra abstract methods. Default and static methods do not prevent an interface from being functional.

JAVA

```java
@FunctionalInterface
interface PaymentRule {
    boolean accepts(Payment payment);
}

PaymentRule positive = p -> p.amount.signum() > 0;
boolean allowed = positive.accepts(payment);
```

The lambda has no independent type without a target. The expression `p -> ...` becomes a `PaymentRule` because that is the type on the left. A lambda can also be passed directly into a method that expects `Predicate<Payment>`.

**Professional answer:** "A lambda is an implementation of a functional interface's one abstract operation. Its target type tells the compiler what its inputs and result mean. I use it for small, local behavior; if the logic becomes long or has several responsibilities, I move it to a named method or class."

## 2.2 The four interfaces to know cold

| Interface | Meaning | Example |
|---|---|---|
| `Predicate<T>` | `T -> boolean` : test a condition | `p -> p.successful` |
| `Function<T,R>` | `T -> R` : transform a value | `p -> p.accountId` |
| `Consumer<T>` | `T -> void` : perform an action | `p -> audit(p)` |
| `Supplier<T>` | `() -> T` : produce a value later | `() -> loadDefault()` |

Also know `UnaryOperator<T>` (`T -> T`), `BinaryOperator<T>` (`(T,T) -> T`), `Comparator<T>` (ordering), and primitive variants such as `IntPredicate` and `ToIntFunction<T>` when boxing matters. `Predicate.and`, `or` and `negate` combine conditions. `Function.andThen` and `compose` combine transformations; their order differs.

JAVA

```
Predicate<Payment> success = p -> p.successful;
Predicate<Payment> usd = p -> "USD".equals(p.currency);
Predicate<Payment> successfulUsd = success.and(usd);
Function<String, String> trim = String::trim;
Function<String, String> upper = String::toUpperCase;
String code = trim.andThen(upper).apply(" usd "); // USD
```

**Interview trap:** A `Consumer` returns no value. `map` needs a `Function` that returns a transformed value. Do not use `forEach` just to build another list by mutating it; use `map(...).collect(...)` or a loop.

## 2.3 Lambda syntax and method references

JAVA

```
x ->x *2
        (a,b)->a.

compareTo(b)

p ->{

audit(p); return p.id; }
String::trim
Payment::getAmount
ArrayList::new
```

The method reference is a shorter spelling when an existing method already expresses the behavior. It does not make the code faster by definition. Use the form that makes input and output easiest to see. `Payment::getAmount` applied to a `Payment` works like `p -> p.getAmount()`.

**Be precise:** A lambda can capture a local variable only if the variable is final or effectively final, meaning you do not reassign it after initialization. This restriction applies to the local reference, not necessarily to the object it points to. Changing a captured mutable list is possible, but it can create confusing behavior or races.

JAVA

```
String tenant = "A";
Predicate<Payment> belongsToTenant =
        p -> tenant.equals(p.tenantId); // tenant is effectively final
```

**Professional answer:** "Captured local variables must be effectively final. That makes the value captured by the lambda stable. It does not make a captured mutable object thread-safe, so I avoid using shared mutable state inside parallel operations."

### 2.4 Comparators: a common coding question

JAVA

```
Comparator<Payment> byTimeThenId =
        Comparator.comparing((Payment p) -> p.createdAt)
                .thenComparing(p -> p.id);
```

Use `reversed()` when the entire comparison order should reverse. If only time should descend, reverse that component before adding the ID tie-breaker. Never compare integers by `a - b`; subtraction can overflow. Use `Integer.compare(a, b)` or comparator factories. A comparator must give consistent ordering, particularly when a sort or ordered map relies on it.

## 3. Streams: how the pipeline runs

### 3.1 Source, intermediate steps and terminal result

JAVA

```
List<String> accounts = payments.stream()       // source
        .filter(p -> p.successful)                    // intermediate
        .map(p -> p.accountId)                        // intermediate
        .distinct()                                   // intermediate
        .collect(Collectors.toList());               // terminal
```

`filter` keeps some values; `map` converts each kept `Payment` into an account ID; `distinct` removes equal IDs; `collect` creates the result. No processing needs to happen when you merely assemble the intermediate steps. The terminal operation starts traversal. Operations can process elements one by one rather than constructing a full list at every intermediate stage. Some steps, such as sorting, may need to buffer data.

**Professional answer:** "A stream pipeline consists of a source, intermediate operations and a terminal operation. Intermediate operations are lazy. This lets Java combine work and stop early for operations such as findFirst, but it also means I should not rely on side effects inside map or filter."

**Important nuance:** Java can optimize away invocation of some intermediate lambdas when their effects are irrelevant to the result. Use streams to compute values, not to guarantee an audit or external write. See Oracle's Stream contract.

### 3.2 Common operations in plain English

| Operation | Plain English | Result or caution |
|---|---|---|
| `filter(predicate)` | Keep values that pass a test | Another stream |
| `map(function)` | Change each value into one new value | Another stream |

| Operation | Plain English | Result or caution |
|---|---|---|
| `flatMap(function)` | Change each value into a stream, then flatten | Another stream |
| `distinct()` | Keep one of each equal value | Uses equality rules |
| `sorted(comparator)` | Return values in an order | May buffer the input |
| `limit(n)` | Take at most `n` values | Often useful after ordering |
| `anyMatch`, `allMatch`, `noneMatch` | Ask a yes/no question | Short-circuiting terminal result |
| `findFirst()` | Return first matching value if present | Terminal `Optional<T>` |
| `count()` | Count values | Terminal `long` |
| `collect(...)` | Build a collection or grouped result | Terminal result |
| `reduce(...)` | Combine values into one value | Terminal result |

**Encounter order:** A stream from a `List` normally has list order; a `HashSet` does not promise an order. `forEach` on a parallel stream does not promise encounter order; `forEachOrdered` preserves it at a potential cost. `findAny` may choose any element. Do not use an unordered source and promise a "first" record.

**One-use rule:** A stream is consumed by a terminal operation and should not be reused. Create a new stream from the source for another calculation. A collection can be reused; the stream is the traversal.

## 3.3 `map` versus `flatMap`: the interview favorite

Suppose each account has a list of payment IDs. `map` keeps one list per account; `flatMap` produces one combined stream of IDs.

JAVA

```
List<List<String>> nested = accounts.stream()
        .map(a -> a.paymentIds)
        .collect(Collectors.toList());

List<String> allIds = accounts.stream()
        .flatMap(a -> a.paymentIds.stream())
        .collect(Collectors.toList());
```

The first result looks like `[[P1, P2], [P3]]`. The second looks like `[P1, P2, P3]`. A `flatMap` mapper returns a **stream**; a `map` mapper returns one value.

**Professional answer:** "I use map for a one-to-one transformation and flatMap when each input produces zero or more outputs that I want in one stream. For example, map gives me a stream of payment lists from accounts; flatMap gives me one stream of payments."

## 3.4 `filter`, `findFirst` and early stopping

JAVA

```
Optional<Payment> firstFailure = payments.stream()
        .filter(p -> !p.successful)
        .findFirst();
```

On an ordered sequential stream, this asks for the first failed payment and need not inspect all later elements. `findFirst` returns an `Optional` because there may be no failure. If the input is unordered, "first" has no business meaning; define the ordering before relying on it. `limit` and matching operations can also stop early, though actual work and performance depend on the pipeline and whether it is parallel.

## 3.5 Collecting and grouping

JAVA

```
Map<String, List<Payment>> byAccount = payments.stream()
        .collect(Collectors.groupingBy(p -> p.accountId));

Map<String, Long> countByAccount = payments.stream()
        .collect(Collectors.groupingBy(
                p -> p.accountId, Collectors.counting()));
```

`groupingBy` collects values under a key. A downstream collector decides what each group contains. The first stores lists; the second stores counts. Group a financial amount by **account and currency**, or first confirm that conversion is intended. For a simple Java 8 total, a loop with `Map.merge` can be clearer than a nested collector.

`toMap` **trap:** If two elements produce the same key, `Collectors.toMap(keyMapper, valueMapper)` throws an exception. Provide a merge function only after deciding the business rule. For payment IDs, silently keeping the first duplicate may hide a serious problem.

JAVA

```
Map<String, String> latestByAccount = payments.stream()
        .collect(Collectors.toMap(
                p -> p.accountId,
                p -> p.id,
                (first, second) -> second));
```

The example keeps the **later encountered** ID, which is not necessarily the latest by timestamp. Sort with an explicit tie-breaker or compare timestamps in the merge function if the requirement is "latest." Know the difference before using this pattern in an interview.

## 3.6 Reduction and money

JAVA

```
BigDecimal total = payments.stream()
        .filter(p -> p.successful && "USD".equals(p.currency))
        .map(p -> p.amount)
        .reduce(BigDecimal.ZERO, BigDecimal::add);
```

The identity is the starting value and the result for an empty stream. Addition is suitable because combining groups in different orders gives the same mathematical total. In practice, validate scale, currency, reversals, duplicate records and the source of truth. Do not use `double` for a financial total. Keep a transaction total distinct from an account balance, which may require stronger consistency.

**Professional answer:** "Reduction combines elements into one result using an identity and an associative combining operation. For a monetary sum I use BigDecimal addition and only combine values with the same currency. I would verify the input's duplicate and reversal rules."

## 3.7 Primitive streams and boxing

`Stream<Integer>` holds boxed Integer references. `IntStream` specializes common numeric work and avoids some boxing overhead. A stream of objects can use `mapToInt`, `mapToLong` or `mapToDouble`. For money, do **not** switch to `double` merely to use a primitive stream; keep decimal arithmetic or an agreed minor-unit representation.

JAVA

```
int sum = numbers.stream().mapToInt(Integer::intValue).sum();
IntSummaryStatistics stats = numbers.stream()
        .mapToInt(Integer::intValue)
        .summaryStatistics();
```

## 3.8 Side effects, parallel streams and when to use a loop

This is risky, especially in parallel:

JAVA

```
List<String> ids = new ArrayList<>();
payments.

parallelStream().

forEach(p ->ids.

add(p.id));
```

`ArrayList` is not safe for concurrent writes, and `forEach` is not guaranteed to preserve order in a parallel stream. Prefer `map(...).collect(...)` for a result. Even a synchronized list would not turn an unclear side-effect pipeline into the best design. A parallel stream is not an automatic speed improvement: it adds coordination cost, may use a shared pool, and can behave poorly with blocking calls or small inputs. Measure a real workload before choosing it.

Use a **loop** when the operation has branching, checked exception handling, early `break` behavior, mutation of a complex result, or side effects whose order matters. Streams are a way to express a computation, not a requirement for modern Java.

**Professional answer:** "I avoid shared mutable state and external side effects in stream operations. I use sequential streams for clear data transformations and consider parallel execution only after checking data size, work per element, ordering requirements, and measured performance."

# 4. Optional: absent values without mystery

## 4.1 Creating and returning Optional

JAVA

```
Optional<String> present = Optional.of("A123");
Optional<String> missing = Optional.empty();
Optional<String> maybe = Optional.ofNullable(rawAccountId);
```

`of(null)` throws immediately. `ofNullable(null)` returns empty. An `Optional` reference itself should never be null. Optional is primarily useful for return values where absence is a normal outcome, such as `findAccount(id)`. Do not automatically put it in every field, parameter or persistence entity; follow the project's conventions and API requirements. See Oracle's Optional API.

**Professional answer:** "Optional makes a possibly absent result visible in a method's return type. I use it for legitimate absence, such as a lookup with no match. I do not use it to hide an error or to replace every nullable field."

## 4.2 `map` and `flatMap` on Optional

JAVA

```
Optional<String> region = findAccount(id)
        .map(account -> account.region);

Optional<String> managerEmail = findAccount(id)
        .flatMap(account -> findManager(account))
        .map(manager -> manager.email);
```

If `findManager` returns `Optional<Manager>`, `map` would produce `Optional<Optional<Manager>>`. `flatMap` avoids that extra layer. `Optional.map` treats a null mapper result as empty; `Optional.flatMap` expects the mapper to return a non-null Optional. Prefer not to return null from either mapper.

## 4.3 `orElse` versus `orElseGet` versus `orElseThrow`

JAVA

```
String a = maybeId.orElse("UNKNOWN");
String b = maybeId.orElseGet(() -> loadDefaultId());
String c = maybeId.orElseThrow(
        () -> new NoSuchElementException("Account ID missing"));
```

`orElse` takes a value. Java evaluates its argument before calling the method, even if the Optional is present. `orElseGet` takes a Supplier and runs it only when empty. For a cheap constant, `orElse` is simple. For an expensive lookup or an operation with side effects, `orElseGet` avoids unnecessary work. `orElseThrow` is for absence that is an error at that boundary. The no-argument `orElseThrow()` is newer than Java 8; the Supplier form above works in Java 8.

**Interview trap:** Calling `get()` without checking presence can throw. A pattern like `if (opt.isPresent()) return opt.get();` may be correct, but often `map`, `orElseGet` or `orElseThrow` expresses the rule more clearly. Do not use `orElse(null)` everywhere; that usually gives up the benefit of Optional at the next line.

### 4.4 Optional and error semantics

"No account found" can be a normal lookup result (`Optional.empty()`), a 404 at an HTTP boundary, or an invalid state in a payment operation. Choose based on context. A database outage is **not** an empty result. Swallowing an exception and returning `Optional.empty()` would make an operational failure look like missing data.

**Professional answer:** "I reserve an empty Optional for expected absence. I propagate or translate real failures separately so callers can distinguish 'not found' from 'could not check.'"

## 5. A complete payment example to trace

The following snippet assumes a `Payment` with `accountId`, `currency`, `amount` and `successful` fields. It totals successful USD payments by account. `Map.merge` is an intentional loop alternative because it makes the accumulation and duplicate policy easy to inspect.

JAVA

```
Map<String, BigDecimal> totals = new HashMap<>();
for(
Payment p :payments){
        if(!p.successful ||!"USD".

equals(p.currency))continue;
        totals.

merge(p.accountId, p.amount, BigDecimal::add);
}
```

A stream can express the same grouping through a downstream reduction:

JAVA

```
Map<String, BigDecimal> totals = payments.stream()
        .filter(p -> p.successful && "USD".equals(p.currency))
        .collect(Collectors.groupingBy(
                p -> p.accountId,
                Collectors.reducing(
                        BigDecimal.ZERO, p -> p.amount, BigDecimal::add)));
```

Both versions assume the input has the right tenant scope and already handles duplicate events according to business rules. Both can be wrong for a payment system if those assumptions are false. The loop may be preferable in a live coding interview because it is short and easy to test. Tell the interviewer why you chose it.

**Question to rehearse:** "How would you adapt this for multiple currencies?" Answer: use a composite key containing account and currency, or a nested map. Define refunds and reversals. Do not add USD and EUR amounts directly.

# 6. Interview questions with polished answers

## What is a functional interface?

"It has one abstract method, so a lambda can supply that method's implementation. Predicate, Function, Consumer and Supplier are common examples. The annotation is optional, but useful because the compiler checks my intent."

## What is lazy evaluation in a stream?

"Intermediate operations describe work but do not produce a result by themselves. A terminal operation starts traversal. This can avoid unnecessary work; for example, findFirst can stop once it finds a match in an ordered sequential stream."

## What is the difference between map and flatMap?

"Map returns one result for each input. FlatMap handles an input that produces a stream of results and combines those streams into one. For account-to-payments, map gives me lists of payments; flatMap gives me one sequence of payments."

## How is Optional different from null?

"Optional makes possible absence part of the return type and provides methods to handle it. It does not guarantee that every value in the system is non-null. I use it for expected absence and keep actual failures separate."

### When would you avoid streams?

"I use a loop when it makes state changes, early exit, exception handling or branching clearer. I also avoid side effects inside a stream pipeline. Readability and correctness matter more than making every operation a stream."

### Are streams faster than loops?

"Not inherently. A stream can express a transformation clearly and may optimize traversal, but it also has overhead. Parallel streams add coordination and ordering costs. I choose based on clarity first and measure performance for important paths."

### Why can `toMap` fail?

"The two-argument toMap collector throws if two inputs generate the same key. I decide whether duplicates are invalid, should be grouped, or need an explicit merge rule. In financial data, I would not silently discard one record."

### Why is a parallel stream with a shared ArrayList unsafe?

"Multiple threads can modify the list at the same time, causing lost or corrupted updates. I would collect the result using a suitable collector or stay sequential. I would also check whether parallel execution actually helps."

## 7. Practice in 45 minutes

**First 15 minutes - trace:** For `[" a ", "", " b ", " a "]`, predict the result of trim, remove empty, uppercase, distinct and collect in that order. Explain what changes if you put `distinct` before trim.

**Next 15 minutes - code:** Given payments, produce the IDs of successful USD payments in encounter order. Then group successful payments by account and currency and total them with BigDecimal. Write a loop first, then a stream version. Add empty input, duplicates and a reversal to your test cases.

**Final 15 minutes - speak:** Answer the eight questions above aloud. Keep each answer to 30-45 seconds. Repeat only the answers you could not explain clearly.

### Quick self-check

1. What type does `p -> p.amount` need for `map`? **Function from Payment to BigDecimal.**
2. What does `flatMap(a -> a.payments.stream())` return? **One stream of payments, rather than a stream of lists or streams.**
3. Does `.filter(...).map(...)` run without a terminal operation? **No traversal is requested yet.**
4. Is `Optional.of(null)` empty? **No; it throws. Use `ofNullable` when null is possible.**
5. Will `present.orElse(expensive())` call `expensive()`? **Yes. Java evaluates the argument first.**
6. Can `toMap` accept duplicate keys automatically? **No; choose a merge rule or grouping collector.**

7. Does `parallelStream()` guarantee faster code or ordered `forEach`? **No.**
8. Is a database error an empty Optional? **No; keep failures distinct from absence.**

# 8. Final recall card

**Lambda = behavior passed through an interface.** Predicate tests, Function transforms, Consumer acts, Supplier produces later. Captured locals are effectively final.

**Stream = source + lazy intermediate steps + terminal result.** `filter` keeps; `map` transforms one-to-one; `flatMap` flattens one-to-many. `collect` builds; `reduce` combines. A stream is generally used once. Avoid stateful side effects.

**Optional = present or absent return value.** `of` requires non-null; `ofNullable` accepts null; `map` transforms; `flatMap` avoids nested Optionals; `orElseGet` computes a fallback only when needed. An error is not the same as absence.

**Sources checked:** Oracle Stream API, Optional API, functional interface and lambda tutorial, and stream package guide. The tutorial was written for JDK 8; check your project's release for newer convenience methods.

# Spring features companion

8 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

1. The three layers to remember ..... 2
2. Component scanning and bean definitions ..... 2
3. Scope, lifecycle, and dependency problems ..... 5
4. Spring Boot startup and configuration ..... 6
5. Spring MVC request handling ..... 9
6. Data access and production API habits ..... 11
7. One connected example: an authorized payment lookup ..... 12
8. Interview drills and fast recall ..... 13

This is a focused companion to the interview study guide. It explains Spring Framework, Spring Boot, and Spring MVC in plain English, then gives short answers you can practice aloud. The examples use modern Spring Boot with `jakarta.*` imports; confirm the versions used by the interviewing team before discussing version-specific behavior. The code fragments illustrate the idea and omit imports and surrounding project setup.

**Source and version notes:** The linked Spring Framework and Spring Boot pages are official documentation checked on October 22, 2026. Their current pages may describe a newer release than the team's runtime. In particular, `jakarta.*` imports and record-based examples suit modern Spring Boot; older projects may use `javax.*` and older Java syntax. The precise default behavior of auto-configuration, endpoint exposure, and shutdown should be checked against the application's Spring Boot version and settings. For the interview, explain the concept first, then qualify version-specific details if asked.

# 1. The three layers to remember

| Layer | Main job | Example |
| --- | --- | --- |
| Spring container | Create application objects (beans), connect dependencies, manage scope and lifecycle | `PaymentService` receives a `PaymentRepository` |
| Spring Boot | Start and configure the application with sensible conditional defaults | Configure an embedded server and a database client |
| Spring MVC | Map HTTP requests to methods, read inputs, validate them, and write responses | `GET /payments/{id}` calls a controller method |

**A request in one sentence:** The embedded server receives the HTTP request, filters run, Spring MVC finds the controller method, validation checks the request, the service applies business rules, the repository reads data, and an HTTP response goes back through the chain.

**Interview answer to practice:** "I separate responsibilities across the Spring container, Boot, and MVC. The container manages dependencies and lifecycle. Boot assembles the application and conditional configuration. MVC maps requests to controller methods. My controller handles the HTTP contract, while services and repositories handle business rules and persistence."

# 2. Component scanning and bean definitions

## Component scanning

**Plain English:** A bean is an object created and managed by Spring. Component scanning searches selected packages for classes marked with component annotations and registers them as beans. Constructor injection then connects those beans.

`@SpringBootApplication` includes component scanning. By default, scanning starts at the package containing the application class and covers its subpackages. Put the application class above the packages it should scan, or configure scanning explicitly. A class outside the scanned packages can exist in the project but never become a bean.

JAVA

```java
package com.example.payments;

@SpringBootApplication
public class PaymentsApplication {
    static void main(String[] args) {
        SpringApplication.run(PaymentsApplication.class, args);
    }
}
```

```
// com.example.payments.service.PaymentService is scanned.
// com.otherteam.LegacyClient is not scanned by default.
```

**Interview answer:** "Component scanning finds annotated classes in the configured package tree and registers them in the application context. If an injection fails, I first check whether the implementation is in the scan path and whether its conditions or profile are active."

**Common trap:** `new PaymentService(...)` creates an ordinary Java object; it does not ask Spring to manage that instance. Spring features applied around managed beans may not apply to manually constructed instances.

### `@Component`, `@Service`, and `@Repository`

All three mark a class as a candidate for component scanning. `@Component` is general. `@Service` communicates business logic. `@Repository` communicates data access and can participate in Spring's persistence exception translation when the relevant infrastructure is configured. Choose the annotation that expresses the class's role; adding both `@Component` and `@Service` is unnecessary.

JAVA

```java
@Service
public class PaymentService {
    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }
}

@Repository
public class PaymentRepository {
    // Database access belongs here.
}
```

**Interview answer:** "These are Spring stereotypes. They all register components, but `@Service` and `@Repository` make the role clear. I use constructor injection so required dependencies are visible and the class can be tested without a full container."

**Nuance:** A repository interface discovered by Spring Data is registered through its repository infrastructure; it is not the same mechanism as scanning a concrete `@Repository` class. Spring component scanning.

### `@Configuration` and `@Bean`

**Plain English:** Use `@Configuration` for a class that declares how beans are built. Use `@Bean` on a method that returns an object Spring should manage. This is useful when configuring a library class you cannot annotate or when construction needs explicit settings.

JAVA

```java
@Configuration
public class ClientConfiguration {
```

```
    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }
}
```

**Interview answer:** "I use component scanning for my application classes and `@Bean` methods for explicit construction, especially third-party clients. I keep bean configuration separate from business logic so dependencies and settings are easy to inspect."

**Important detail:** A full `@Configuration` class normally protects calls between `@Bean` methods so they follow container semantics. With `@Configuration(proxyBeanMethods = false)`, a direct Java call is just a direct call. Prefer method parameters to request dependencies from Spring rather than relying on one `@Bean` method calling another. Spring `@Bean` documentation.

### `@Qualifier` and `@Primary`

If Spring finds two beans of the same required type, it needs a selection rule. `@Primary` sets the preferred bean for an ordinary single-valued injection. `@Qualifier` narrows candidates at a particular injection point. An explicit qualifier is appropriate when the caller needs a specific implementation. Do not assume these annotations create the beans; they only help choose among registered candidates.

JAVA

```java
interface PaymentGateway {
    Receipt charge(Payment p);
}

@Component("bankA")
@Primary
class BankAGateway implements PaymentGateway { /* ... */
}

@Component("bankB")
class BankBGateway implements PaymentGateway { /* ... */
}

@Service
class InternationalPaymentService {
    InternationalPaymentService(
            @Qualifier("bankB") PaymentGateway gateway) {
        this.gateway = gateway;
    }

    private final PaymentGateway gateway;
}
```

**Interview answer:** "When several implementations share a type, I use `@Primary` for a sensible default and `@Qualifier` where a particular implementation is required. I prefer descriptive qualifiers because they reveal intent better than depending on an incidental field name." Qualifier selection; primary selection.

# 3. Scope, lifecycle, and dependency problems

## Singleton, prototype, and request scope

| Scope | What it means | Good use | Trap |
|---|---|---|---|
| Singleton | One bean instance for each bean definition in one application context | Usually stateless services, repositories, clients | Shared mutable request data can race; singleton does not mean thread-safe |
| Prototype | A new instance each time the container is asked for the bean | A stateful helper with a short, explicit use | Injecting it once into a singleton gives that singleton one instance |
| Request | One bean instance per HTTP request in a web-aware context | Request-specific context | A singleton needs a proxy/ provider to obtain the current request bean |

**Plain English:** Scope answers how long a Spring-managed instance lives and when a new one is obtained. It does not control database transaction boundaries or make code safe for concurrent requests.

JAVA

```java
@Component
@Scope("prototype")
class ReportBuilder { /* mutable, short-lived state */
}

@Service
class ReportService {
    private final ObjectProvider<ReportBuilder> builders;

    ReportService(ObjectProvider<ReportBuilder> builders) {
        this.builders = builders;
    }

    Report build() {
        ReportBuilder builder = builders.getObject();
        return builder.finish();
    }
}
```

The `ObjectProvider` call asks Spring for a new prototype on each use. Request-scoped beans can use a scoped proxy or a provider. If no HTTP request scope is active, asking for a request bean is invalid. A prototype's destruction callbacks are generally not managed automatically after the container hands it to you, so its owner must clean up resources when appropriate.

**Interview answer:** "Spring beans are singleton by default within an application context. I keep service state local to each method call. If I need a fresh stateful helper, I request a prototype when needed; injecting one prototype into a singleton at startup does not create a new instance per method call." Bean scopes.

### Initialization and destruction

Spring constructs a bean, injects dependencies, runs initialization callbacks, and later runs destruction callbacks when it closes the managed scope. Common lifecycle mechanisms are `@PostConstruct`, `@PreDestroy`, and explicit `initMethod` / `destroyMethod` on `@Bean`.

JAVA

```java
@Component
class CertificateStatusCache {
    @PostConstruct
    void initialize() { /* validate initial configuration */ }

    @PreDestroy
    void close() { /* release owned resources */ }
}
```

**Interview answer:** "I use initialization for lightweight validation or setup after dependencies are available, and destruction for releasing owned resources. For a web service, I also consider graceful shutdown so in-flight requests have time to finish. I avoid slow remote calls that make startup unreliable without a clear readiness strategy."

**Nuance:** A clean context shutdown runs these callbacks for managed beans; process termination without orderly shutdown may not. Prototype cleanup is a special case. Bean lifecycle.

### Circular dependencies

**Plain English:** A cycle exists when A needs B to be constructed and B needs A to be constructed. With constructor injection, neither can be finished first. This often shows that responsibilities are mixed together.

**Better design:** If `PaymentService` needs `AuditService` and `AuditService` needs `PaymentService`, extract a smaller `PaymentReader` or publish an event after the core action. Keep the dependency graph directional. `@Lazy` or a provider can defer construction, but that may hide the design issue.

**Interview answer:** "I treat a circular dependency as a design signal. I inspect why the two services know about each other, then extract shared behavior or change the direction of the dependency. I use lazy resolution only when the cycle has a justified lifecycle reason." Dependency and cycle handling.

## 4. Spring Boot startup and configuration

### `@SpringBootApplication` and auto-configuration

`@SpringBootApplication` combines application configuration, component scanning, and Boot auto-configuration. Boot examines the libraries on the classpath, existing beans, and configuration properties before creating many default beans. For example, it can configure a web server and application plumbing when the relevant web starter is present.

**Back-off rule:** Conditional auto-configuration often creates a default bean only when you have not already provided a matching bean. It is conditional, not a blanket "my bean always wins" guarantee. The exact condition matters: type, name, classpath, property, or another criterion.

**How to investigate:** Start the application with `--debug` to view the conditions report in logs, or inspect Actuator's conditions endpoint if it is included and appropriately exposed. Look for a positive or negative match and the reason. This is more reliable than guessing which auto-configuration ran.

**Interview answer:** "Spring Boot removes setup work through conditional auto-configuration. When behavior surprises me, I check the conditions report and the actual bean definitions. If I provide my own bean, I confirm which condition backs off and which defaults remain active." Auto-configuration and conditions report.

## Profiles

**Plain English:** A profile enables configuration only in selected environments. You can mark a component or configuration class with `@Profile("dev")` and select an active profile using configuration such as `spring.profiles.active=dev`.

JAVA

```java
@Configuration
@Profile("local")
class LocalGatewayConfiguration {
    @Bean
    PaymentGateway paymentGateway() {
        return new SandboxGateway();
    }
}
```

Use a local or test implementation only where intended. A profile is not an authorization boundary or a secret store. For production, externalize credentials and check the effective active profile during startup and deployment.

**Interview answer:** "Profiles help me select environment-specific beans and settings, while keeping the application code consistent. I make the active profile visible in deployment configuration and verify that a local mock cannot accidentally be selected in production." Spring Boot profiles.

## Configuration properties and configuration validation

`@ConfigurationProperties` binds a related group of settings to a typed object. This is easier to check than spreading many `@Value` expressions through the code. Add `@Validated` and Jakarta Bean Validation constraints to fail startup when required settings are missing or invalid. Register a properties class using configuration-properties scanning or an enabling annotation.

JAVA

```java
@ConfigurationProperties(prefix = "payments.client")
@Validated
public record PaymentClientProperties(
        @NotBlank String baseUrl,
```

```
        @Min(100) long timeoutMillis) {
}

@SpringBootApplication
@ConfigurationPropertiesScan
class PaymentsApplication { /* main method */
}
```

PROPERTIES

```
payments.client.base-url=https://example.invalid
payments.client.timeout-millis=1500
```

The example uses a Java record, so it needs a Java version that supports records. In a Java 8 project, use an ordinary mutable properties class with getters/setters and validation annotations. If you use nested objects, apply cascading validation where needed. Avoid putting secret values into logs or error messages.

**Interview answer:** "For grouped configuration, I use typed configuration properties and validate them at startup. This catches a missing URL or an unsafe timeout before the service begins handling traffic. I reserve profiles for selecting environment-specific settings, not for storing secrets." External configuration and validation.

## Embedded server, health endpoints, and graceful shutdown

**Embedded server:** A Boot web application can start with its server packaged into the application. A servlet-based starter typically includes Tomcat by default, and another supported server can be selected. `server.port` changes the listening port; web-server choice is a dependency/configuration decision, not a controller annotation.

**Health:** With Actuator, `/actuator/health` reports application health. Liveness asks whether this instance should be restarted. Readiness asks whether it should receive traffic. Avoid a liveness check that fails merely because a shared database is temporarily unavailable; restarting every instance can make an outage worse. Check which Actuator endpoints are exposed and protect sensitive endpoint details.

**Graceful shutdown:** On a proper shutdown signal, the server stops taking new requests and gives active requests a bounded time to finish. Set a timeout compatible with the deployment platform's termination window. For long-running jobs or message consumers, also coordinate their stop and checkpoint behavior; an HTTP grace period alone is insufficient.

PROPERTIES

```
server.port=8080
spring.lifecycle.timeout-per-shutdown-phase=20s
```

**Interview answer:** "Boot packages a server with the application and exposes operational features through Actuator. I use readiness to control traffic, liveness for true process failure, and graceful shutdown to give in-flight work time to complete. I verify the deployment platform gives the process at least that much termination time." Embedded servers; Actuator endpoints; graceful shutdown.

# 5. Spring MVC request handling

## `@RestController` and mappings

`@RestController` marks a controller whose method return values normally become response bodies, commonly JSON. `@RequestMapping` can establish a base path or define a mapping; method-specific forms such as `@GetMapping` and `@PostMapping` also make the HTTP method explicit. Spring MVC binds a path segment with `@PathVariable`, a query value with `@RequestParam`, and a body with `@RequestBody`.

JAVA

```java
@RestController
@RequestMapping("/api/payments")
class PaymentController {
    private final PaymentService service;

    PaymentController(PaymentService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    PaymentResponse get(@PathVariable String id) {
        return service.getAuthorizedPayment(id);
    }
}
```

This example leaves authentication details out of the fragment. The service method's name reminds you that looking up a payment by ID must also enforce the caller's permission to access it.

**Interview answer:** "I make mappings explicit about path and HTTP method. The controller translates HTTP input into a service call and returns a response DTO. It does not own database logic or trust a resource ID as proof of access." Spring MVC request mapping.

## DTOs and `@Valid`

A DTO is a data transfer object: a deliberate shape for API input or output. An input DTO allows only fields the client is permitted to send. An output DTO avoids accidentally exposing internal fields or database relationships. `@Valid` triggers Bean Validation for a suitable request body when a validation provider is configured.

JAVA

```java
public record CreatePaymentRequest(
        @NotBlank String accountId,
        @NotNull @DecimalMin("0.01") BigDecimal amount,
```

```
        @NotBlank String currency) {
}

@PostMapping
ResponseEntity<PaymentResponse> create(
        @Valid @RequestBody CreatePaymentRequest request) {
    PaymentResponse result = service.create(request);
    return ResponseEntity.status(201).body(result);
}
```

The DTO does **not** contain fields such as `approved`, `role`, `tenantId`, or `createdBy` for a client to set. Derive identity and tenant from trusted authentication context. `@Valid` checks field constraints; it does not check that the caller owns an account, that funds are available, or that the request is safe to repeat. Those are business and authorization rules. Check how validation failures are handled and converted into consistent client errors.

**Interview answer:** "I use request and response DTOs to control the API contract. I validate input shape at the boundary, then enforce authorization and business invariants in the service. I avoid binding clients directly to persistence entities, especially for privileged or internal fields." MVC validation.

## `@ControllerAdvice` and stable error responses

`@ControllerAdvice` applies cross-controller behavior, including exception handling. `@RestControllerAdvice` is convenient when handlers return response bodies. A handler should map a known problem to a stable HTTP status and an error code clients can rely on. Keep unexpected internal details out of the response, and log enough context for operators to diagnose the incident.

JAVA

```
record ApiError(String code, String message) {
}

@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(PaymentNotFoundException.class)
    ResponseEntity<ApiError> missing(PaymentNotFoundException e) {
        return ResponseEntity.status(404)
                .body(new ApiError("PAYMENT_NOT_FOUND",
                        "Payment was not found"));
    }
}
```

For validation errors, define an intentional 400 response; for an authorization failure, use a status and message consistent with the application's security policy; for an unexpected failure, return a safe generic 500 response and log the internal cause once at the right boundary. Do not return database errors, stack traces, payment secrets, or raw personal data to the client. Spring also supports structured `ProblemDetail` responses; choose one stable external contract and apply it consistently.

**Interview answer:** "I centralize error translation so clients receive stable codes and appropriate HTTP statuses. I preserve the internal cause for diagnosis, but I do not leak internal details in the response. I handle validation, missing resources, forbidden access, and dependency failures deliberately." Controller advice; exception handlers.

### Filters and interceptors

**Filter:** A Servlet filter wraps requests and responses around the servlet chain. It can run before Spring MVC selects a controller. Good uses include correlation IDs, request/response wrapping, and integration with a security filter chain.

**Interceptor:** A Spring MVC interceptor runs around a mapped handler. It can use handler information for MVC-specific work, such as certain request measurements or locale handling. It is not the preferred sole security layer; Spring's documentation recommends Spring Security or a similar early filter-chain approach for security.

**Order to remember:** Request → filters → DispatcherServlet → handler interceptor → controller → service → repository; response and completion callbacks occur on the way out. The exact behavior can change with async dispatch and error paths, so test filters/interceptors under those conditions.

**Interview answer:** "Filters operate at the servlet boundary, while MVC interceptors operate around selected handlers. I use filters for cross-cutting request concerns and Spring Security for authentication and authorization. I use an interceptor when the behavior depends on the MVC handler." Servlet filters; MVC interceptors.

## 6. Data access and production API habits

### Parameterized queries

**Plain English:** SQL text should stay separate from user values. A bound parameter supplies data to a prepared statement, so a quote in a user value is not interpreted as part of SQL syntax.

JAVA

```
String sql = "SELECT id, status FROM payment " +
        "WHERE tenant_id = :tenant AND id = :id";
MapSqlParameterSource params = new MapSqlParameterSource()
        .addValue("tenant", tenantId)
        .addValue("id", paymentId);
return jdbc.

query(sql, params, rowMapper);
```

`NamedParameterJdbcTemplate` handles named values. It does not make dynamically concatenated table names, column names, or sort expressions safe. For those SQL identifiers, select from a server-side allowlist. Always scope payment queries to the authenticated tenant and authorize the specific resource.

**Interview answer:** "I bind all user-provided values as query parameters and use an allowlist for any dynamic SQL identifier. I also include the tenant and access boundary in the query or service logic. Parameter binding prevents SQL injection through values; it does not replace authorization." Spring JDBC core.

### Bounded pagination

Unbounded list endpoints can exhaust database, service, or client resources. Give page size a default and maximum, use a deterministic sort with a tie-breaker, and be clear whether the API returns a total count. Offset pagination is straightforward but can grow expensive or skip/duplicate items when rows are inserted during browsing. Keyset/cursor pagination is often better for a changing feed, but it needs a stable ordering key and a contract for the cursor.

JAVA

```java
int size = Math.max(1, Math.min(requestedSize, 100));
Pageable page = PageRequest.of(pageNumber, size,
        Sort.by("createdAt").descending()
                .and(Sort.by("id").descending()));
```

Before this code, validate that `pageNumber` is nonnegative and decide whether invalid or excessive sizes should be rejected or capped. A sort field coming from the client also needs an allowlist. The database query must still enforce tenant and authorization constraints. A `Page` may require a count query; a `Slice` can avoid that when you only need to know whether another page exists.

**Interview answer:** "I bound page size, use deterministic sorting, and make the count contract explicit. If the dataset changes frequently or deep offsets become costly, I consider cursor pagination. I never let a pagination parameter bypass tenant filtering." Spring Data pagination reference.

### Intentional logging

Log events that help answer a production question: what operation failed, when, on which safe correlation ID, and at which boundary. Keep secrets, bearer tokens, passwords, card details, and unnecessary personal data out of logs. Avoid logging the same exception in every layer; choose a boundary with enough context and preserve the cause. Pair logs with metrics and traces for latency investigations.

**Interview answer:** "I log enough structured context to trace an operation without recording sensitive payment or identity data. I preserve correlation IDs across calls and log unexpected exceptions once at the boundary that can add useful context. Stable error codes help connect client reports with internal diagnostics."

## 7. One connected example: an authorized payment lookup

1. A request reaches the embedded server: `GET /api/payments/p-123`.
2. The security filter chain authenticates the caller; a correlation filter attaches a safe request ID.
3. Spring MVC selects the `@GetMapping("/{id}")` method and binds `p-123` to `id`.
4. The controller calls `PaymentService.getAuthorizedPayment(id)`.

5. The service determines the trusted tenant and caller permissions, then asks the repository for the payment.
6. The repository runs a parameterized, tenant-scoped query. It returns only the fields needed for the response.
7. The service maps the result to `PaymentResponse`. The controller returns JSON. If the record is absent or access is denied, advice or security handling returns the defined error contract.
8. Metrics and correlation-aware logs show the outcome without exposing sensitive data.

**Professional version to memorize:** "For a payment lookup, I keep the controller thin. The security layer authenticates the caller, the service checks the caller's permission for the account, and the repository uses a parameterized, tenant-scoped query. I return a response DTO with only approved fields. The error handler produces stable client codes, while internal logs retain a correlation ID for investigation."

# 8. Interview drills and fast recall

| Question | Strong short answer |
| --- | --- |
| What does component scanning do? | It finds eligible annotated classes in configured packages and registers them as beans. |
| `@Component` versus `@Service` versus `@Repository`? | All register components; the latter two express business and data access roles. |
| `@Bean` versus `@Component`? | `@Bean` registers the result of a method; `@Component` registers a scanned class. |
| What if two beans share one interface? | Use an explicit qualifier for a required implementation or one primary default. |
| Is a singleton bean thread-safe? | No. Scope controls instance count; code and state management determine thread safety. |
| Why did a prototype seem shared? | It was injected once into a singleton; use a provider or suitable scoped lookup. |
| How do you resolve a circular dependency? | Change the dependency graph or extract a shared responsibility; do not hide the design issue by default. |
| What does Boot auto-configuration do? | It creates conditional defaults based on dependencies, settings, and existing beans. |
| How do you explain a missing Boot bean? | Inspect component scanning, active profiles, and the conditions report. |
| What is a profile? | A way to select environment-specific beans or settings; it is not a secret store. |
| What does `@Valid` guarantee? | It checks configured constraints on input; it does not authorize access or prove business correctness. |

| Question | Strong short answer |
| --- | --- |
| Why use DTOs? | They define the external contract and prevent unintended binding or exposure of internal fields. |
| Filter versus interceptor? | Filter surrounds the servlet chain; interceptor surrounds a Spring MVC handler. |
| Why use `@ControllerAdvice`? | To apply consistent exception translation across controllers. |
| Does `NamedParameterJdbcTemplate` secure dynamic column names? | No. Bind values; allowlist dynamic SQL identifiers. |
| Why bound page size? | To protect database and service resources and keep API behavior predictable. |

**Final 45-second answer:** "In Spring, I use component scanning and explicit bean configuration to make dependencies clear. Boot adds conditional application setup, typed configuration, and operational features such as health checks and graceful shutdown. In MVC, I keep controllers focused on HTTP: map requests, validate DTOs, and return stable responses. Services enforce authorization and business rules; repositories run tenant-scoped, parameterized queries. I watch for scope mistakes, circular dependencies, unbounded pagination, and accidental exposure of internal fields."

# Oral Drill — Technical Interview Practice

16 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

1. Why must equals and hashCode agree or why must you override both equals and hashCode? ..... 2
2. Why is volatile count++ unsafe? ..... 7
3. When does ConcurrentHashMap still need higher-level coordination? ..... 10
4. What is the difference between thenApply and thenCompose? ..... 13
5. Why can JDBC block a Vert.x event loop? ..... 15
6. What does Spring Boot add to Spring? ..... 18
7. Why might @Transactional not take effect? ..... 20
8. How do you detect and fix N+1 queries? ..... 28
9. How do you prevent lost updates? ..... 29
10. Why can a LEFT JOIN unexpectedly remove unmatched rows? ..... 31
11. How do you invalidate Redis data safely enough for the use case? ..... 32
12. What must you validate in a JWT besides its signature? ..... 34
13. How do you prevent cross-tenant access? ..... 36
14. What does a payment timeout actually tell you? ..... 42
15. What did you personally implement in your strongest project? ..... 44
Practice checklist ..... 45

**Use this guide:** Speak the **30–60 second answer** first. Use the **Follow-up** and **Small example** when asked. Open **Additional details** to understand the topic deeply. Each question also contains a **Full original notes** section: your source material is preserved (including original examples, diagrams and drafts).

**Content integrity:** All 15 original question headings and the complete body of your original notes appear below. The short spoken answers and further examples come from the earlier expanded study guide. For questions 8, 9, 12, 14 and 15, the original had no answer; added technical material is labeled and Q15 remains a personal template. The original source is preserved even where it contains code formatting or typing errors; organized code examples above it are corrected.

# 1. Why must equals and hashCode agree or why must you override both equals and hashCode?

## 30–60 second answer

`equals()` defines whether two Java objects are logically the same, while `hashCode()` helps hash-based collections locate them. The contract says that **if two objects are equal, their hash codes must also be equal**. The reverse is not true: different objects can share a hash code, which is called a collision. `HashMap` and `HashSet` first use the hash to narrow the search, then use `equals()` to confirm a match. If I override `equals()` but keep an incompatible `hashCode()`, two logically equal objects may end up in different buckets. That can cause failed lookups or duplicate entries in a set. I therefore implement both using the same stable equality fields.

## Follow-up

**What if two unequal objects have the same hash code?**

That is allowed. A collision means the collection must compare candidate entries with `equals()` to tell them apart. It can reduce performance, but does not by itself break correctness. A **changing hash code** is a different problem: if an object is already a key in a `HashMap`, changing a field used by `equals()` and `hashCode()` may make it unreachable via normal lookup. Prefer immutable keys, and make sure equality is reflexive, symmetric, transitive and consistent. An object's hash is not required to stay the same between separate JVM executions.

## Small example

JAVA

```java
record Person(String name) {} // Generates consistent equals and hashCode

Set<Person> people = new HashSet<>();
people.add(new Person("Alice"));
people.add(new Person("Alice"));
System.out.println(people.size()); // 1
```

Both records represent the same name, so the set keeps one entry. **Remember:** equal ⇒ same hash; same hash ⇏ equal.

## Additional details

### The contract

- `equals()` checks logical equality; `hashCode()` helps hash collections find the right bucket.
- **Equal objects must have equal hash codes.** Different objects can share a hash code; this is a collision, not a violation.
- `HashMap` first locates candidate entries with the hash, then compares with `equals()`.

### What breaks if you override only `equals()`?

The original `Person` example compares objects by `name`, but inherits `Object.hashCode()`. Two separate objects named Alice can satisfy `p1.equals(p2)` while producing different hash codes. A `HashSet` may then store both; a `HashMap` lookup using an equivalent key may fail.

### Correct implementation from your notes

JAVA

```java
import java.util.Objects;

class Person {
    String name;

    Person(String name) { this.name = name; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Person other)) return false;
        return Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
}

Person p1 = new Person("Alice");
Person p2 = new Person("Alice");
System.out.println(p1.equals(p2)); // true
System.out.println(p1.hashCode() == p2.hashCode()); // true

Set<Person> people = new HashSet<>();
people.add(p1);
people.add(p2);
System.out.println(people.size()); // 1
```

### Lookup flow

EXAMPLE

```
hashCode() -> candidate bucket -> equals() -> actual matching entry
```

The original examples, including the incorrect version and the longer flow diagram, are preserved below. The snippets above fix line-wrap errors in the original source.

**Full original notes (preserved)**

## Source wording for question 1

**The contract between `equals()` and `hashCode()`**

If two objects are equal according to `equals()`, they must have the same `hashCode()`.

**The reverse is not required:** Two objects can have the same hash code but still be unequal. This is called a **hash collision**.

EXAMPLE

```
a.equals(b) == true
    => a.hashCode() == b.hashCode()
```

Hash-based collections like HashMap and HashSet rely on both methods to find and identify objects correctly.

While `equals()` defines logical equality, `hashCode()` helps hash-based collections locate objects efficiently.

Their contract ensures that logically equal objects can be found in the same bucket and correctly identified by `equals()`.

**What happens if you violate the contract?**

Suppose you create a Person class that overrides equals () but not hashCode ().

JAVA

```java
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Person other)) return false;
        return name.equals(other.name);
    }
}
```

Now consider:

JAVA

```
Person p1 = new Person("Alice");
Person p2 = new Person("Alice");

System.out.

println(p1.equals(p2)); // true
        System.out.

println(p1.hashCode() ==p2.

hashCode()); // Usually false
```

**Why?** Because the inherited `Object.hashCode()` generally uses object identity rather than the name field. The two objects are logically equal but may have different hash codes.

**The consequence in a `HashSet`:**

JAVA

```
Set<Person> people = new HashSet<>();

people.

add(p1);
people.

add(p2);

System.out.

println(people.size()); // Can be 2, not 1
```

You expected one person because both objects represent Alice. But the set may put them in different hash buckets and fail to recognize that they are equal.

Similarly, a `HashMap` may fail to retrieve a value using a different but logically equal key.

**The correct implementation:**

Override both methods using the same fields that define logical equality.

JAVA

```
import java.util.Objects;

class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
```

```
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Person other)) return false;

        return Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
}
```

Now:

JAVA

```
Person p1 = new Person("Alice");
Person p2 = new Person("Alice");

System.out.

println(p1.equals(p2)); // true
        System.out.

println(p1.hashCode() ==p2.

hashCode()); // true

Set<Person> people = new HashSet<>();
people.

add(p1);
people.

add(p2);

System.out.

println(people.size()); // 1
```

### How HashMap uses both methods?

EXAMPLE

```
1. Calculate hashCode()
Determine the hash bucket to search.
    |
    v
2. Find candidate entries
Use the hash to narrow down the search.
    |
    v
```

```
3. Compare with equals()
Determine whether a candidate key is actually equal.
```

If equal objects produce different hash codes, the map may search the wrong bucket and never compare those objects using equals ().

**Official references:** Java `Object` contract.

---

# 2. Why is volatile count++ unsafe?

## 30–60 second answer

`volatile` gives Java threads visibility and memory-ordering guarantees, but it does **not** turn a compound operation into one atomic action. `count++` is really three steps: read the current value, add one, and write the result. Two threads can both read five, both calculate six, and both write six. One increment is lost, even though the field is volatile. To implement a shared counter safely, I would use `AtomicInteger.incrementAndGet()` or protect the increment with `synchronized`. `volatile` is useful for a simple state flag that one thread writes and other threads read, but it is not enough for a read-modify-write operation.

## Follow-up

**When would you choose `AtomicInteger` versus `synchronized`?**

Choose `AtomicInteger` for a single atomic numeric variable, including increments and compare-and-set. Use `synchronized` or another lock when **several fields must change together** and the whole invariant needs protection. For a highly contended statistics counter, `LongAdder` can offer better throughput, but `sum()` is not an atomic snapshot under concurrent updates. A shutdown flag is different: `volatile boolean shutdown` is suitable when workers only need to see its latest written state. Do not use `final` for a field you intend to increment or change.

## Small example

JAVA

```java
AtomicInteger count = new AtomicInteger();

// Two threads each call this 1,000 times:
count.incrementAndGet();
// After both threads complete, count.get() is 2,000.
```

**Remember:** visibility is not atomicity.

## Additional details

### Why the increment is unsafe

`count++` is a **read → add one → write** sequence. If two threads read `5` before either writes, they may both write `6` instead of reaching `7`. `volatile` does not make those three steps indivisible.

### What `volatile` guarantees

- **Visibility:** Another thread can observe a write to a volatile field.
- **Ordering:** Volatile reads and writes have Java memory-ordering effects.
- **Not atomicity:** It does not protect a read-modify-write sequence.

### Three choices from your notes

| Choice | Use when | Code |
|---|---|---|
| `AtomicInteger` | One shared counter | `count.incrementAndGet()` |
| `synchronized` | Protect a complete critical section | `public synchronized void increment() { count++; }` |
| `volatile` flag | Visibility of a simple state change | `private volatile boolean shutdown = false;` |

JAVA

```java
private final AtomicInteger count = new AtomicInteger(0);
public void increment() { count.incrementAndGet(); }

// Alternative in a class, not mixed with the AtomicInteger example:
private int otherCount = 0;
public synchronized void incrementOther() { otherCount++; }
```

**Source-code correction:** The original note says `final int count = 0; count++;` and `final boolean shutdown = false`. Both use `final` incorrectly for a value that needs to change. These corrected versions retain the intended teaching point.

#### Full original notes (preserved)

### Source wording for question 2

**Interview response:** volatile guarantees visibility, not atomicity.

Since count++ is a `read-modify-write` operation, multiple threads can overwrite each other's updates.

Use AtomicInteger or synchronization when you need a correct shared counter.

**Explanation:**

Consider this Java code:

```java
private final int count = 0;

// Called by multiple threads
count++;
```

Although count++ looks like one operation, it actually involves three steps:

1. Read the current value of count.
2. Increment the value by one.
3. Write the new value back to count.

These steps are not atomic.

In Java, volatile provides two important guarantees:

- **Visibility:** When one thread writes a new value to a volatile variable, other threads can observe that write.
- **Ordering:** It establishes memory-ordering guarantees around volatile reads and writes.

However, it does not make compound operations such as count++ atomic.

Think of it this way:

- `volatile` makes individual reads and writes visible.
- `synchronized` or `atomic` classes can make the entire increment operation safe.

**How to fix it?**

**Option A:** Use AtomicInteger (usually the simplest choice)

```java
import java.util.concurrent.atomic.AtomicInteger;

private final AtomicInteger count = new AtomicInteger(0);

public void increment() {
    count.incrementAndGet();
}
```

`incrementAndGet()` performs the increment atomically, so concurrent updates are not lost.

**Option B:** Use synchronized

```java
```

```
private int count = 0;

public synchronized void increment() {
    count++;
}
```

Only one thread at a time can execute this synchronized method on the same object, preventing concurrent increments from interfering.

**Option C:** Use volatile alone

JAVA

```
private final boolean shutdown = false;
```

This is appropriate when you need visibility for simple reads and writes, such as a shutdown flag:

**Official references:** Java `AtomicInteger` API.

---

# 3. When does ConcurrentHashMap still need higher-level coordination?

## 30–60 second answer

`ConcurrentHashMap` makes its individual map operations thread-safe, and methods such as `putIfAbsent()` and `computeIfAbsent()` give atomic operations for a mapping. But that does not mean an entire business workflow is atomic. I still need higher-level coordination when correctness depends on **multiple map calls, multiple keys, external state, or a consistent view of the whole map**. For example, checking `containsKey()` and then calling `put()` leaves a race between those calls. I would replace that with `putIfAbsent()` or `computeIfAbsent()`. For a transfer between two account balances, I would use a suitable lock or a database transaction so both changes succeed or fail together.

## Follow-up

**Why doesn't `computeIfAbsent()` solve every race?**

It makes the computation and insertion atomic **for the relevant map key**; it is not a transaction spanning several keys or a database. Its mapping function should be short, and it must not modify that same map while computing. The function can also fail, leaving the mapping absent. If initializing a resource also writes to a database and publishes an event, I need an explicit workflow: for example, a state machine, coordinated transaction, or reliable outbox. Bulk traversal of a concurrently changing map also does not guarantee one globally consistent snapshot.

## Small example

JAVA

```java
// Race-prone:
if (!map.containsKey("job-1")) map.put("job-1", new Job());

// Atomic insertion of a single key:
map.computeIfAbsent("job-1", key -> new Job());
```

**Remember:** thread-safe operations do not imply a thread-safe sequence.

## Additional details

### Map safety versus workflow safety

`ConcurrentHashMap` protects its internal map operations. It cannot make an unrelated sequence or business invariant atomic.

### All four situations from your notes

1. **Check then act:** `containsKey()` followed by `put()` has a gap where another thread can act. Use `putIfAbsent()` or `computeIfAbsent()` as appropriate.
2. **Multiple keys:** A bank transfer updates two account balances. Each map operation can be safe while the overall transfer is inconsistent; use coordination or a transaction.
3. **Global invariant:** Checking total allocation and adding to `map` and `totalAllocated` must be one coordinated operation to enforce a capacity limit of 100.
4. **Multi-step lifecycle:** Initializing a service, registering it and publishing status needs a lock, `Future`, or explicit state machine when it must happen once.

JAVA

```java
// Check-then-act race:
if (!map.containsKey(key)) {
    map.put(key, createValue());
}

// Atomic computation for one mapping:
map.computeIfAbsent(key, k -> createValue());
```

Keep `computeIfAbsent()` mapping functions short; do not modify other mappings in the same map from the function. Neither this method nor the map itself coordinates external state.

**Full original notes (preserved)**

### Source wording for question 3

**Interview Response:**

ConcurrentHashMap protects its internal data structure and provides atomic operations at the mapping level.

Higher-level coordination is needed when correctness depends on :

1. multiple operations
2. multiple keys,
3. external state, or
4. consistent global view.

**Common situations requiring higher-level coordination**

1. **Check-then-act operations**

   Even though every map operation is thread-safe, the entire check-and-update sequence is not atomic.

   ```java
   if (!map.containsKey(key)) {
     map.put(key, createValue());
   }
   ```

   Another thread can insert the key between the check and the insertion.

   Use `putIfAbsent()` or `computeIfAbsent()` when appropriate:

   ```java
   map.computeIfAbsent(key, k -> createValue());
   ```

   > Note: The mapping function should be short and should not attempt to update other mappings in the same map.

2. **Multiple keys must change together**

   Suppose a bank transfer moves money between two accounts. Each update may be atomic for its own key, but the transfer as a whole isn't atomic. You may need a shared lock, a transaction mechanism, or another design that coordinates both accounts.

3. **A global invariant must remain true**

   Imagine a system where the sum of all allocations must never exceed 100: Making map a `ConcurrentHashMap` does not coordinate the map update with `totalAllocated` or make the capacity check atomic.

   ```java
   if (totalAllocated < 100) {
       map.put(id, allocation);
   ```

```
        totalAllocated += allocation;
    }
```

4. **A multi-step lifecycle must happen only once**

   For example, multiple threads might attempt to initialize a service, register a resource, and publish its status. Atomicity of one map entry doesn't automatically coordinate all those steps.

   Use an appropriate lifecycle mechanism, such as a Future, a lock, or an explicit state machine.

**Official references:** Java `ConcurrentHashMap` API.

---

# 4. What is the difference between thenApply and thenCompose?

## 30–60 second answer

Both methods continue a `CompletableFuture` after an earlier stage produces a result. I use `thenApply()` when I want to **transform a value into another ordinary value**. For example, take a user's name and convert it to uppercase. I use `thenCompose()` when the next function **starts or returns another asynchronous operation**. It flattens the nested future so the result is one `CompletableFuture` rather than a future containing another future. A simple rule is: `thenApply` is like `map`; `thenCompose` is like `flatMap`. Neither method automatically makes blocking code non-blocking; I still need to consider which thread executes the callback.

## Follow-up

**When would you use `thenCombine()` instead?**

Use `thenCompose()` when the second operation **depends on the result of the first**, such as fetching an account after fetching the account ID. Use `thenCombine()` when two futures can be initiated independently and you want to combine their results once both complete, such as retrieving a price and an exchange rate in parallel. Also distinguish `thenApply()` from `thenApplyAsync()`: the latter schedules the function with an asynchronous execution facility, or a supplied executor. An `Async` suffix is not a license to run blocking work on an inappropriate executor.

## Small example

JAVA

```java
CompletableFuture<String> name = CompletableFuture.completedFuture("Sam");

CompletableFuture<String> upper = name.thenApply(String::toUpperCase);

CompletableFuture<User> user = name.thenCompose(this::fetchUserAsync);
// fetchUserAsync(String) returns CompletableFuture<User>
```

**Remember:** `thenApply` transforms; `thenCompose` chains and flattens.

## Additional details

### Two continuation methods

- `thenApply()` accepts the preceding value and returns a new ordinary value. Its result is a `CompletableFuture<T>`.
- `thenCompose()` accepts the preceding value and returns another `CompletionStage<T>`; composition flattens the nested future.

### Original `thenApply()` example, formatted

JAVA

```java
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(() -> "Hello")
        .thenApply(s -> s + " World");

System.out.println(future.join()); // Hello World
```

### Follow-on async operation

JAVA

```java
CompletableFuture<User> user =
    fetchUserIdAsync().thenCompose(id -> fetchUserAsync(id));
```

Choose the method based on **what the continuation returns**, not just whether it runs after another stage. The earlier draft included the first example and the flattening rule; the second code example is an added illustration.

#### Full original notes (preserved)

### Source wording for question 4

In Java's CompletableFuture, the main difference is:

- `thenApply()` transforms the result of a future into another value.

JAVA

```java
CompletableFuture<String> future =
        CompletableFuture.supplyAsync(() -> "Hello")
                .thenApply(s -> s + " World");

System.out.

println(future.join()); // Hello World
```

- `thenCompose()` transforms the result into another future and flattens it into a single future.

**Official references:** Java `CompletableFuture` API.

# 5. Why can JDBC block a Vert.x event loop?

## 30–60 second answer

Vert.x event loops are designed to handle many events with a small number of threads. They work well only when handlers return quickly. Traditional JDBC is a blocking API: executing a query or waiting for a connection can hold the calling thread until the operation finishes. If I call JDBC directly in an event-loop handler, that loop cannot process other requests or timers while it waits. Even a normal database delay can therefore hurt many unrelated requests. I would run blocking JDBC on a worker thread with `executeBlocking()`, or preferably use an asynchronous Vert.x SQL client where appropriate. I would also avoid calling `join()` or `get()` from the event-loop thread.

## Follow-up

**Does wrapping JDBC in `CompletableFuture.supplyAsync()` fix the problem?**

Only if the JDBC work actually runs on a suitable executor **and the event loop never blocks waiting for it**. Calling `supplyAsync(...).join()` from the event loop still blocks that loop. Using the common pool for a large volume of blocking database work can also exhaust the pool's useful capacity, so executor and connection-pool sizing matter. Vert.x `executeBlocking()` places blocking work on a worker pool and returns a future; callbacks can continue asynchronously. A reactive SQL client avoids occupying a thread while the request waits for a database response, but the database query can still be slow.

## Small example

JAVA

```java
vertx.executeBlocking(() -> jdbcQuery())
    .onSuccess(result -> request.response().end(result))
    .onFailure(err -> request.response().setStatusCode(500).end());
```

**Remember:** moving work off-loop helps only if you do not wait on-loop.

## Additional details

### Why it hurts an event loop

A Vert.x event loop serves many connections using relatively few threads. It needs each handler to return quickly. JDBC waits synchronously for a connection or query to finish; delays can come from the pool, locks, network latency, and the database query itself.

### Option A: Offload blocking JDBC

JAVA

```java
vertx.executeBlocking(() -> jdbcQuery())
    .onSuccess(result -> {
        // Continue on the original Vert.x context.
        request.response().end(result);
    })
    .onFailure(err -> request.response().setStatusCode(500).end());
```

Vert.x runs the blocking call on a worker thread and completes the returned future on the original context.

### Option B: Use a reactive SQL client

JAVA

```java
pool.preparedQuery("SELECT name FROM users WHERE id = ?")
    .execute(Tuple.of(userId))
    .onSuccess(rows -> {
        // Continue without waiting synchronously on the event loop.
    });
```

### Trap: asynchronous work followed by a blocking wait

JAVA

```java
CompletableFuture.supplyAsync(() -> jdbcQuery()).join();
```

Although `jdbcQuery()` may execute on another thread, `join()` blocks the thread calling it. If that is an event loop, the problem remains. Compose callbacks instead of waiting.

### Consequences and diagnosis

- **Latency:** unrelated requests wait while the loop is blocked.
- **Throughput:** fewer requests per second are handled.
- **Scalability:** widespread blocked event loops can make the service unresponsive.
- **Diagnostic clue:** Vert.x can log blocked-thread warnings, such as `vertx-eventloop-thread-3 has been blocked`.

**Full original notes (preserved)**

### Source wording for question 5

#### How the Vert.x event loop works

A Vert.x event loop typically handles events for many connections on a small number of threads. As such, they depend on handlers returning quickly so it can continue processing other requests, timers, and I/O events.

Traditional JDBC operations are synchronous (blocking). So, even if the database is healthy, network latency, connection-pool waits, locks, or a slow query can keep the thread blocked.

The problem isn't necessarily the database being slow. It's that the event loop is waiting synchronously.

**How to fix it?**

**Option A: Use executeBlocking ()**

EXAMPLE

```
vertx.executeBlocking(() -> {
    return jdbcQuery(); // Runs on a worker thread
}).onSuccess(result -> {
    // Handle result on the original Vert.x context
    request.response().end(result);
}).onFailure(err -> {
    request.response().setStatusCode(500).end();
});
```

This keeps the JDBC call off the event loop. Vert.x runs the blocking callable in its worker pool and completes the returned future on the original context.

**Option B: Use a reactive database client**

For a more naturally non-blocking architecture, use an appropriate Vert.x reactive database client, such as a reactive SQL client supported for your database.

EXAMPLE

```
pool.preparedQuery("SELECT name FROM users WHERE id = ?")
    .execute(Tuple.of(userId))
    .onSuccess(rows -> {
        // Handle results without blocking the event loop
    });
```

Here, the database operation is asynchronous: the event loop can process other events while waiting for the database response.

### Does wrapping JDBC in a CompletableFuture solve it?

Not by itself.

This is still dangerous when called from a Vert.x event loop:

EXAMPLE

```
CompletableFuture.supplyAsync(() -> jdbcQuery())
   .join();
```

Although `supplyAsync()` usually runs the JDBC operation on a different thread, `join()` blocks the calling thread while waiting for the result. If that calling thread is an event loop, you still have the same problem.

Instead, compose asynchronous stages without blocking: This avoids blocking the event loop, provided the callback itself doesn't perform blocking work.

EXAMPLE

```
vertx.executeBlocking(() -> jdbcQuery())
.onSuccess(result -> {
// Continue asynchronously
});
```

**Why does this matter for performance?**

**Latency:** Other requests on the blocked event loop wait longer.

**Throughput:** Fewer requests can be processed per second.

**Scalability:** If all event loops become blocked, the entire Vert.x instance can become unresponsive.

**Diagnostics:** Vert.x can log warnings such as vertx-eventloop-thread-3 has been blocked

**Official references:** Vert.x Core Manual — Don't block the event loop.

# 6. What does Spring Boot add to Spring?

## 30–60 second answer

Spring is the underlying framework for dependency injection, application components, web development and other infrastructure. Spring Boot builds on it to reduce setup and provide a simpler way to run applications. Its main additions are **auto-configuration**, **starter dependencies**, **embedded server support**, **externalized configuration**, and **production features through Actuator**. For example, adding a web starter lets Boot configure much of a typical web application based on the dependencies present, while allowing me to override defaults. I can often package and run the application directly as an executable JAR. Boot does not replace Spring; it makes common Spring application setups faster and more consistent.

## Follow-up

**How does auto-configuration know what to create?**

Boot evaluates conditions, including which classes and beans exist and which configuration properties are set. If relevant libraries are available and I have not supplied my own bean, Boot may configure a default. If I define a compatible explicit bean, the auto-configuration can back off. `@SpringBootApplication` enables key

application setup, including auto-configuration and component scanning. Starter dependencies help assemble compatible libraries; they are not themselves the runtime mechanism that creates every bean. Actuator exposes operational features such as health and metrics, subject to dependencies and endpoint exposure settings.

## Small example

JAVA

```java
@SpringBootApplication
public class App {
    static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

With appropriate web dependencies, Boot can set up an embedded web server and the surrounding web infrastructure.

## Additional details

### What Spring Boot adds to Spring

Spring remains the core framework. Boot reduces setup and deployment work through defaults and conventions.

| Addition | Meaning | Example or outcome |
| --- | --- | --- |
| **Auto-configuration** | Configures components based on available libraries and application settings | Beans for an appropriate web stack |
| **Starters** | Groups useful dependencies | `spring-boot-starter-web` |
| **Embedded servers** | Packages the app with a server when applicable | Run a Spring MVC service using `java -jar` with embedded Tomcat |
| **Actuator** | Adds production-oriented health, metrics and diagnostic endpoints | Health checks |
| **External configuration and conventions** | Supports properties, YAML, environment and CLI arguments with overridable defaults | `application.yaml` |

### What to emphasize in an interview

Boot **does not replace Spring**. It makes Spring applications quicker to set up, run and operate. A starter supplies dependencies; auto-configuration is the separate mechanism that chooses configuration based on conditions.

**Full original notes (preserved)**

### Source wording for question 6

Spring Boot makes Spring applications easier to configure, run, and deploy.

Spring provides the core framework and features; Spring Boot adds sensible defaults, automatic configuration, and tools that reduce the amount of manual setup.

There are five major additions.

1. **Auto-configuration** Spring Boot automatically configures beans and components based on the libraries on your classpath and the settings you provide.
2. **Starter dependencies** Starters bundle commonly used dependencies so you don't have to select every library individually.

   For example, `spring-boot-starter-web` brings together the dependencies commonly needed for a Spring MVC web application.
3. **Embedded servers** Boot can package an application with an embedded server, such as Tomcat, so you can run it directly with java -jar instead of deploying a WAR to a separately installed server.
4. **Production-ready features** Spring Boot Actuator provides features such as health checks, application metrics, and diagnostic endpoints, depending on configuration.
5. **Externalized configuration and conventions** Boot makes it convenient to configure applications through `application.properties`, `application.yaml`, environment variables, and command-line arguments. It also provides sensible defaults that you can override.

**Official references:** Boot auto-configuration · Boot Actuator.

# 7. Why might @Transactional not take effect?

## 30–60 second answer

The first thing I check is whether the method call went through a **Spring-managed transaction proxy**. In Spring's default proxy mode, a method calling another annotated method on the same object bypasses the proxy, so the inner annotation does not start a new transactional boundary. I also check that Spring created the bean, the method can be intercepted, and the correct transaction manager is configured. Sometimes the transaction exists but the result looks wrong because of rollback rules: by default, Spring rolls back for unchecked exceptions and errors, not ordinary checked exceptions. Finally, asynchronous work or operations on unrelated database connections may be outside the transaction.

## Follow-up

**How would you debug a transaction that unexpectedly committed?**

I would check whether the method was invoked through a proxy and whether the transaction was active on that thread. Then I would inspect the exception: was it checked, caught and swallowed, or deliberately configured not to roll back? For a checked exception, I can use `@Transactional(rollbackFor = Exception.class)` when that behavior is required. I would also verify propagation settings such as `REQUIRED` versus `REQUIRES_NEW`, the chosen transaction manager, and whether the database work uses an enlisted connection. These are separate failure modes; an annotation alone cannot make unrelated external services part of one transaction.

## Small example

JAVA

```java
@Service
class OrderService {
    void placeOrder() { save(); }     // Self-call bypasses proxy advice

    @Transactional
    public void save() { /* persist order */ }
}
```

Move `save()` to an injected Spring-managed `OrderWriter` service and call it through that bean, or design the public transactional boundary on `placeOrder()` itself.

## Additional details

### Troubleshoot in this order

Your original notes identify **10 separate reasons**. Group them by the kind of failure so you can debug quickly without forgetting any.

### A. The transactional advice never runs

1. **Self-invocation:** A method calls another method in the same object (`saveOrder()` or `this.saveOrder()`); the call skips the Spring proxy.
2. **Method/class not proxyable:** Standard Spring proxies cannot advise private or static methods; class-based proxies cannot override final methods or subclass final classes. Visibility support depends on proxy strategy and Spring version.
3. **Bean created with `new`:** It is not the proxied, Spring-managed instance.
4. **Transaction management misconfigured:** Check transaction enablement and the actual bean and proxy configuration.

JAVA

```java
@Service
class OrderService {
    public void processOrder() { saveOrder(); } // Self-call; skips proxy advice

    @Transactional
```

```
    public void saveOrder() { /* database operations */ }
}
```

**Fix:** Call the transactional method through a Spring-managed service, or define the transaction on an eligible proxied public entry method.

JAVA

```
@Service
class OrderWriter {
    @Transactional
    public void saveOrder() { /* database operations */ }
}

@Service
class OrderService {
    private final OrderWriter writer;
    OrderService(OrderWriter writer) { this.writer = writer; }
    public void processOrder() { writer.saveOrder(); }
}
```

### B. It runs but does not roll back as expected

1. **Exception/rollback rules:** Spring normally rolls back for `RuntimeException` and `Error`, but not checked exceptions by default. An exception that is caught and swallowed may lead to a commit.
2. **Wrong transaction manager:** With multiple data sources, the selected manager might not protect the repository in question.
3. **Propagation settings:** `REQUIRED`, `REQUIRES_NEW`, `SUPPORTS`, `NOT_SUPPORTED`, and `NEVER` affect whether work joins, starts, suspends, or avoids a transaction.
4. **Operation outside the transaction:** Different connections, unenlisted resources, an unsupported storage engine, autocommit, replicas or a later async action can make a correctly active transaction appear ineffective.

JAVA

```
// Roll back on a checked exception if that is the required policy:
@Transactional(rollbackFor = Exception.class)
public void saveOrder() throws IOException {
    repository.save(order);
    throw new IOException("Failed");
}

// Select the manager that protects the intended resource:
@Transactional("ordersTransactionManager")
public void saveOrderWithManager() { /* ... */ }

// Explicitly run without a transaction:
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void updateOrder() { /* ... */ }
```

**Propagation at a glance**

| Setting | Behavior |
| --- | --- |
| `REQUIRED` | Join existing or start a new transaction |
| `REQUIRES_NEW` | Suspend existing and start a new transaction |
| `SUPPORTS` | Join if there is one; otherwise proceed without |
| `NOT_SUPPORTED` | Suspend existing; proceed without one |
| `NEVER` | Fail if a transaction exists |

### C. The boundary or framework does not match

1. **Async/thread boundary:** Imperative Spring transactions are generally thread-bound. `@Async` work does not automatically share the caller's transaction; create an appropriate transaction in the worker if needed. Reactive code uses a reactive transaction manager and context.
2. **Wrong annotation/infrastructure:** Distinguish Spring's `org.springframework.transaction.annotation.Transactional`, Jakarta's `jakarta.transaction.Transactional`, and legacy `javax.transaction.Transactional`. Check JDBC/JPA, JTA or reactive setup.

**Other original cautions retained:** Catching and logging an exception without rethrowing can commit; neither Spring annotations nor propagation automatically make separate databases or external services one atomic transaction. The full original code and troubleshooting list appear below.

#### Full original notes (preserved)

### Source wording for question 7

Spring's transaction management is usually proxy-based. The method must be invoked through the `Spring-managed proxy` for the transaction advice to run.

Here are the most common causes, how to identify them, and how to fix them.

1. **Self-invocation (the most common cause)** Calling a transactional method from another method in the same class directly using `this` and bypasses the Spring proxy.

JAVA

```
@Service
public class OrderService {

    public void processOrder() {
        saveOrder(); // Self-invocation: transaction may not start
    }

    @Transactional
    public void saveOrder() {
```

```
            // Database operations
        }
    }
```

**Fix:** Move transactional method into another Spring-managed service and inject it, or call the method through a proxy.

JAVA

```
@Service
public class OrderWriter {

    @Transactional
    public void saveOrder() {
        // Database operations
    }
}

@Service
public class OrderService {

    private final OrderWriter orderWriter;

    public OrderService(OrderWriter orderWriter) {
        this.orderWriter = orderWriter;
    }

    public void processOrder() {
        orderWriter.saveOrder(); // Goes through the proxy
    }
}
```

2. **The method is not eligible for proxy interception**

   Depending on the proxy mechanism and Spring version, method visibility and class structure can prevent interception.

   - **Private methods:** Cannot be intercepted by standard proxy-based transaction management.
   - **Final methods:** Cannot be overridden by class-based proxies, so transaction advice cannot be applied to them.
   - **Final classes:** Cannot be subclassed by class-based proxies, although interface-based proxies may still work.
   - **Static methods:** Are not intercepted by standard Spring transaction proxies.

   **Fix:** Use an eligible, typically public, instance method on a non-final Spring-managed class. Package-private and protected methods can work with class-based proxies in modern Spring versions, but behavior depends on the proxy strategy and configuration.

3. **The object was created with new** Spring can apply transactions only when the invocation passes through the relevant Spring proxy.

JAVA

```
OrderService service = new OrderService(); // Not Spring-managed
service.saveOrder();
```

**Fix:** Let Spring create and inject the bean. Inject it into the calling component instead of constructing it manually.

JAVA

```
@Service
public class OrderService {

    @Transactional
    public void saveOrder() {
        // ...
    }
}
```

4. **The exception does not trigger a rollback**

   By default, Spring rolls back on `RuntimeException` and `Error`, but not on **checked exceptions**.

JAVA

```
@Transactional
public void saveOrder() throws IOException {
    repository.save(order);
    throw new IOException("Failed");
}
```

The transaction may commit despite the checked exception, depending on how the exception is handled and transaction completion proceeds.

**Fix:** Specify rollback rules when needed.

JAVA

```
@Transactional(rollbackFor = Exception.class)
public void saveOrder() throws IOException {
    repository.save(order);
    throw new IOException("Failed");
}
```

Also watch for exceptions that are caught and swallowed:

JAVA

```
@Transactional
public void saveOrder() {
    try {
        repository.save(order);
```

```
            performOperation();
        } catch (Exception e) {
            log.error("Operation failed", e);
            // No exception escapes; Spring may commit
        }
    }
```

If the transaction should roll back, rethrow the exception or explicitly mark the transaction rollback-only.

5. **Transaction management is not configured correctly**
6. **The wrong transaction manager is being used**

Applications with multiple data sources or transaction managers can accidentally start a transaction against the wrong resource.

JAVA

```
@Transactional("ordersTransactionManager")
public void saveOrder() {
    // ...
}
```

**Fix:** Specify the correct transaction manager explicitly, if necessary, and verify that the repository and transaction manager operate on the intended data source.

Also, transactions do not automatically span arbitrary independent databases or external services. That requires appropriate distributed transaction infrastructure or a different consistency strategy.

1. **Transaction propagation changes the behavior**

Propagation settings can cause a method to join an existing transaction or start a separate one.

- `REQUIRED` — joins an existing transaction or creates one.
- `REQUIRES_NEW` — suspends an existing transaction and starts a new one.
- `SUPPORTS` — participates in a transaction if one exists, but does not require one.
- `NOT_SUPPORTED` — executes without a transaction, suspending an existing one if applicable.
- `NEVER` — requires that no transaction exists.

**For example:**

JAVA

```
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void updateOrder() {
    // Executes without a transaction
}
```

**Fix:** Check the propagation setting on both the caller and the called method, not just the annotation on the method you're debugging.

1. **The transaction is active, but the data is not rolled back**

Sometimes `@Transactional` works correctly, but the observed behavior makes it appear ineffective.

**Common reasons include:**

- The operation uses a different database connection or data source.
- A database operation is performed through a resource not enlisted in the transaction.
- The database or storage engine does not support the expected transactional behavior.
- The code commits manually or uses autocommit outside Spring's transaction management.
- The code reads from a replica or another connection and observes different state.
- The transaction commits before an asynchronous operation executes.

**Fix:** Verify the actual connection, transaction manager, database resource, and transaction boundaries. Do not assume that annotating a method makes every operation it invokes transactional.

1. **Async execution or thread boundaries**

Imperative Spring transactions are generally thread-bound. They do not automatically propagate to another thread.

JAVA

```
@Transactional
public void processOrder() {
    asyncService.processAsync(); // Runs on another thread
}
```

If `processAsync()` uses @Async, its work generally runs outside the caller's transaction unless it establishes its own transaction.

**Fix:** Put `@Transactional` on the method executed by the async worker if that work needs a transaction. For reactive applications, use the appropriate reactive transaction manager and preserve the reactive execution context.

1. **The wrong annotation or framework is being used**

Make sure the annotation matches the transaction infrastructure:

- `org.springframework.transaction.annotation.Transactional` — Spring's annotation.
- `jakarta.transaction.Transactional` — Jakarta Transactions annotation, supported in appropriate Spring configurations.
- `javax.transaction.Transactional` — legacy Java EE annotation, relevant to older stacks.

Also verify whether you are using Spring JDBC/JPA, JTA, or reactive persistence, since their transaction requirements differ.

**Official references:** Spring transaction annotations and proxy behavior.

# 8. How do you detect and fix N+1 queries?

## 30–60 second answer

N+1 is a database access pattern where I load N parent records with one query, and then issue an additional query for each parent's related data. For example, loading 100 orders and lazily reading each order's customer can result in 101 queries. To detect it, I enable SQL logging or query-count metrics and observe how the query count grows when the number of returned records grows. To fix it, I choose a fetching strategy that matches the use case: a join fetch or entity graph for the needed relations, batch fetching for multiple lazy associations, or a projection that selects only required columns. I verify the improvement with a realistic dataset.

## Follow-up

**Why not make every relationship eager?**

Eager loading can over-fetch unnecessary data and may still result in extra SQL selects, depending on the query and ORM fetching strategy. Fetch-joining several to-many associations can multiply rows and complicate pagination. I prefer explicit fetching for a particular read path instead of making the whole model eager. For a list API, a DTO projection is often efficient because it avoids building a large object graph. I would compare SQL count, rows returned, total latency and memory, not just the number of queries. The correct solution depends on whether the relationship is to-one, to-many, paginated, or used by multiple endpoints.

## Small example

JAVA

```java
// JPQL: load orders and their customer in one SQL join
@Query("select o from Order o join fetch o.customer")
List<Order> findOrdersWithCustomer();
```

For a **to-one** relation this can avoid per-order lazy loads. Test carefully before fetch-joining **to-many** collections with pagination.

## Additional details

**Added explanation — the original answer was blank.**

### Detect an N+1 pattern (added material; the original answer was blank)

1. Load the parent collection and record SQL/query-count metrics.

2. Access the related data for each parent; observe whether each access triggers another select.
3. Repeat with a larger result set: if query count grows approximately `1 + N`, you may have N+1.

### Fix only the read path that needs it

- `JOIN FETCH` or an entity graph for required relations.
- Batch fetching for lazy relationships.
- DTO projection for list endpoints that need only a few fields.

Avoid changing all associations to eager fetching. To-many joins can multiply rows and interfere with pagination; compare query counts, total rows, memory and response time.

JAVA

```java
@Query("select o from Order o join fetch o.customer")
List<Order> findOrdersWithCustomer();
```

This is an illustration for a to-one relationship, not a universal solution.

**Full original notes (preserved)**

### Source wording for question 8

---

**Official references:** Hibernate User Guide — fetching strategies.

---

# 9. How do you prevent lost updates?

## 30–60 second answer

A lost update occurs when two requests read the same value and later one write overwrites the other request's change. The common case is two users editing the same record based on an old version. My default choice for ordinary JPA entity updates is **optimistic locking** with a `@Version` column. The database update checks that the version has not changed; if another transaction already updated the row, the later update fails with a conflict instead of silently overwriting data. I can then ask the caller to reload or safely retry. For counters, I prefer an atomic database update, and for high-contention workflows I consider row locks.

## Follow-up

**Optimistic or pessimistic locking — which would you choose?**

Optimistic locking is a good starting point when conflicts are infrequent, because reads do not require a long-held exclusive lock. A conflict becomes visible at update or commit and the application must decide whether a retry is safe. Pessimistic locking, such as `SELECT ... FOR UPDATE`, can be appropriate when a short transaction

must serialize access to a row, but it increases waiting and deadlock risk. For simple increments, a single SQL statement is often better than read-then-write. I also check transaction boundaries and isolation: simply putting several requests in transactions does not automatically prevent every lost-update scenario.

## Small example

JAVA

```java
@Entity
class Product {
    @Id Long id;
    @Version Long version;
    int stock;
}
```

If two transactions loaded version 3, the first can update it to version 4; the second then gets an optimistic-lock conflict instead of silently replacing the first update.

## Additional details

**Added explanation — the original answer was blank.**

### What a lost update looks like (added material; the original answer was blank)

EXAMPLE

```
Read by A: stock = 10
Read by B: stock = 10
A writes 9
B writes 9 -> A's change is lost (expected 8)
```

### Three approaches

| Strategy | When useful | Trade-off |
| --- | --- | --- |
| **Optimistic locking (`@Version`)** | Conflicts are uncommon | Detect conflict and retry or return an error |
| **Pessimistic locking (`FOR UPDATE`)** | Short, high-contention changes needing serialization | More waiting and possible deadlocks |
| **Atomic SQL update** | Counter or single-field adjustment | Operation must fit one statement |

JAVA

```java
@Entity
class Product {
    @Id Long id;
    @Version Long version;
```

```
    int stock;
}
```

If two transactions loaded the same version, one update can advance the version and the other receives a conflict instead of silently overwriting. Do not blindly retry workflows that create side effects.

**Full original notes (preserved)**

### Source wording for question 9

---

**Official references:** Hibernate optimistic locking.

---

# 10. Why can a LEFT JOIN unexpectedly remove unmatched rows?

## 30–60 second answer

A `LEFT JOIN` normally returns every row from the left table, even when there is no matching row on the right. For an unmatched row, columns from the right table become `NULL`. The surprise comes when I add a condition on the right table in the `WHERE` **clause**, such as `WHERE payment.status = 'PAID'`. For unmatched rows, that comparison is not true, so the `WHERE` clause removes them. The query then behaves like an inner join for that condition. If I want all left-side rows but only qualifying matches from the right table, I put the right-side filter in the `ON` **clause** instead.

## Follow-up

**When is a right-table condition in `WHERE` correct?**

It is correct when the business question intentionally wants only rows that have a qualifying match. For example, "customers who have a paid order" legitimately excludes customers without a paid order. But "all customers, showing paid orders if any" requires a `LEFT JOIN` with the paid filter in `ON`. There is another useful pattern: `LEFT JOIN ... WHERE right.id IS NULL` to find left-side rows with **no matching right-side record**. I always translate the SQL back into the expected business result and test it with one matched row and one unmatched row.

## Small example

SQL

```sql
-- Keeps customers even if they have no PAID order:
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id AND o.status = 'PAID';
```

**Remember:** `ON` affects matching; `WHERE` filters the result after the join.

## Additional details

### Why unmatched rows disappear

`LEFT JOIN` preserves all left-table rows initially. When no right-side record matches, that side's columns are `NULL`. A right-table condition in `WHERE`, such as `o.status = 'PAID'`, removes those null-extended rows.

### Compare the two queries

SQL

```sql
-- Keeps only customers with a paid order:
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'PAID';

-- Keeps all customers; attaches paid orders when available:
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id AND o.status = 'PAID';
```

The original note identifies `WHERE` versus `ON` as the key cause. Use `WHERE` when excluding nonmatches is intentional. Use `ON` for a condition that should restrict matched right-side rows without dropping left-side rows.

#### Full original notes (preserved)

### Source wording for question 10

A LEFT JOIN can unexpectedly remove unmatched rows when a condition in the WHERE clause filters out the NULL values generated for rows that have no match on the right-hand table.

The most common cause is putting a condition on the right-hand table in WHERE instead of ON.

**Official references:** PostgreSQL table expressions and joins.

# 11. How do you invalidate Redis data safely enough for the use case?

## 30–60 second answer

I start with the business requirement: how stale can the data be, and what happens if it is wrong? For ordinary read-heavy data, I often use **cache-aside**: read from Redis first, load from the database on a miss, and store the value with a TTL. When a write succeeds in the database, I invalidate the specific Redis key so the next

reader reloads it. The TTL limits how long a forgotten or missed invalidation can persist. I also plan for races where a concurrent reader loads an old value and repopulates the cache after invalidation. For sensitive data, such as authorization or balances, I use stronger consistency controls or bypass the cache.

## Follow-up

**What is the invalidation race and how do you prevent it?**

Imagine reader A sees a cache miss and starts loading version 1 from the database. Writer B commits version 2 and deletes the cache key. Then A finishes late and writes version 1 back into Redis. The key is stale **even though deletion happened after the database write**. Possible controls include a versioned cache key, storing and checking a record version, serializing loads and writes for the key, or using change events with carefully designed ordering. Exact choice depends on tolerated staleness. For hot keys, a short-lived request-coalescing mechanism can reduce stampedes. A TTL is a safety net, not proof of immediate consistency.

## Small example

EXAMPLE

```
GET product:42 -> cache miss
DB SELECT product 42 -> price 20
SET product:42 (value=20, TTL=60s)

On successful price update:
DB UPDATE product 42 -> price 25
DEL product:42
```

This is the basic cache-aside flow; production code must address concurrent repopulation.

## Additional details

### First establish the acceptable consistency level

Your original note asks three questions:

1. **How stale may cached data be?**
2. **How costly is rebuilding it after invalidation?**
3. **What is the consequence of showing a briefly stale value?**

### Your three rules, made concrete

- **Invalidate narrowly:** delete the exact affected key rather than a broad collection of unrelated keys.
- **Make staleness detectable:** include a version, freshness timestamp or controlled TTL when it helps the use case.
- **Prevent stale repopulation:** a concurrent read may load an old database value and write it into Redis after the writer invalidated the key.

### Cache-aside example (added follow-up)

EXAMPLE

```
GET product:42 -> cache miss
DB SELECT -> value 20
SET product:42 to 20 with a TTL

DB UPDATE -> new value 25
DEL product:42 after successful commit
```

For stronger consistency, use a version check, ordered invalidation mechanism or a design that avoids caching sensitive values. A TTL limits staleness duration but does not eliminate write/read races.

**Full original notes (preserved)**

### Source wording for question 11

The safest way to invalidate Redis data depends on

- **how stale the data can be**,
- **how expensive it is to rebuild**, and
- **what happens if a stale value is served briefly**.

**The general principle is:**

- invalidate narrowly,
- make stale data detectable, and
- ensure concurrent requests cannot repopulate the cache with outdated values.

**Official references:** Redis cache-aside pattern.

# 12. What must you validate in a JWT besides its signature?

## 30–60 second answer

A valid JWT signature only proves that the token was signed with an accepted key and has not been modified. It does **not** automatically mean the token should be accepted by my API. I also validate its expected **issuer**, **audience**, and time-based claims such as **expiration** and **not-before**. I restrict the allowed signing algorithms, select keys from a trusted source, and confirm the token type and intended use, such as access token versus ID token. Then I validate application-specific identity, scopes or roles and resource permissions. If my system supports revocation, I also check whether the token or session has been revoked.

## Follow-up

### Why is an audience check important?

An identity provider might issue tokens for several APIs. A token that has a valid signature could still be intended for a different service. Checking `aud` prevents another application's token from being accepted by mine. Similarly, `iss` binds the token to a trusted issuer, and the signing key must belong to that issuer. I must not let the token header arbitrarily choose an algorithm or an untrusted remote key source. `exp` and `nbf` are time validity checks; small clock-skew allowances may be configured. The claims identify and constrain the caller, but authorization must still be enforced at the requested resource.

## Small example

JSON

```
{
  "iss": "https://auth.example.test",
  "aud": "orders-api",
  "sub": "user-123",
  "exp": 1900000000
}
```

The `orders-api` must reject this token if its issuer is untrusted, it is expired, or it is not an intended audience—even if its signature verifies.

## Additional details

**Added explanation — the original answer was blank.**

### Validate token context, not just cryptography (added material; the original answer was blank)

- **Key and algorithm:** use a trusted signing key and an explicitly allowed algorithm; reject untrusted key selection.
- **Issuer (`iss`):** matches the expected identity provider.
- **Audience (`aud`):** this API is an intended receiver.
- **Time claims:** check `exp` and `nbf` with the configured clock-skew policy.
- **Token type/use:** do not accidentally accept an ID token where an access token is required.
- **Identity and permissions:** validate required subject/claims and still authorize the exact requested action and resource.
- **Revocation/session state:** apply checks where supported by your system.

JSON

```
{"iss":"https://auth.example.test", "aud":"orders-api", "sub":"user-123", "exp":1900000000}
```

A correct signature with `aud = another-api` is **not** sufficient to access `orders-api`.

Full original notes (preserved)

### Source wording for question 12

**Official references:** RFC 7519: JWT claims · RFC 8725: JWT security guidance.

# 13. How do you prevent cross-tenant access?

## 30–60 second answer

In a multi-tenant system, I apply **defense in depth**. I first authenticate the user, then verify that they belong to the tenant they want to access and have permission for the requested resource. The backend creates a trusted tenant context; it must not assume that a tenant ID sent in a header or URL is authorized. Every data-access operation must enforce that scope. In a shared PostgreSQL database, I would use tenant-filtered queries backed by row-level security where practical. I would also namespace cache keys and file paths by tenant, carry tenant context into background jobs, and run automated negative tests proving tenant A cannot access tenant B's data.

## Follow-up

**Is `WHERE tenant_id = ?` enough?**

It is necessary in many designs, but not enough on its own. A developer could forget the predicate, a background job could reuse the wrong tenant context, or a cache key could omit the tenant. PostgreSQL row-level security adds a database enforcement layer. When using a transaction-scoped setting such as `app.tenant_id`, set it from an **already verified backend identity** inside the transaction and use a database role that cannot bypass RLS. Superusers and `BYPASSRLS` roles bypass those policies; table owners normally bypass them unless `FORCE ROW LEVEL SECURITY` applies. I would test reads, writes, files, caches and pooled connections.

## Small example

SQL

```sql
-- tenant_id must come from trusted, authorized server context:
SELECT id, title
FROM documents
WHERE id = :document_id
  AND tenant_id = :verified_tenant_id;
```

Even if a user guesses another document ID, the query does not return data outside the verified tenant.

## Additional details

### The defense-in-depth rule from your notes

Authenticate the caller, authorize tenant membership and permissions, and enforce tenant isolation at every data boundary. **Never trust an unverified `tenant_id` sent by the frontend.**

### Security controls across the full stack

| Layer | What must be enforced |
|---|---|
| Authentication | Verify identity and session/token |
| Authorization | Verify tenant membership and resource permissions |
| API | Enforce verified tenant scope to prevent IDOR/BOLA |
| Database | Tenant-filtered queries and, when suitable, RLS or separated tenant databases |
| Cache | Tenant-prefixed keys |
| Object storage | Tenant-scoped access to files |
| Background jobs | Carry and validate the correct tenant context |
| Monitoring | Detect and log cross-tenant access attempts |

### Request flow and ownership of tenant context

EXAMPLE

```
Client request (may propose X-Tenant-ID)
    |
    v
Validate JWT / authenticate user
    |
    v
Verify selected tenant membership + resource permissions
    |
    v
Create trusted, server-side TenantContext
    |
    +--> API authorization
    +--> Tenant-scoped SQL / RLS
    +--> Cache and object storage
    +--> Background-job tenant context
    |
    v
Return only authorized data
```

Tenant selection can arrive through a token, session, URL or header; **authorization is decided by the backend**. A valid JWT does not authorize access to arbitrary tenant IDs.

### Database isolation: unsafe versus safer

SQL

```sql
-- Unsafe: only a resource ID is checked.
SELECT * FROM documents WHERE id = :document_id;

-- Safer: verified tenant ID must also match.
SELECT * FROM documents
WHERE id = :document_id
  AND tenant_id = :verified_tenant_id;
```

### PostgreSQL RLS as the second guard

SQL

```sql
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON documents
USING (tenant_id = current_setting('app.tenant_id')::uuid)
WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);
```

Set the tenant context from a **verified identity** within each transaction:

SQL

```sql
BEGIN;
SELECT set_config('app.tenant_id', :verified_tenant_id, true);
SELECT * FROM documents;
COMMIT;
```

The `true` argument makes it transaction-local. Use a role that cannot bypass RLS. PostgreSQL superusers and `BYPASSRLS` roles bypass policies; be mindful of table-owner behavior. Reused connections and background workers must not leak old tenant context.

### Negative tests from your notes

- Tenant A cannot **read, update or delete** Tenant B's rows.
- Changing object ID or tenant ID does not bypass server authorization.
- Cache entries and stored files do not leak across tenants.
- Pooled database connections do not retain the previous tenant's context.
- Background jobs cannot execute with the wrong tenant.

The original recommended three-part baseline is **deny-by-default application authorization + database tenant scoping/RLS + automated negative tests in CI/CD**. Both original ASCII request diagrams and the more detailed RLS SQL are retained below.

**Full original notes (preserved)**

### Source wording for question 13

Use defense-in-depth approach.

The core rule is:

**Every request must be authenticated, authorized, and enforce tenant isolation at every data-access boundary.**

Never rely on a single tenant_id filter or frontend access control.

1. **Key security controls**

| Layer | Control | Purpose |
|---|---|---|
| Authentication | Validate identity and session | Identify the caller |
| Authorization | Verify tenant membership and permissions | Prevent unauthorized tenant access |
| API | Enforce tenant scope server-side | Prevent IDOR/BOLA attacks |
| Database | Row-Level Security (RLS) or tenant-isolated databases | Block cross-tenant queries |
| Cache | Namespace keys by tenant | Prevent cached data leakage |
| Storage | Tenant-scoped object access | Protect files across tenants |
| Background Jobs | Carry and validate tenant context | Prevent asynchronous data leakage |
| Monitoring | Log and alert on tenant access violations | Detect attempted or accidental breaches |

1. **Enforce tenant isolation on every request** A secure request flow looks like this:

EXAMPLE

```
Client Request
      |
      v
Authenticate User
      |
      v
Resolve Tenant Context
      |
      v
Verify Tenant Membership
```

```
      |
      v
Check Resource Permissions
      |
      v
Apply Database Tenant Isolation
      |
      v
Return Authorized Data
```

**Important:** Never trust a tenant_id supplied by the client without verifying that the authenticated user is allowed to access that tenant.

2. **Example: Safe database access**

   Unsafe — resource ID only

```sql
SELECT *
FROM documents
WHERE id = :document_id;
```

   Safer — tenant-scoped query

```sql
SELECT *
FROM documents
WHERE id = :document_id
  AND tenant_id = :verified_tenant_id;
```

   The verified_tenant_id must come from a trusted, authorized server-side context.

   **For PostgreSQL,** add `Row-Level Security (RLS)` as another protection layer:

```sql
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

CREATE
POLICY tenant_isolation
ON documents
USING (
  tenant_id = current_setting('app.tenant_id')::uuid
)
WITH CHECK (
  tenant_id = current_setting('app.tenant_id')::uuid
);
```

Set the tenant context from the verified identity within the database transaction. Use an application database role that cannot bypass RLS, and reset transaction context before reusing connections.

PostgreSQL documents important RLS exceptions, including superusers and roles with BYPASSRLS.

3. **Test cross-tenant isolation** At minimum, test these scenarios:
   - Tenant A cannot read Tenant B's records.
   - Tenant A cannot update or delete Tenant B's records.
   - Changing object IDs or tenant IDs does not bypass authorization.
   - Cache and file access cannot expose another tenant's data.
   - Reused database connections do not retain a previous tenant's context.
   - Background jobs cannot run under an incorrect tenant context.
4. **Recommended approach** For a SaaS application with a shared database, use three independent controls:
   1. **Application:** Centralized, deny-by-default authorization.
   2. **Database:** Tenant-scoped queries backed by RLS.
   3. **Testing:** Automated cross-tenant negative tests in CI/CD.

   The main principle is **tenant isolation must remain effective even when one application-layer check is missed.**

### How tenant scope is set

Tenant scope is set by the **backend after authenticating the user and verifying their access to the tenant.**

The frontend may request a tenant, but it must never decide which tenant the user is authorized to access.

1. **Request flow**

EXAMPLE

```
Client Request
    |
    | Authorization: Bearer <JWT>
    | X-Tenant-ID: tenant_123
    v
API Gateway / Backend
    |
    v
1. Validate JWT
    |
    v
2. Identify user
    |
    v
3. Verify user belongs to tenant_123
    |
    v
4. Create trusted TenantContext
    |
    | user_id   = user_456
    | tenant_id = tenant_123
    | role      = admin
```

```
    v
5. Enforce tenant scope
    |
    +---- API authorization
    |
    +---- Database queries / RLS
    |
    +---- Cache and storage
    |
    v
Return tenant-specific data
```

2. **Where does tenant_id come from?**

   Select tenant from token, session, URL, or header. Store verified tenant_id server-side

   **Important:** A valid JWT proves the token is authentic. It does not automatically prove access to every tenant requested by the caller.

3. **How is tenant scope applied to the database?** For `PostgreSQL with Row-Level Security (RLS)`, tenant scope is typically set **per transaction**.

   POSTGRESQL

   ```postgresql
   BEGIN;
   -- Set verified tenant context
   SELECT set_config(
   'app.tenant_id',
   :verified_tenant_id,
   true
   );

   -- RLS automatically filters rows
   SELECT * FROM documents;

   COMMIT;
   ```

The `true` argument makes the setting transaction-local, preventing tenant context from unintentionally carrying into a later transaction on a reused connection.

This assumes RLS policies are configured and the database role cannot bypass them.

**Official references:** PostgreSQL row-level security.

# 14. What does a payment timeout actually tell you?

## 30–60 second answer

A payment timeout tells me that **my system did not receive a definitive response within the allowed time**. It does not prove that the charge failed. The payment provider may have received and completed the request, but the response could have been delayed or lost. Therefore, blindly retrying with a new request can create a

duplicate charge. I treat the payment state as **unknown or pending reconciliation**, not automatically failed. I use an idempotency key for safely retrying the same logical operation, check the payment status using the provider's authoritative API or supported webhooks, and record the final state reliably before telling the customer the outcome.

## Follow-up

**What would a reliable payment flow look like?**

Before sending a create or capture request, persist an internal payment attempt with a unique identifier and idempotency key. Use the same key for retries of that same operation, following the provider's documented retention and retry rules. On timeout, do not initiate a fresh independent charge. Query the existing payment, receive provider webhook events with verified authenticity, deduplicate event handling and reconcile if the response remains uncertain. Model states such as `PENDING`, `SUCCEEDED` and `FAILED`, and define when the order may be fulfilled. Different gateways have different semantics, so I would follow the selected provider's API rather than assume all timeouts behave identically.

## Small example

EXAMPLE

```
Order 77 -> payment attempt pay-77, idempotency key pay-77
POST charge request -> timeout
Do NOT submit a new charge under key pay-77-retry-2
Check status of pay-77; retry safely under the original key if supported
Mark SUCCEEDED or FAILED only after authoritative confirmation
```

**Remember:** timeout ≠ payment failure.

## Additional details

**Added explanation — the original answer was blank.**

### What the timeout means (added material; the original answer was blank)

The client did not receive a definitive response within the limit. The provider **may still have completed the payment**. Do not treat timeout as failed, and do not start a fresh independent charge without checking.

### Safer workflow

1. Persist a local payment attempt and a stable idempotency key before calling the gateway.
2. On a timeout, mark the result **UNKNOWN/PENDING** for reconciliation.
3. Query the provider's authoritative status or consume verified, deduplicated webhooks.
4. If the provider supports safe retries, retry the same logical operation with the original idempotency key under its documented rules.
5. Fulfill the order only after a known final state.

EXAMPLE

```
Order 77 -> attempt pay-77 -> timeout
Do not issue an unrelated second charge
Reconcile existing attempt -> SUCCEEDED or FAILED
```

Specific idempotency-key retention, status codes and webhook semantics depend on the payment provider.

**Full original notes (preserved)**

### Source wording for question 14

**Official references:** Stripe idempotent requests — example of provider-specific retry semantics. Different payment providers may differ.

## 15. What did you personally implement in your strongest project?

### 30–60 second answer

"My strongest project was **[project name or business problem]**. Its goal was to **[specific outcome for users or the business]**. I personally owned **[module, service or feature]**, rather than the whole project. I designed and implemented **[two specific technical changes]**, using **[actual technologies]**. One important challenge was **[real constraint, such as latency, consistency or security]**. I addressed it by **[your decision and why]**, and validated the result using **[test, production metric or review method]**. The outcome was **[measured result, or a precise qualitative result if no metric exists]**. My main learning was **[practical trade-off or lesson]**."

### Follow-up

**How do I know that was your contribution and not your team's?**

Be concrete about the boundary of your work. Explain what you implemented yourself, what your teammates owned, and where you collaborated. Be ready to draw the request path, identify a failure mode, explain the alternative designs you rejected, and show how you tested the solution. Interviewers often look for one decision you can defend with a trade-off: for example, simplicity versus throughput or consistency versus latency. **Never invent percentage improvements or scale numbers**; if you did not measure them, describe the observable result and how you would measure it next time.

### Small example

EXAMPLE

```
Problem:       [Users experienced duplicate work on retries]
My ownership:  [Implement idempotency check in one service]
```

```
Decision:      [Store request ID and result in a durable table]
Trade-off:     [Extra database write for safer retries]
Validation:    [Concurrent duplicate-request integration test]
Outcome:       [Repeated request reused the original result]
```

**Remember:** use **I** for your actual work and **we** for team accomplishments.

### Additional details

**Template only — no project history was supplied.**

#### Make this answer about your actual contribution

The original file contains only this interview question; it provides **no project facts**. Keep the 30–60 second template above until you fill it with verified facts.

Prepare these exact details:

1. **Problem:** What user or business issue was your project solving?
2. **Ownership:** Which endpoint, component, workflow or feature did you personally implement?
3. **Design:** What two technical choices did you make and why?
4. **Challenge:** What broke or was risky, and how did you handle it?
5. **Validation:** What tests, logs or metrics proved it worked?
6. **Outcome:** What changed? Give a measured result only if it was actually measured.
7. **Trade-off:** What alternative did you reject, and what would you improve now?

Use **“I”** for your work and **“we”** for team results. Expect follow-up questions about failure modes, edge cases, test strategy and architecture.

**Full original notes (preserved)**

#### Source wording for question 15

## Practice checklist

1. Answer the question directly in the first sentence.
2. Explain the mechanism in plain words, not only keywords.
3. State the failure mode or trade-off.
4. Give one concrete example only when helpful.
5. Stop at 60 seconds; keep the rest for the follow-up.
6. For question 15, fill in all placeholders and practice with your real project details.

## Corrections made from the original draft

- Question 2: `final int count = 0; count++` was invalid Java; `final boolean shutdown = false` cannot later change. The revised examples use appropriate mutable or atomic state.
- Question 1: the original Java snippet had broken line breaks; it has been replaced with a smaller, valid record example.
- Questions 4, 8, 9, 12 and 14: expanded missing or incomplete explanations.
- Question 7: self-invocation is **one** common reason an annotation appears ineffective; a transaction can also be active but still commit because of rollback rules.
- Question 15: kept explicitly as a fill-in template rather than attributing invented experience to you.

# Interview Study Guide

7 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

3. Java: simple explanations and professional answers ..... 1
4. Concurrency, JVM and Vert.x ..... 9
5. Spring, APIs and tests ..... 12
6. Databases and caching ..... 14
7. Security and payment correctness ..... 16
8. Production skills and brief secondary review ..... 18
14. Final recall sheet ..... 19

## 3. Java: simple explanations and professional answers

### 1. Object-oriented design, interfaces and composition [A]

**Plain English:** An object keeps related data and behavior together. Encapsulation protects its rules. An interface describes what a component can do. Polymorphism lets different implementations satisfy that interface. Composition means using another component; inheritance means being a specialized version of it.

**Say it professionally:** "I use interfaces to define contracts and keep implementations replaceable. I prefer composition when I need flexible behavior, and inheritance when the subtype truly satisfies the parent contract. I keep business rules inside the component responsible for them."

**Example and follow-up:** A PaymentService depends on a PaymentGateway interface. Tests inject a fake gateway. Explain how Strategy selects a payment method and why a class with many unrelated responsibilities is hard to test. Know overloading versus overriding, interface versus abstract class, and checked versus unchecked exceptions.

### 2. SOLID and practical design patterns [A]

**Plain English:** SOLID encourages small responsibilities, replaceable implementations, and dependencies on useful contracts. It is a guide to controlling change, not a requirement to create many classes.

**Say it professionally:** "I apply design principles where they reduce coupling and make change safer. For example, a strategy separates payment-specific rules from orchestration, while an adapter hides the details of an external provider. I avoid adding abstractions without a clear need."

**Know:** Single responsibility; open for extension without repeated edits; substitutable subtypes; small client-focused interfaces; dependency inversion. Factory creates implementations; Builder constructs complex values; Decorator adds behavior. Dependency injection is how dependencies are supplied, not a substitute for good boundaries.

| SOLID principle | Plain English | In the gateway example below |
|---|---|---|
| **S** - Single Responsibility | Give a class one main reason to change. | `PaymentService` handles the business step; each gateway translates for one provider. |
| **O** - Open/Closed | Add behavior without repeatedly changing stable processing logic. | Add `BankBGateway`; keep `PaymentService.process` unchanged. |
| **L** - Liskov Substitution | A replacement must honor the parent type's contract. | Both gateways must return a receipt on success and report failure according to the same agreed rules. This applies to interface implementations as well as class inheritance. |
| **I** - Interface Segregation | Give callers only the operations they need. | The service needs a small `charge` contract, not every operation in a bank SDK. |
| **D** - Dependency Inversion | Business code depends on a contract it controls. | `PaymentService` depends on `PaymentGateway`; provider-specific code implements it. |

**Composition versus inheritance:** Composition means a class has another object. `PaymentService` has a `PaymentGateway`, which can be replaced. Inheritance means a class is a specialized kind of another class. A `WirePayment` might extend an abstract `Payment` only if it can fulfill every rule promised by `Payment`; if it must disable a parent operation, reconsider the hierarchy. Open/Closed is a design goal that either technique can support. Liskov applies to every subtype, including a class that implements an interface.

## Complete example: adding Bank B without changing PaymentService

Save the following as `GatewayDemo.java`. It is one runnable Java 8 class. Both bank clients simulate external APIs; there are no real charges or network calls. The different request and response shapes are intentional: each gateway hides its bank's details from the service.

JAVA

```java
import java.math.BigDecimal;
import java.math.RoundingMode;

public class GatewayDemo {
```

```
    record Payment(BigDecimal amount) {
        Payment(String amount) {
            this(new BigDecimal(amount)
                    .setScale(2, RoundingMode.UNNECESSARY));
            if (this.amount.signum() <= 0)
                throw new IllegalArgumentException("Amount must be positive");
        }
    }

    record Receipt(String reference) {
    }

    interface PaymentGateway {
        Receipt charge(Payment payment);
    }

    record BankAResponse(String transactionId) {
    }

    static class BankAClient {
        BankAResponse submitPayment(BigDecimal amount) {
            System.out.println("Bank A received $" + amount);
            return new BankAResponse("A-1001");
        }
    }

    static class BankBClient {
        String createCharge(long amountInCents) {
            System.out.println("Bank B received " + amountInCents + " cents");
            return "B-2001";
        }
    }

    static class BankAGateway implements PaymentGateway {
        private final BankAClient client;

        BankAGateway(BankAClient client) {
            this.client = client;
        }

        public Receipt charge(Payment payment) {
            BankAResponse response = client.submitPayment(payment.amount);
            return new Receipt(response.transactionId);
        }
    }

    // New provider adapter: Bank B's API uses cents and returns a String.
    static class BankBGateway implements PaymentGateway {
        private final BankBClient client;

        BankBGateway(BankBClient client) {
            this.client = client;
        }

        public Receipt charge(Payment payment) {
            long cents = payment.amount.movePointRight(2).longValueExact();
            String reference = client.createCharge(cents);
            return new Receipt(reference);
        }
    }
```

```
    static class PaymentService {
        private final PaymentGateway gateway;

        PaymentService(PaymentGateway gateway) {
            this.gateway = gateway;
        }

        Receipt process(Payment payment) {
            return gateway.charge(payment);
        }
    }

    static void main(String[] args) {
        PaymentService bankAService = new PaymentService(
                new BankAGateway(new BankAClient()));
        Receipt first = bankAService.process(new Payment("10.50"));
        System.out.println("Receipt: " + first.reference);

        PaymentService bankBService = new PaymentService(
                new BankBGateway(new BankBClient()));
        Receipt second = bankBService.process(new Payment("7.25"));
        System.out.println("Receipt: " + second.reference);
    }
}
```

Run `javac GatewayDemo.java` and then `java GatewayDemo`. You should see Bank A receive $10.50 and return `A-1001`, then Bank B receive 725 cents and return `B-2001`. These are two separate simulated payments. The clients (`BankAClient` and `BankBClient`) represent provider-specific SDK or HTTP code; the gateway adapters translate their results into our `Receipt`.

**What changed to add Bank B:** Add `BankBClient` and `BankBGateway`, then choose the new gateway when constructing the service. `PaymentService` still calls `gateway.charge(payment)`. In a Spring application, configuration or dependency injection would do the construction. The provider's contract must still define failure and retry behavior; this short example demonstrates structure, not production payment safety.

**Hexagonal architecture connection:** The application owns the `PaymentGateway` port. A bank gateway is an external adapter that implements it. The source-code dependency points toward the application contract, even though a payment call travels from the service outward to the bank. This is dependency inversion across architectural layers, not simply another name for composition.

**Professional answer to practice:** "I use composition to give the payment service a gateway through a small interface. Adding a bank means implementing another adapter and selecting it at application setup. The service stays focused on the payment flow. Each adapter must honor the gateway contract so it can safely replace another implementation."

### Strategy pattern: a runnable payment example

**Plain English:** The overall payment process is the same, but wire and check payments have different limits and fees. A strategy owns those changing rules. `PaymentService` selects a strategy and handles steps that are common to every payment. In this example it checks for a positive amount and calculates the total; saving and submitting a real payment are outside this small demonstration.

JAVA

```java
import java.math.BigDecimal;
import java.util.EnumMap;
import java.util.Map;

public class StrategyExample {
    enum PaymentType {WIRE, CHECK}

    interface PaymentStrategy {
        void validate(BigDecimal amount);

        BigDecimal calculateFee(BigDecimal amount);
    }

    static class WireStrategy implements PaymentStrategy {
        public void validate(BigDecimal amount) {
            if (amount.compareTo(new BigDecimal("10000")) > 0) {
                throw new IllegalArgumentException("Wire exceeds the limit");
            }
        }

        public BigDecimal calculateFee(BigDecimal amount) {
            return new BigDecimal("15.00");
        }
    }

    static class CheckStrategy implements PaymentStrategy {
        public void validate(BigDecimal amount) {
            if (amount.compareTo(new BigDecimal("5000")) > 0) {
                throw new IllegalArgumentException("Check exceeds the limit");
            }
        }

        public BigDecimal calculateFee(BigDecimal amount) {
            return new BigDecimal("2.00");
        }
    }

    static class PaymentService {
        private final Map<PaymentType, PaymentStrategy> strategies =
                new EnumMap<>(PaymentType.class);

        PaymentService() {
            strategies.put(PaymentType.WIRE, new WireStrategy());
            strategies.put(PaymentType.CHECK, new CheckStrategy());
        }

        BigDecimal process(PaymentType type, BigDecimal amount) {
            if (type == null || amount == null ||
                    amount.compareTo(BigDecimal.ZERO) <= 0) {
                throw new IllegalArgumentException("Invalid payment");
            }

            PaymentStrategy strategy = strategies.get(type);
            strategy.validate(amount);
            BigDecimal total = amount.add(strategy.calculateFee(amount));

            // Common steps in a real service: save, submit, record the outcome.
            return total;
```

```
        }
    }

    static void main(String[] args) {
        PaymentService service = new PaymentService();
        System.out.println(service.process(
                PaymentType.WIRE, new BigDecimal("100.00"))); // 115.00
        System.out.println(service.process(
                PaymentType.CHECK, new BigDecimal("100.00"))); // 102.00
    }
}
```

**How to explain it:** `PaymentStrategy` defines the operations that vary. `WireStrategy` and `CheckStrategy` implement their own limits and fees. `PaymentService` chooses the right implementation and keeps the shared flow in one place. To add a new payment type, add its strategy and register it. An adapter would be a separate component that translates our request into an external provider's format.

**Professional answer to practice:** "I use the Strategy pattern when several payment types share an overall process but have different business rules. The service selects a strategy, and that strategy handles type-specific validation and fee calculation. This keeps the common payment flow easier to read and test."

## Other design principles: KISS, YAGNI, DRY and more [A]

These are decision guides, not absolute laws. In an interview, give the meaning, a concrete example, and one tradeoff. You probably mean **YAGNI**, pronounced "yag-nee": **You Aren't Gonna Need It**.

| Principle | Plain English | Professional interview line |
|---|---|---|
| KISS - Keep It Simple | Choose the simplest design that meets the real requirements. Simple does not mean skipping correctness, security, or tests. | "I start with a clear implementation that meets today's requirements, then add complexity only when it solves a demonstrated problem." |
| YAGNI - You Aren't Gonna Need It | Do not build features or extension points for a future need that nobody has confirmed. | "I avoid speculative abstractions. I keep the code easy to change, and introduce a new layer when there is a concrete second use case or requirement." |
| DRY - Don't Repeat Yourself | Keep each business rule in one authoritative place so changes do not require inconsistent edits. Similar-looking code is not always the same rule. | "I remove duplicated knowledge, but I avoid forcing unrelated workflows into one abstraction just because they share a few lines." |
| Separation of concerns | Give different responsibilities clear homes: HTTP handling, business rules, persistence, and security. | "I separate concerns so each part has a clear reason to change and can be tested independently." |
| High cohesion, low coupling | Keep closely related behavior together and reduce unnecessary dependencies between components. | "I aim for focused components with small contracts, so a change in one area does not ripple across the service." |

| Principle | Plain English | Professional interview line |
|---|---|---|
| Composition over inheritance | Assemble behavior from components when a rigid parent-child hierarchy does not fit. | "I prefer composition for behavior that varies, such as payment methods, because I can replace or test each strategy independently." |
| Principle of least astonishment | Make an API behave as its name and contract suggest. Avoid hidden side effects. | "I design methods and endpoints so callers can predict their effects and error behavior from the contract." |
| Fail fast at boundaries | Reject invalid inputs early, with a clear error, before they can corrupt business state. | "I validate requests at the boundary and preserve deeper database constraints as a final defense for concurrent writes." |

**How they work together:** KISS asks whether the current design is understandable. YAGNI asks whether a proposed feature is actually needed. DRY asks whether one business fact is maintained in several places. They can pull in different directions: extracting every repeated three-line block may increase complexity and couple unrelated rules. First confirm that the duplicated code represents the **same knowledge**, then decide whether sharing it makes future changes safer.

**Payment example:** Suppose two endpoints have similar validation code. Share a stable rule such as "amount must be positive" when both endpoints truly mean the same thing. Keep different authorization and retry rules separate if one endpoint creates a payment and the other reverses one. Do not build a universal payment workflow engine only because more payment types might arrive later.

**A 30-second answer to memorize:** "I use KISS to keep a solution understandable, YAGNI to avoid building for unconfirmed future needs, and DRY to keep each business rule in one place. I apply them together rather than mechanically. In a payment API, I would share a genuinely common amount rule, but keep different payment and reversal workflows separate when their failure behavior differs."

## 3. Collections and complexity [A]

**Plain English:** A List keeps ordered items and allows duplicates. A Set enforces uniqueness. A Map connects a key to a value. Choosing the right collection often turns slow code into simple code.

**Say it professionally:** "I choose collections from the access pattern. ArrayList provides constant-time indexed access and amortized constant-time append. HashMap usually gives constant-time lookup with good hashing, while TreeMap provides sorted keys with logarithmic operations. I also consider ordering, memory, and concurrency."

**Know:** ArrayList insertion in the middle is O (n); LinkedList indexed lookup is O (n), so it is not automatically faster. HashMap has no guaranteed iteration order. LinkedHashMap preserves defined iteration order. PriorityQueue exposes the smallest element by default; offer/poll are O (log n), peek O (1). ArrayDeque is a useful stack or queue. HashSet operations depend on hashing.

## 4. equals, hashCode and mutable keys [A]

**Plain English:** equals asks whether two objects represent the same value. hashCode helps a hash collection find the area where that value belongs. Equal objects must have the same hash code. Different objects may share a hash code.

**Say it professionally:** "When I define value equality, I implement equals and hashCode consistently. I use stable fields for map keys because changing a key's equality or hash after insertion can make the entry unreachable through normal lookup. A hash collision does not imply equality."

**Follow-up:** `==` compares object references; equals can compare logical values. Override both methods together. In a payment deduplication key, include tenant and the real business identifier. A mutable object's current hash can point to a different bucket from the one used at insertion.

## 5. Immutability, String and pass-by-value [A]

**Plain English:** An immutable object's value cannot change after construction. This makes sharing easier. Java passes a copy of a value into a method; for objects, that value is a reference. The method can mutate the referenced object, but reassigning its local reference does not replace the caller's reference.

**Say it professionally:** "I favor immutable value objects because they reduce accidental changes and simplify concurrency. I use defensive copies for mutable members. A final reference cannot be reassigned, but it does not make the referenced object immutable."

**Know:** String is immutable; StringBuilder is mutable and useful for repeated construction. Records are concise data carriers but only shallowly immutable. An unmodifiable view can still reflect changes in its backing collection. Know primitives versus wrappers, autoboxing, and null-unboxing failures.

## 6. Exceptions and resource management [A]

**Plain English:** An exception reports a failure. Handle it where you can make a useful decision: retry, translate, compensate, or return an appropriate error. Database connections and streams must be closed even when something fails.

**Say it professionally:** "I catch exceptions at boundaries where I can add context or choose a recovery action. I preserve the cause, avoid silently swallowing failures, and use try-with-resources for AutoCloseable resources. I distinguish validation failures from temporary dependency failures."

**Follow-up:** Checked exceptions require handling or declaration; unchecked exceptions do not. finally normally runs, but is not guaranteed after process termination. Never use return in finally. Do not log sensitive payment data or repeatedly log the same exception in every layer. Explain how an exception affects transaction rollback.

## 7. Generics and type safety [A]

**Plain English:** Generics let the compiler catch type mistakes. `List<Payment>` promises payment values. Wildcards allow flexibility when a method reads or writes related types.

**Say it professionally:** "I use generics to make contracts explicit and avoid unsafe casts. For flexible APIs, I use extends when a collection produces values and super when it consumes values. I avoid raw types because they move errors from compile time to runtime."

**Know:** `List<Integer>` is not a subtype of `List<Number>`. `? extends Number` allows reading as Number but not safely adding arbitrary Number values. `? super Integer` permits adding Integer values. Type erasure limits runtime access to generic type arguments.

### 8. Lambdas, streams and Optional [A]

**Plain English:** A stream describes steps for processing data. Intermediate steps such as filter and map are lazy; a terminal operation runs the pipeline. Optional represents a possibly missing result.

**Say it professionally:** "I use streams for clear transformations and aggregation. map changes each element, while flatMap combines nested results into one stream. I avoid shared mutable state in stream operations and use a loop when it makes control flow or debugging clearer."

**Know:** filter, map, flatMap, distinct, sorted, collect, groupingBy, reduce; Comparator.comparing; functional interfaces Predicate, Function, Consumer and Supplier. Streams are generally single-use. `orElse` evaluates its argument eagerly; `orElseGet` uses a supplier when needed. Parallel streams add overhead and shared-pool concerns; they are not a default performance optimization.

### 9. Money and modern Java features [A/B]

**Plain English:** Binary floating-point cannot represent many decimal amounts exactly. Financial calculations need an explicit decimal representation and rounding rules. New language features can simplify code, but only if the project's Java version supports them.

**Say it professionally:** "For monetary values, I use BigDecimal or an agreed integer minor-unit representation, with explicit currency, scale, and rounding rules. I avoid constructing BigDecimal from a binary floating-point value. I choose language features based on the supported runtime."

**Know:** Prefer `new BigDecimal("0.10")`; division can need a rounding mode. `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is false, while compareTo returns zero. Know Java 8 lambdas/streams/Optional, Java 17-era records/sealed classes, and Java 21 virtual threads. Do not guess which Java release the team uses.

## 4. Concurrency, JVM and Vert.x

### 10. Race conditions, atomicity and visibility [A]

**Plain English:** A race occurs when the result depends on the timing of concurrent operations. Atomicity means an operation cannot be observed half-complete. Visibility means one thread can see another thread's change.

**Say it professionally:** "I separate visibility from atomicity. volatile provides visibility and ordering guarantees for a field, but it does not make a read-modify-write operation such as increment atomic. I use an atomic operation or a lock depending on the invariant I need to protect."

**Example:** Two threads can both read 10 and both write 11 for `count++`. AtomicInteger.incrementAndGet prevents that lost increment. A transfer involving two balances needs a shared consistency boundary, not two unrelated atomic counters. Explain happens-before as a guarantee that earlier writes become visible through a defined synchronization relationship.

## 11. synchronized, locks and deadlocks [A]

**Plain English:** A lock lets one thread at a time protect shared state. A deadlock occurs when threads wait for each other forever. Long work while holding a lock makes other threads wait.

**Say it professionally:** "I keep critical sections small and establish a consistent lock order when multiple locks are needed. synchronized is a good default for simple mutual exclusion. I consider ReentrantLock when I need features such as timed acquisition or interruptible waiting, and release it in finally."

**Follow-up:** Instance synchronization locks that instance; static synchronization locks the Class object. Locks are reentrant. A concurrent container does not make a multi-step business workflow atomic. Inspect thread dumps for blocking and lock cycles; do not add more threads as the first response.

## 12. Concurrent collections and executors [A]

**Plain English:** ConcurrentHashMap supports concurrent access. However, "check then insert" is two operations unless you use an atomic method. An executor manages task execution, and its limits protect the application from overload.

**Say it professionally:** "I use operations such as putIfAbsent or compute when a per-key update must be atomic. I also bound concurrent work and queue growth, because an unbounded backlog can cause latency and memory failures even when the individual data structures are thread-safe."

**Know:** Thread, Runnable, Callable, Future, ExecutorService, bounded queues, rejection policies, shutdown, interruption. ConcurrentHashMap rejects null keys/values. Avoid slow remote work inside compute. `Executors.newFixedThreadPool` uses an unbounded work queue; a fixed thread count alone does not bound pending work. See Java concurrency API.

## 13. CompletableFuture and asynchronous composition [A]

**Plain English:** A future is a result that will arrive later. Chaining futures lets you describe dependent work without waiting after every step. Independent work can run concurrently if resources allow.

**Say it professionally:** "I use thenApply to transform a result and thenCompose when the next step also returns a future. I use thenCombine for independent results. I make executor choice, deadlines, and exception handling explicit so asynchronous code remains predictable."

**Know:** join/get wait for completion; avoid them on event-loop threads. An async stage without an explicit executor commonly uses the common pool. Non-async stages may run on the thread completing the prior stage. A future timeout does not necessarily stop the underlying I/O; configure client timeouts and cancellation where supported. Distinguish exceptionally, handle and whenComplete.

## 14. JVM memory, garbage collection and performance [B]

**Plain English:** The JVM manages object memory and reclaims unreachable objects. A memory leak still happens when the program keeps references to objects it no longer needs. High latency may come from CPU, waiting, memory pressure, or a dependency.

**Say it professionally:** "I start performance investigation with measurements. I compare latency, throughput, CPU, allocation, garbage collection, thread state, and dependency timing. I use thread dumps or profiling for execution problems and heap analysis for retained-memory problems before changing JVM settings."

**Know:** Heap, thread stacks, class metadata/metaspace, stack overflow versus out-of-memory, JIT compilation, allocation rate, retained size. Objects are generally heap-managed, with optimizations such as escape analysis. System.gc is not a reliable fix. An ever-growing map or retained listener can cause a leak. Increasing heap may postpone a failure without fixing the cause.

## 15. Virtual threads [B]

**Plain English:** Virtual threads make it cheaper to have many tasks waiting for I/O. They do not create extra CPU cores or make database connections unlimited.

**Say it professionally:** "Virtual threads can simplify blocking-style code for high-concurrency I/O workloads. I still limit access to scarce dependencies such as database connections. They improve scalability for suitable workloads, not the speed of CPU-intensive computation."

**Version detail:** Virtual threads became final in Java 21. Pinning behavior changed in newer JDKs; do not repeat Java 21 synchronization limitations as a universal rule for every version. Know the runtime before discussing those details. See Oracle's virtual-thread guide.

## 16. Vert.x event loops and worker execution [A]

**Plain English:** An event loop handles many small pieces of work. If one handler waits on a blocking call, other work assigned to that loop must wait too. Nonblocking I/O lets the loop do other work while waiting for a result.

**Say it professionally:** "Vert.x uses event loops to handle concurrent I/O efficiently. I keep event-loop handlers short and nonblocking. For unavoidable blocking work, I use worker execution or executeBlocking and control the workload so the worker pool does not become the next bottleneck."

**Follow-up:** Blocking JDBC, Thread.sleep, waiting on a future, and long CPU work can stall an event loop. Prefer an appropriate asynchronous client when available. A standard verticle's context simplifies concurrency, but shared state across contexts still needs care. Know verticles, handlers, Future/Promise, map/compose, routing and error handling. See Vert.x core guide.

## 17. Backpressure and event delivery [A/B]

**Plain English:** If data arrives faster than you can process it, a queue grows. Backpressure slows the producer or limits how much work you accept. Simply making code asynchronous does not solve overload.

**Say it professionally:** "I treat overload as an explicit behavior. I bound in-flight work, monitor queue depth, and pause, reject, or shed load when needed. I also distinguish in-process messaging from durable delivery requirements."

**Follow-up:** A Vert.x event bus is useful for communication, but should not be assumed to provide durable payment-event storage. For durable workflows, explain persistence, acknowledgement, retry and duplicate handling. For streams, understand pause/resume and drain handling. See Vert.x context guide.

# 5. Spring, APIs and tests

## 18. Spring dependency injection and bean scope [A]

**Plain English:** Spring creates and connects application components. Instead of creating a database client inside a service, the service receives it. This makes dependencies visible and tests easier.

**Say it professionally:** "I prefer constructor injection for required dependencies because it makes the contract explicit and supports straightforward unit testing. I keep singleton services stateless, since a singleton bean is shared and Spring does not automatically make its fields thread-safe."

**Know:** Component scanning; @Component, @Service, @Repository; @Configuration and @Bean; @Qualifier and @Primary; singleton versus prototype/request scope; bean initialization and destruction; circular dependencies. A Spring singleton is normally per application context and bean definition, not one global instance across every process.

## 19. Spring Boot and configuration [A]

**Plain English:** Spring provides the framework. Spring Boot helps assemble and configure an application with useful defaults. Those defaults depend on the libraries and configuration you provide.

**Say it professionally:** "Spring Boot reduces setup through starters and conditional auto-configuration. I still inspect the effective configuration, especially database pools, security, timeouts, and production endpoints. I externalize environment-specific values and keep secrets out of source control."

**Know:** @SpringBootApplication, profiles, configuration properties, validation, embedded server, health endpoints and graceful shutdown. Auto-configuration can back off when you supply your own beans; use the conditions report to understand why. See Spring Boot auto-configuration.

## 20. Request flow, validation and error handling [A]

**Plain English:** A request enters through security and routing, is validated, runs business logic, accesses data, and becomes a response. Each layer should have a clear job.

**Say it professionally:** "I keep controllers focused on HTTP concerns, services on business rules, and repositories on persistence. I validate request structure early, enforce authorization and business invariants in the appropriate layer, and translate failures into consistent responses with a correlation identifier."

**Know:** @RestController, mappings, DTOs, @Valid, @ControllerAdvice, filters/interceptors. Do not expose persistence entities by default or allow clients to set privileged fields. Syntactic validation is not authorization. Use parameterized queries, bounded pagination, stable error codes, and intentional logging.

## 21. Transactions and the proxy trap [A]

**Plain English:** A transaction groups database work so it succeeds or fails together. Spring commonly applies transaction behavior through a wrapper called a proxy. Calling a method directly within the same object bypasses that wrapper.

**Say it professionally:** "I place transaction boundaries around cohesive business operations and keep them short. In Spring's default proxy mode, self-invocation bypasses transactional advice. I also check rollback rules explicitly: the usual default rolls back for runtime exceptions and errors, not every checked exception."

**Follow-up:** REQUIRED joins an existing transaction or starts one. REQUIRES_NEW uses an independent transaction and can need another connection. Catching and swallowing an exception can change rollback behavior. Thread-bound transactions do not automatically follow newly created threads. Configuration can override defaults. See Spring transaction annotations and transaction implementation.

## 22. Persistence, JPA and the N+1 problem [A/B]

**Plain English:** An object mapper can hide SQL details, but the database still executes queries. N+1 means one query loads a group of rows and additional queries load related data one item at a time.

**Say it professionally:** "I inspect the SQL generated by the persistence layer. For N+1 behavior, I choose a fetch join, entity graph, batch fetch, or projection based on the use case. I verify query count and pagination behavior instead of changing every relationship to eager loading."

**Know:** Entity lifecycle, persistence context, dirty checking, lazy loading, flush versus commit, optimistic @Version locking. Flush sends changes toward the database; it is not the same as committing the transaction. Spring Data is not synonymous with JPA: know whether your project uses JPA, JDBC, or another repository technology.

## 23. HTTP and REST contracts [A]

**Plain English:** HTTP methods describe the intended operation. A good API tells clients what happened and what to do next. Idempotency means repeating an operation has the same intended effect as doing it once.

**Say it professionally:** "I define API behavior around resources, validation, authorization, and stable contracts. GET should be safe; PUT and DELETE are idempotent in their intended effect. POST can be made retry-safe with an explicit idempotency contract. I do not equate identical responses with idempotency."

**Know:** GET/POST/PUT/PATCH/DELETE; PUT replacement versus PATCH partial update; PATCH is not inherently idempotent. 200 success, 201 creation, 202 accepted for later processing, 204 no body, 400 invalid request, 401 missing/invalid authentication, 403 forbidden, 404 not found, 409 conflict, 429 rate limit, 500 internal error, 503 unavailable. Explain pagination, backward-compatible changes, versioning, TLS and connection reuse.

### 24. Testing and code quality [A]

**Plain English:** Different tests find different mistakes. Unit tests check local logic. Integration tests check real interactions. Contract tests check agreements between services. A small set of end-to-end tests checks complete flows.

**Say it professionally:** "I choose tests around failure risk. I unit-test business rules, use integration tests for database constraints and transaction behavior, and verify API contracts. For payment operations, I explicitly test duplicate requests, concurrent updates, timeout ambiguity, and authorization boundaries."

**Know:** JUnit assertions and parameterized tests; Mockito stubbing and verification; test doubles versus real dependencies. Do not mock away the behavior you need to verify. Prefer deterministic concurrency tests with coordination primitives over arbitrary sleeps. High coverage alone does not prove useful assertions. Static analysis complements review and testing.

## 6. Databases and caching

### 25. SQL fundamentals and data modeling [A]

**Plain English:** Tables hold related facts. Keys identify rows and relationships. SQL combines, filters, groups, and orders those facts. Constraints protect rules even when multiple application instances write at once.

**Say it professionally:** "I use primary, foreign, and unique constraints to protect data integrity, and I choose joins and aggregates from the business question. I pay attention to NULL semantics and row multiplication in joins because a query can be syntactically correct but financially wrong."

**Know:** INNER/LEFT JOIN, WHERE/HAVING, GROUP BY, COUNT (*) versus COUNT (column), SUM, subqueries, CTEs, ROW_NUMBER and DENSE_RANK. Use IS NULL. A filter on the right table in WHERE can unintentionally remove unmatched LEFT JOIN rows. Joining payments to multiple allocation rows can double-count payment totals.

### 26. Indexes and slow-query diagnosis [A]

**Plain English:** An index is an extra data structure that helps find rows. It costs storage and write work. An index helps only if the query and data distribution make it useful.

**Say it professionally:** "I start with the execution plan, actual row counts, and query pattern. I consider composite-index order, selectivity, sorting, and coverage. I then measure the read improvement against the extra cost on inserts and updates."

**Example:** For tenant-scoped queries filtered by status and ordered by time, consider `(tenant_id, status, created_at, id)` and verify the actual plan. Avoid claiming it is automatically optimal. Functions or implicit casts on indexed columns can prevent useful access paths. Check statistics, locks, connection-pool waiting, and query count before blaming the database engine.

## 27. ACID, isolation and lost updates [A]

**Plain English:** Atomicity is all-or-nothing. Consistency means rules remain valid. Isolation controls interference between concurrent transactions. Durability means committed work survives under the database's configured guarantees.

**Say it professionally:** "I choose isolation and locking based on the business invariant. I use unique constraints and atomic updates where possible. For concurrent edits, optimistic locking detects a stale version; pessimistic locking prevents conflicting access by making transactions wait. I plan for deadlock or serialization retries."

**Know:** Dirty read = uncommitted data; nonrepeatable read = the same row changes between reads; phantom = matching row set changes. Read committed prevents dirty reads; stronger levels constrain more anomalies, with engine-specific behavior. Serializable targets the effect of serial execution but may abort transactions. Verify DB2, Oracle or MySQL semantics rather than assuming they match another engine. Isolation reference and examples.

## 28. Redis and cache correctness [A/B]

**Plain English:** A cache keeps a copy of data for faster reads. The database remains the authority in a cache-aside design. Cached data can become old, and many simultaneous misses can overload the database.

**Say it professionally:** "I use cache-aside for suitable repeated reads, with tenant-aware keys, expiration, and an invalidation strategy. I measure hit rate and tail latency, and plan for cache failure and stampedes. I do not treat cached permissions or payment state as automatically authoritative."

**Know:** Read cache → on miss query database → populate cache. After a committed write, invalidate or update according to the consistency design. TTL limits an entry's lifetime but does not eliminate stale-write races. Use versioning, coordinated refresh, or stronger reads when required. TTL jitter and request coalescing can reduce stampedes. Eviction differs from expiry. A cache outage needs bounded fallback, not unlimited database traffic. Redis cache-aside guide.

## 29. Couchbase and SQL versus document storage [B]

**Plain English:** A document database stores related information in documents. It can suit flexible records and key-based access. You still need deliberate schemas, indexes and concurrency control.

**Say it professionally:** "I choose storage based on access patterns and consistency requirements. In Couchbase, I distinguish key-value operations from indexed SQL++ queries. I use CAS for optimistic concurrency and select query consistency deliberately when recent writes must be visible."

**Know:** CAS changes when a document changes; a mismatched CAS rejects a stale mutation. Secondary indexes may lag writes. request_plus waits for indexing through the request's consistency point; it does not promise inclusion of every later concurrent write. Do not say all NoSQL databases lack transactions. See Couchbase CAS and SQL++ consistency.

# 7. Security and payment correctness

## 30. Authentication, authorization, IAM and MFA [A]

**Plain English:** Authentication establishes who you are. Authorization decides what you may do. MFA strengthens authentication with independent factor types. A valid login does not grant access to every account.

**Say it professionally:** "I separate identity verification from access decisions. I validate the principal, then enforce tenant, resource, and action-level authorization on every sensitive operation. RBAC provides role-based permissions, while contextual policies can handle additional restrictions. I follow least privilege and audit important access changes."

**Follow-up:** A password and another password are not two different factor types. Explain enrollment, recovery, lockout, session expiry, deprovisioning and permission revocation. For `/accounts/{id}`, never rely only on possession of the ID. Test cross-tenant access and privilege escalation. OWASP object-level authorization.

## 31. OAuth, OpenID Connect and JWT [A/B]

**Plain English:** OAuth delegates access. OpenID Connect adds an identity layer. JWT is a token format, not a login protocol. A signed token can be readable even though changes to it can be detected.

**Say it professionally:** "I distinguish the authorization protocol from the token format. For a JWT, I verify the signature with trusted keys, restrict accepted algorithms, and check issuer, audience, time claims, and token purpose. I do not assume a valid signature alone is enough."

**Know:** Access token versus ID token, scopes versus application permissions, expiry, key rotation, refresh-token handling. Do not accept an ID token as an API access token without an explicit appropriate contract. Authorization code with PKCE is relevant for user-facing clients; client credentials is for suitable service-to-service access. Explain revocation needs. JWT security guidance.

## 32. API security beyond login [A]

**Plain English:** An authenticated client can still send malicious input, request another customer's record, or overwhelm a service. Security checks must match each operation.

**Say it professionally:** "I validate inputs, parameterize database access, enforce authorization on server-side resources and properties, and bound request size and rate. I protect secrets, minimize sensitive logging, and test negative cases alongside the successful flow."

**Know:** Injection, broken object authorization, mass assignment, server-side request forgery, dependency vulnerabilities and sensitive error messages. Allowlist writable fields. Use destination restrictions for server-side fetches. SAST inspects code; DAST exercises a running application; dependency scanning identifies known component issues. None proves the application is secure by itself.

## 33. Message signing and certificate renewal [A/B]

**Plain English:** Encryption protects confidentiality. A digital signature helps verify the signer and detect modification. Certificate renewal replaces expiring identity material; it must also reach every system that depends on it.

**Say it professionally:** "For certificate automation, I treat renewal as a lifecycle: inventory, expiry detection, issuance, secure deployment, trust propagation, verification, and alerting. Where supported, I allow a controlled overlap between old and new certificates and retain a rollback path. I verify use of the new certificate before retiring the old one."

**Follow-up:** Distinguish TLS certificates from the message-signing certificates named in your resume. Explain keystore versus truststore, public/private keys, certificate chain, renewal versus key rotation, and failure recovery. State the tooling you actually used. Avoid exposing private keys in logs or repositories. Your resume does not establish which certificate authority, scheduler or secret store you used.

## 34. Idempotency and duplicate payment requests [A]

**Plain English:** A client may retry because the response was lost even though the payment was accepted. The service must recognize that retry instead of creating another payment.

**Say it professionally:** "I make retry safety an explicit contract. I scope an idempotency key to the caller or tenant, bind it to the request, and enforce uniqueness in durable storage. Repeated matching requests return the existing outcome, while conflicting payloads are rejected. I also handle concurrent duplicates and in-progress work."

**Follow-up:** Persist key, payload fingerprint, status and result/reference with a defined retention policy. A local map fails across instances and restarts. A Redis-only key with expiry does not necessarily guarantee durable financial deduplication. A database record alone does not deduplicate an external charge: use provider idempotency and reconciliation when the provider outcome is uncertain.

## 35. Timeouts, retries, messaging and partial failure [A/B]

**Plain English:** A timeout means you did not receive a result in time. It does not prove that the other system did nothing. Retrying can help temporary failures but can also repeat side effects or worsen overload.

**Say it professionally:** "I set deadlines and retry only failures that are transient and safe to repeat. I use bounded attempts with backoff and jitter. For an uncertain payment outcome, I reconcile using a stable reference rather than blindly submitting a new charge."

**Know:** Circuit breaker avoids repeated calls to a failing dependency; bulkhead limits the resources one dependency can consume. At-least-once delivery requires duplicate-safe consumers. An outbox stores a business change and event record in one local transaction; a relay publishes later and may publish duplicates. It does not make an external payment atomic. Transactional outbox reference.

# 8. Production skills and brief secondary review

## 36. Logs, metrics, traces and incident response [A/B]

**Plain English:** Logs describe events, metrics show trends, and traces connect steps in a request. A slow request can spend most of its time waiting in a pool or dependency rather than executing your code.

**Say it professionally:** "I first assess customer impact and recent changes, then use latency, errors and saturation to narrow the problem. Distributed traces show where time is spent, while correlated logs provide context. I mitigate safely, verify recovery, and follow with root-cause analysis and prevention."

**Know:** p50/p95/p99, throughput, error rate, queue depth, database pool usage, event-loop delay, GC pauses. Average latency can hide a bad tail. Propagate trace context across service boundaries. Use Jaeger and Splunk examples only to the depth you actually know. Distinguish trace IDs from business IDs; avoid secrets and high-cardinality metric labels.

## 37. CI/CD, blue-green and canary releases [B]

**Plain English:** A delivery pipeline makes changes repeatable. Blue-green switches traffic between two environments. Canary sends a limited amount of traffic to a new version while you watch the result.

**Say it professionally:** "I combine automated checks with a deployment strategy that limits exposure. I define health and business signals for promotion or rollback, and keep database and API changes compatible across versions. A rollback plan must account for data changes, not just application binaries."

**Know:** Build/test/scan/package/deploy/verify; immutable artifacts; Git review and revert; feature flags; expand-and-contract schema changes. A green health endpoint alone does not prove business correctness. Review error rate, tail latency, duplicate transactions and authorization failures where relevant.

## 38. Reliability, DR and Kubernetes basics [B/C]

**Plain English:** High availability keeps a service running through failures. Disaster recovery restores it after a larger disruption. Recovery time and acceptable data loss are separate targets.

**Say it professionally:** "I define recovery objectives and test the recovery process, including data integrity and dependency readiness. For containers, I distinguish readiness to receive traffic from liveness that determines whether a restart is needed. I avoid restart behavior that amplifies a shared dependency outage."

**Know:** RTO = target recovery time; RPO = target data-loss window; SLI = measured indicator; SLO = target; SLA = service agreement. Startup probes protect slow startup, readiness removes unready pods from service routing, and liveness can trigger restart. Know Pod, Deployment, Service, requests/limits, autoscaling and graceful shutdown. Kubernetes probe guide.

## 39. AWS, Node.js and integration technologies [C]

**Plain English:** Learn what the named technology does and how it relates to your work. Do not attempt a cloud-certification syllabus before this coding interview.

**Say it professionally:** "My strongest hands-on area is [your actual area]. For [secondary technology], my experience is [implemented / reviewed / conceptual]. I can explain its role and tradeoffs, and I would confirm service-specific configuration before making a production decision."

**Quick review:** AWS identity roles, network boundaries, load balancing, compute, object storage, managed databases and monitoring. Node.js uses an event loop for JavaScript callbacks; asynchronous APIs and worker facilities handle other work, but long CPU callbacks can block progress. SOAP uses XML messaging/contracts; REST is an architectural style, commonly implemented with HTTP. Spring Integration provides message-oriented components such as channels, routers, transformers and adapters. For EDI/ETL, review validation, mapping, acknowledgements, control totals, duplicates, checkpoints and safe reprocessing from your actual projects.

### 40. Airflow and Spark [C]

**Plain English:** Airflow schedules and coordinates tasks. Spark processes data across workers. The scheduler and the data-processing engine solve different problems.

**Say it professionally:** "I distinguish workflow orchestration from distributed processing. I design retried pipeline tasks to be idempotent and tied to a specific input partition. In Spark, I pay attention to shuffles and skew because data movement and uneven partitions can dominate runtime."

**Know:** Airflow DAGs, dependencies, retries and backfills; pass references to large data rather than large payloads between tasks. Spark transformations are lazy, actions trigger computation, and shuffles redistribute data. Review only if asked or if you choose these technologies as experience examples. See Airflow practices and Spark programming guide.

## 14. Final recall sheet

Review this during the last 30 minutes of study.

- **Coding:** clarify → example → approach → invariant → code → edge tests → complexity.
- **Design:** KISS = understandable solution; YAGNI = no speculative features; DRY = one source for each business rule.
- **Hash collections:** equals/hashCode contract; stable keys; expected lookup cost; no guaranteed HashMap order.
- **Concurrency:** visibility is not atomicity; compound operations need a consistency boundary; bound queues and dependencies.
- **Java:** immutable versus final; pass-by-value; BigDecimal; comparator correctness; exception propagation.
- **Async:** compose dependent work; do not block event loops; timeout does not necessarily cancel work.
- **Spring:** constructor injection; singleton services should avoid mutable request state; proxy boundaries and rollback rules.
- **Database:** constraints protect invariants; inspect query plans; handle lost updates; keep transactions short.
- **SQL:** tenant scope; currency grouping; NULL semantics; LEFT JOIN filters; deterministic ranking.

- **Cache:** freshness policy; invalidation; stampedes; bounded fallback; cache is not automatically authoritative.
- **Security:** authenticate, then authorize the tenant, resource and action; validate token purpose and claims.
- **Payments:** durable idempotency; safe retries; uncertain outcomes need reconciliation.
- **Production:** customer impact → evidence → mitigation → verify → root cause → prevention.
- **Stories:** my contribution, actual mechanism, tradeoff, failure behavior, verified metric.
- **Communication:** answer first, explain simply, give an example, state a limitation, stop.

Prepare two questions for the interviewer: "What are the main backend engineering challenges for this team?" and "How does the team validate reliability and correctness before releasing changes?"

Check the meeting time and time zone, coding environment, audio and connectivity. Keep a blank sheet for examples. Do not start a new topic immediately before the interview.

# Position yourself as an engineer who has adapted to AI-assisted development

3 MAIN SECTIONS • PRINT-READY EDITION

## 1. Main statement — Professional and confident

I want to be transparent about the coding exercise. I haven't spent much time preparing for traditional live coding interviews.

Over the last couple of years, my development workflow has shifted significantly toward AI-assisted engineering.

As a senior engineer, I spend more time understanding requirements, breaking down complex problems, designing systems, making architectural decisions, identifying edge cases, reviewing AI-generated code, and ensuring production reliability.

I still understand the code I deliver and take full responsibility for its correctness. However, I no longer spend most of my day writing implementations from scratch without tools.

I'm comfortable explaining my technical approach, discussing trade-offs, and walking through how I would solve a problem. But I recognize that I may be less practiced at writing code from memory under timed interview conditions.

**That doesn't mean I've moved away from hands-on engineering. It means the tools and the way I build software have evolved.**

## 2. If challenged — "A senior engineer should still be able to code without AI."

I agree. Coding fundamentals remain essential.

A senior engineer must be able to reason about code, debug issues, understand performance trade-offs, identify incorrect implementations, and take ownership of the final solution.

At the same time, I believe engineering effectiveness should be measured by more than the ability to write code from memory under time pressure.

What matters most is the ability to solve complex problems, make sound technical decisions, build reliable systems, and use modern tools effectively.

**My day-to-day work is focused on delivering engineering outcomes, not optimizing for how quickly I can reproduce an algorithm from memory.**

## 3. My recommendation

I'm not questioning the importance of coding fundamentals. I'm highlighting the difference between demonstrating engineering competence and demonstrating speed in a tool-free coding exercise.

# Interview Preparation: Four AI and MCP Projects

7 MAIN SECTIONS • PRINT-READY EDITION

## In this guide

1. Travel Planner MCP ..... 1
2. CodeGuard Code Review Assistant ..... 4
3. MCP Studio ..... 7
4. ADK Calendar Assistant ..... 9
5. Interview claims that need careful wording ..... 11
6. Five deeper architecture questions to prepare ..... 12
7. Recommended positioning in the interview ..... 12

I reviewed the source code in all four project ZIP files. The points below are grounded in the implementations, not just the README files.

For each project, this guide includes a **60-second explanation, technical points you can defend, likely interview questions, code references, and limitations**.

The focus is on solution architecture, engineering decisions, AI governance, security, and system integration.

## 1. Travel Planner MCP

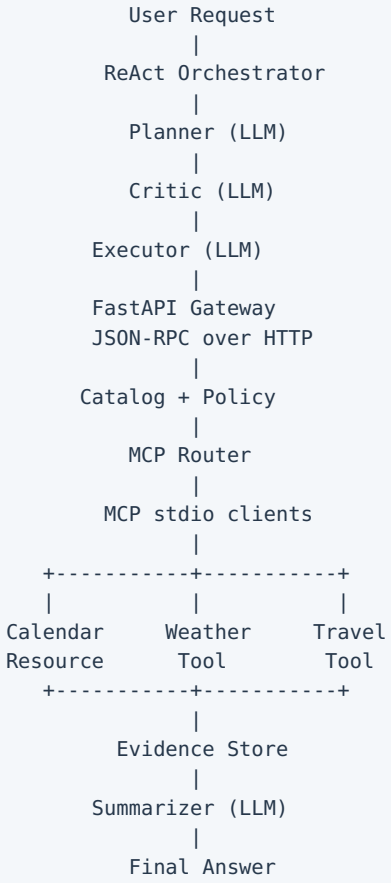
**Strongest project for AI agent architecture**

## 60-second interview explanation

> I built a proof-of-concept travel planning agent to explore how an LLM can coordinate multiple services without having direct access to their implementations.
>
> I separated the workflow into four logical roles: Planner, Critic, Executor, and Summarizer. The Planner selects the next action, the Critic reviews it, the Executor prepares tool arguments, and the Summarizer produces an evidence-based response.
>
> The orchestrator communicates with a FastAPI JSON-RPC gateway, which authorizes and routes requests to separate calendar, weather, and travel MCP servers.
>
> I added Pydantic response contracts, tool authorization, bounded execution, timeouts, evidence tracking, and correlated logs.
>
> My main architectural goal was to separate probabilistic AI reasoning from deterministic service execution and authorization.

## Architecture to explain

EXAMPLE



## Technical points you can justify

| Talking point | Code evidence |
| --- | --- |
| Explicit ReAct loop and bounded execution | `src/orchestrator/react_loop.py` — `run_react_loop()`, `max_steps=8` |
| Four role-based LLM interactions | `src/orchestrator/reasoning_adapter.py` — `plan_next_step()`, `review_step()`, `build_tool_args()`, `summarize()` |
| Structured AI outputs | `src/orchestrator/schemas.py` — `NextStep`, `CriticDecision`, `ToolArgs`, `FinalAnswer` |
| Catalog authorization | `src/gateway/catalog.py` and `src/gateway/policy.py` |
| MCP service isolation | `src/gateway/mcp_router.py` — `ClientSession`, `stdio_client` |
| Timeouts | `src/gateway/mcp_router.py` — `asyncio.wait_for()` |
| Correlated logging | `src/shared/logging.py`, `src/orchestrator/react_loop.py` |
| Evidence tracking | `src/orchestrator/memory.py` — `EvidenceStore` |

## Likely interview questions

**Q: Why did you create four roles instead of using a single AI agent?**

A: I wanted separation of responsibilities. Planning, reviewing, preparing tool arguments, and summarizing have different requirements. Separate prompts and output contracts make the process easier to inspect and test. However, these are four logical roles using the same reasoning adapter, not four independently deployed agents.

**Q: Why introduce a gateway between the LLM and MCP servers?**

A: The gateway creates a controlled boundary. It accepts JSON-RPC requests, validates parameters, checks a catalog, identifies the domain, and delegates execution through MCP sessions. The LLM does not directly choose arbitrary server processes.

**Q: How did you prevent hallucinated tool calls?**

A: I provided the allowed tool catalog to the planner and constrained the expected output using Pydantic models. More importantly, the gateway independently rejects unknown tool names. Model output validation and authorization are separate controls.

**Q: How does ReAct work in this project?**

A: The loop follows Reason → Act → Observe. The planner selects an action, the system executes it through the gateway, and the result is stored as evidence for the next step. The implementation currently uses a prescribed calendar → weather → travel → finish sequence rather than completely open-ended planning.

### Important limitations to acknowledge

- Calendar, weather, and flight responses are deterministic simulated data, not live bookings or forecasts.
- The critic is LLM-based. Gateway authorization is the deterministic control.
- The policy checks a static catalog but does not enforce tenant-specific permissions.
- Evidence is stored in memory for the running workflow.
- The gateway exposes application-specific JSON-RPC methods such as `tools.call`. The router uses the MCP SDK internally.

**Best architectural takeaway:** An LLM should propose actions, but application-controlled services should validate and execute them.

---

## 2. CodeGuard Code Review Assistant

**Strongest project for AI-assisted engineering governance**

### 60-second interview explanation

> I developed a Python proof-of-concept for AI-assisted code review that combines deterministic metadata extraction with LLM-based risk interpretation.
>
> The system compares Git revisions, parses Python code using Tree-sitter, detects possible hardcoded secrets using regular expressions, and checks Python dependencies against the OSV vulnerability database.
>
> Instead of sending raw source code to the LLM, it generates structured metadata containing complexity metrics, risk indicators, secret findings, and vulnerability information.
>
> The triage layer can use either OpenAI or Claude to generate an assessment. The output is checked against a JSON Schema, with retries for invalid responses, before producing JSON and Markdown reports.
>
> The key design decision was to make the LLM interpret collected evidence rather than rely on it to discover every issue from raw code.

### Architecture

EXAMPLE




## Technical points you can justify

| Talking point | Code evidence |
| --- | --- |
| Three-stage review pipeline | `src/run.py` — `run_pipeline()` |
| Git revision comparison | `extract_structure.py` and `extract_secrets.py` — `get_diff_files()` |
| Tree-sitter analysis | `src/extractors/extract_structure.py` |
| Cyclomatic complexity and nesting depth | `calculate_cyclomatic_complexity()`, `calculate_nesting_depth()` |
| Redacted secret findings | `src/extractors/extract_secrets.py` — `scan_file_for_secrets()` |
| Dependency vulnerability lookup | `src/extractors/extract_dependencies.py` — `query_osv_api()` |
| Alternative LLM providers | `src/llm/triage_agent.py`, `triage_agent_openai.py` |

| Talking point | Code evidence |
| --- | --- |
| Validation and retry | `src/validators/validate_schema.py`, `assess_risk()` |
| Evidence-based reports | `src/reporters/generate_report.py` |

## Likely interview questions

**Q: Why use Tree-sitter instead of asking an LLM to review all the code?**

A: Tree-sitter gives reproducible structural information such as function boundaries, parameters, complexity, and nesting depth. That reduces dependence on the LLM for facts that can be extracted programmatically.

**Q: How did you reduce the exposure of sensitive source code?**

A: The pipeline passes derived metadata to the LLM. The secrets extractor records finding IDs, file locations, secret categories, entropy, and confidence rather than the matched secret values. It is a metadata-minimization design, although file paths and other metadata can still be sensitive.

**Q: Why use JSON Schema validation and retries?**

A: LLM responses are not always valid application data. The assessment is parsed and validated against a defined schema. If parsing or validation fails, the code retries with error feedback. This checks structural correctness; it does not independently prove every risk claim is factual.

**Q: Does the system use OpenAI and Claude together?**

A: No. The current CLI selects one provider for a run. The two adapters provide alternative implementations of the same triage stage. There is no cross-model voting or consensus engine.

**Q: How are security findings translated into merge recommendations?**

A: The LLM generates risk levels, findings, recommendations, and an `allow_merge` result using the extracted metadata and prompt rules. This is advisory output, not a separately enforced CI/CD merge gate.

## Important limitations

The implementation is a Python-focused POC. Git diff analysis identifies changed files, but does not precisely analyze only changed lines. Tree-sitter scans Python files in the working tree. The dependency scanner focuses on `requirements.txt` and direct dependencies.

I also identified two code issues that should be corrected before a live demonstration:

- `src/llm/triage_agent_openai.py`, line 49, prints the raw OpenAI API key. This is a security issue. Remove that statement and rotate any key that may have reached logs.
- The Claude adapter uses a schema path that is inconsistent with `SchemaValidator`'s schema directory. This can cause Claude validation to fail.

**Best architectural takeaway:** Use deterministic tools to collect evidence and AI to help interpret it, with an explicit validation and human-review boundary.

# 3. MCP Studio

## 60-second interview explanation

> I built MCP Studio to understand the MCP client-server lifecycle and the behavior of Streamable HTTP.
>
> The first part demonstrates an MCP client establishing a session, initializing the protocol, discovering tools, and invoking tools against a FastAPI-hosted MCP server.
>
> I also implemented ASGI middleware to inspect HTTP headers and request payloads, making protocol behavior easier to troubleshoot.
>
> The second part explores streaming. I implemented shared asynchronous progress-generation logic and exposed it through two mechanisms: MCP progress notifications and a standard REST Server-Sent Events endpoint.
>
> Finally, I containerized the services with Docker Compose and configured Nginx to disable response buffering so progress updates can reach clients incrementally.

## Technical points you can justify

| Talking point | Code evidence |
| --- | --- |
| Stateful MCP HTTP server | `streamable_basics/src/mcp_studio/foundation/server.py` |
| Session initialization and tool discovery | `streamable_basics/src/mcp_studio/foundation/client.py` |
| Request inspection | `streamable_basics/src/mcp_studio/debug/request_inspector.py` |
| Shared async progress generator | `streaming_response/server/src/services/number_stream.py` |
| MCP progress notifications | `streaming_response/server/src/tools/stream_tool.py` |
| REST SSE endpoint | `streaming_response/server/src/main.py` |
| Docker service networking | `streaming_response/docker-compose.yaml` |

| Talking point | Code evidence |
| --- | --- |
| Streaming proxy configuration | `streaming_response/nginx/nginx.conf` |

## Likely interview questions

**Q: What happens during MCP initialization?**

A: The client opens the Streamable HTTP transport, creates a `ClientSession`, calls `initialize()`, receives server information, requests `list_tools()`, and then calls a selected tool with typed arguments. The SDK handles the protocol lifecycle.

**Q: How is MCP progress different from REST SSE?**

A: MCP progress is a protocol-level notification associated with an MCP operation. In this project, the tool uses `ctx.report_progress()`, and the MCP client receives updates through a progress callback.

REST SSE is a conventional HTTP streaming response. The FastAPI endpoint sends text/event-stream frames directly. Both use the same number-progress generator, but they expose it through different protocols.

**Q: Why disable Nginx buffering?**

A: Buffering can hold response chunks before forwarding them. That defeats the purpose of incremental progress updates. The Nginx configuration disables proxy buffering and caching for the MCP and REST paths and allows longer reads for streaming connections.

**Q: What did you learn about ASGI middleware?**

A: The inspector wraps the ASGI `receive` callable and observes incoming body chunks without replacing the MCP SDK's protocol handling. That helps separate transport-level troubleshooting from MCP application logic.

## Important limitations

- The MCP clients are command-line applications; the **servers** are FastAPI-hosted.
- The streaming example generates numbered progress events, not LLM-generated token streams.
- The request inspector prints raw headers and payloads. It needs redaction and controlled logging before production use.
- The code demonstrates protocol behavior, not a complete production MCP security platform.

**Best architectural takeaway:** Protocol design, session management, streaming behavior, and reverse-proxy configuration must work together for reliable distributed integrations.

---

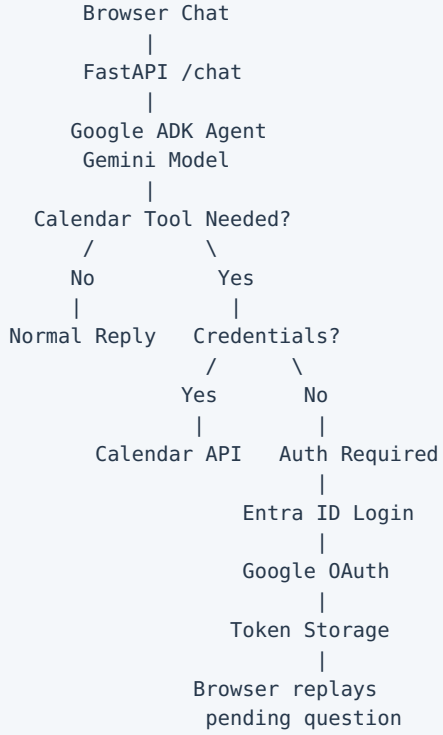
# 4. ADK Calendar Assistant

## 60-second interview explanation

> I built a read-only calendar assistant using Google's Agent Development Kit, Gemini, FastAPI, and the Google Calendar API.
>
> The agent supports conversational requests to retrieve upcoming events and check events on a requested date.
>
> The interesting part was the identity integration. I implemented a two-stage authentication process: Microsoft Entra ID for user sign-in, followed by delegated Google OAuth to grant read-only calendar access.
>
> I used an authentication sentinel so the agent only requests authentication when a calendar tool actually needs credentials.
>
> I also added token refresh handling, ADK in-memory conversation sessions, and a browser chat flow that saves a pending question during authentication and replays it after the user returns.

## Architecture

EXAMPLE



## Technical points you can justify

| Talking point | Code evidence |
| --- | --- |
| Gemini-powered ADK agent | `agent/calendar_agent.py` — `create_calendar_agent()` |
| Read-only Google Calendar operations | `agent/calendar_tools.py` |
| Entra ID OIDC authentication | `auth/entra_oidc.py` |
| Delegated Google OAuth with PKCE | `auth/google_oauth.py` |
| Token refresh | `agent/calendar_tools.py` — `_service()` |
| ADK session reuse | `agent/runner.py` — `InMemorySessionService` |
| Authentication callback handling | `server/app.py` |
| Pending-question replay | `server/static/chat.html` |

## Likely interview questions

**Q: Why use two authentication providers?**

A: Entra ID establishes the application's signed-in user identity. Google OAuth separately grants permission to access that user's Google Calendar. This separates application identity from delegated access to an external resource.

**Q: Why request read-only calendar permissions?**

A: The assistant only needs to read events. The code requests the `calendar.readonly` scope and exposes no create, update, or delete tools. That limits the authority granted to the assistant.

**Q: How does token refresh work?**

A: Before constructing the Calendar API service, the code checks whether the stored credentials have expired. When a refresh token is available, it invokes the Google credentials refresh method.

**Q: How does the assistant continue after authentication?**

A: When a calendar tool needs authorization, it returns an authentication sentinel. The runner translates that into an `auth_required` response. The browser saves the question in `sessionStorage`, starts authentication, and resubmits the question after the OAuth redirects complete.

This is **question replay**, not restoration of an interrupted ADK tool execution.

### Important limitations

The date-availability implementation filters up to 50 upcoming events by date. It does not use Google's FreeBusy API or query the full requested day's time range. Therefore, it should be described as a basic date-based event check, not an authoritative availability service.

The in-memory session and token stores are suitable for a demonstration. Before production use, the project would need persistent storage, stronger session isolation, and explicit validation that the Google OAuth callback is linked to the correct signed-in Entra user.

**Best architectural takeaway:** Agent tool access should follow authenticated identity, delegated authorization, and least-privilege design.

## 5. Interview claims that need careful wording

These are the most important distinctions I found in the actual code.

| Project | Avoid saying | Better wording |
|---|---|---|
| Travel Planner | Fully autonomous multi-agent system | Role-separated ReAct workflow with a prescribed execution sequence |
| Travel Planner | Production-grade tenant security | Catalog authorization prototype; tenant-specific controls remain future work |
| CodeGuard | Complete automated secure code review | AI-assisted metadata-based risk assessment POC |
| CodeGuard | Both LLMs jointly assess risks | Selectable OpenAI or Claude triage adapters |
| MCP Studio | Production-ready MCP platform | MCP protocol and streaming integration experiments |
| ADK Calendar | Full calendar availability engine | Upcoming-event retrieval and basic date-based availability checks |
| ADK Calendar | Agent execution resumes after OAuth | Browser replays the pending question after sign-in |

# 6. Five deeper architecture questions to prepare

These are especially relevant to a senior solutions architecture interview.

1. **Where do you enforce authorization when an AI agent calls tools?**
   Explain why a prompt is not sufficient. Use Travel Planner's gateway policy and ADK's delegated OAuth as two different authorization boundaries.
2. **How do you validate AI-generated decisions?**
   Contrast Travel Planner's Pydantic output models with CodeGuard's JSON Schema validation and retry loop. Explain why schema-valid does not mean factually correct.
3. **How do you handle failures in distributed AI workflows?**
   Use Travel Planner's maximum-step limit, MCP call timeouts, JSON-RPC error handling, and correlated logs. Acknowledge that retries, circuit breakers, and recovery are not fully implemented.
4. **How do you protect sensitive enterprise information when using LLMs?**
   Describe CodeGuard's metadata-minimization approach, ADK's least-privilege Google scope, and the risks of logging credentials or raw request bodies.
5. **How would you evolve these prototypes into an enterprise platform?**
   Discuss centralized authorization, per-tenant isolation, audit logging, persistent state, deterministic evaluation, human approval for consequential actions, observability, automated security tests, and controlled deployment.

# 7. Recommended positioning in the interview

Rather than presenting these as four unrelated applications, use them to demonstrate four related areas of architectural learning.

| Project | Primary skill to demonstrate |
| --- | --- |
| **Travel Planner MCP** | AI orchestration, separation of duties, service integration, and agent guardrails |
| **CodeGuard** | AI-assisted engineering governance, evidence extraction, and output validation |
| **MCP Studio** | Protocol-level integration, Streamable HTTP, and asynchronous communication |
| **ADK Calendar Assistant** | Agent development, enterprise identity integration, and delegated authorization |

A strong final connection across the projects is:

> These projects helped me explore different parts of responsible enterprise AI integration. Travel Planner focused on separating AI decisions from controlled execution. CodeGuard focused on using deterministic evidence to support AI-assisted decisions. MCP Studio helped me understand communication protocols and streaming behavior. The Calendar Assistant focused on identity, delegated access, and conversational state.
>
> Across all four, the recurring architectural principle was to keep AI reasoning separate from authentication, authorization, validation, and service execution.

---

**Verification scope:** The repository code was inspected and Python syntax was checked. The complete applications were not executed against live LLM providers, OAuth providers, or external services. The interview explanations distinguish what is implemented from what still requires production hardening.

# Professional Project Narratives

0 MAIN SECTIONS • PRINT-READY EDITION

## Evidence sheet: fill this in for each story

| Field | Your notes |
| --- | --- |
| Project and dates | |
| Business problem and affected users | |
| My exact responsibility and code ownership | |
| Request/data flow in 5-7 steps | |
| Technologies and versions I can verify | |
| Hardest technical decision and alternative | |
| One failure scenario and recovery | |
| Tests and rollout checks | |
| Metric: baseline, result, period and measurement | |
| What I would improve now | |

If you cannot verify a metric, say: "I don't recall the exact baseline, so I don't want to overstate the percentage. The improvement we observed was [accurate description]."

## 1. Commercial User Management — REST APIs, Java, Vert.x, and Redis

"One relevant project was our Commercial User Management platform at American Express. My responsibility included designing and developing RESTful services using Java and Vert.x and integrating them with our React/Redux frontend.

The main challenge was building a scalable self-service platform that allowed corporate clients to manage users, roles, and account information without relying heavily on manual support.

I contributed to backend API development, frontend integration, and role-management features. For frequently accessed data, we introduced Redis caching, which reduced response times by 35%, particularly during peak traffic.

The overall self-service platform also reduced client onboarding time by 30%.

This project gave me hands-on experience designing APIs, improving distributed application performance, and connecting technical decisions with customer outcomes."

## 2. Microservices Observability — Jaeger Distributed Tracing

"Another project involved improving observability for our Commercial User Management platform.

My responsibility included integrating Jaeger distributed tracing into our microservices environment.

The main challenge was troubleshooting issues across multiple services, where understanding the complete request flow was important for identifying performance problems.

We introduced distributed tracing to provide better visibility into service interactions and help engineers investigate failures more efficiently.

The implementation reduced debugging time by 40%.

This project strengthened my understanding of microservices observability and the importance of building operational visibility into distributed systems."

## 3. Commercial Reconciliation — Third-Party Platform Migration

"One of my recent projects involved migrating BTA-Connect clients from a third-party solution to an internally managed commercial reconciliation platform.

My responsibility included leading the MVP design and migration strategy, working closely with Product, Architecture, and engineering teams.

The main challenge was understanding the existing third-party capabilities, identifying essential features, and designing an enterprise-aligned solution that would reduce external dependencies.

I helped analyze the current platform, identify capability gaps, define the MVP scope, and guide architectural decisions for the in-house platform.

An important architectural consideration was balancing the initial MVP scope with the long-term product roadmap.

This project demonstrated my ability to connect business requirements, technical architecture, and platform modernization strategy."

## 4. Enterprise Identity and Access Management — IAM and MFA

"Another important initiative was implementing Identity and Access Management and multi-factor authentication across corporate payment and reconciliation applications.

My responsibility was leading the technical implementation and helping ensure alignment with enterprise security standards.

The main challenge was strengthening authentication and access controls across business-critical applications that handle sensitive payment information.

I worked with engineering teams to implement IAM and MFA capabilities and address security vulnerabilities based on OWASP practices.

We also focused on building scalable security capabilities that aligned with enterprise-wide requirements.

This experience strengthened my understanding of application security architecture, authentication, access management, and cross-platform integration."

## 5. Certificate Renewal Automation — Operational Resilience

"One project focused on automating message-signing certificate renewal for critical payment platforms.

The main challenge was that certificate maintenance required manual operational effort, and expired certificates could disrupt business-critical integrations.

I developed a certificate renewal automation tool to reduce dependency on manual processes and improve operational reliability.

The implementation reduced manual effort by 90% and achieved zero certificate-expiration incidents.

Beyond the efficiency improvement, this initiative helped strengthen operational compliance and continuity for payment services.

It reinforced the value of designing operational automation as part of an enterprise platform's reliability strategy."

## 6. Commercial Payments Modernization — File-Based to Real-Time Services

"One of the larger modernization initiatives I led involved our Commercial Payments platforms at American Express.

My responsibility included architecture, design, and development across four corporate payment products.

The main challenge was modernizing legacy file-based integrations that supported complex payment workflows and dependencies across enterprise systems.

I worked on replacing file-based interfaces with real-time services and collaborated with architecture and business teams to align integration decisions with broader platform modernization goals.

This transformation improved processing performance and contributed to approximately $500,000 in reported cost savings.

The key experience was working through cross-platform integration challenges while delivering modernization across business-critical payment applications."

## 7. Commercial Payments — Architecture Strategy and $2.35M Funding

"Another significant project involved defining a modernization strategy for our Commercial Payments applications.

My responsibility was partnering with the portfolio architect to develop a cross-platform architecture proposal and align the solution with business and technology priorities.

The challenge was identifying a modernization approach that could support long-term platform objectives while addressing existing application dependencies.

I contributed to architecture planning, technical design discussions, and stakeholder alignment across the payment portfolio.

The resulting architecture proposal supported a product-funded release strategy worth $2.35 million.

This project demonstrated how I approach solution architecture beyond individual services, considering enterprise roadmaps, cross-platform dependencies, and business investment decisions."

## 8. Payment Platform Reliability — Blue-Green, Canary, and Splunk

"One important initiative focused on improving release reliability and operational support for critical payment applications.

My responsibility included driving deployment modernization and improving how teams monitored production systems.

The main challenge was reducing disruption during application releases while improving our ability to detect and resolve production issues.

We introduced blue-green and canary deployment strategies to make releases more controlled and reduce production downtime.

These improvements reduced deployment-related downtime by 90%.

We also improved incident response through Splunk monitoring and anomaly detection, reducing Mean Time to Resolution by 40%.

This work demonstrated the connection between deployment architecture, observability, and operational resilience."

## 9. Enterprise Data Integration — Mercator to Informatica Migration

"Earlier in my career at Harbor Payments, I led a major enterprise data integration modernization initiative.

The project involved migrating legacy Mercator-based transformations to Informatica Complex Data Exchange and PowerCenter.

My responsibility included technical design, development, migration leadership, and coordination with distributed teams.

The main challenge was modernizing complex data transformation processes used in corporate invoicing and payment workflows.

I worked with data transformations involving ANSI X12 EDI, XML, and flat-file formats, along with Java programs and UNIX scripts supporting integration workflows.

The project represented an approximately $3 million enterprise migration effort.

It gave me extensive experience with enterprise data integration, legacy modernization, complex interfaces, and coordinating large technical migrations."

## 10. Enterprise SaaS Modernization — E-Invoicing and Purchase-to-Pay

"At Verian, I led technical delivery for an enterprise e-Invoicing and Purchase-to-Pay platform.

One of our major initiatives involved transforming an installer-based application into a cloud-hosted SaaS solution.

My responsibility included architecture and development leadership, coordinating globally distributed teams, and working with product stakeholders and clients.

The main challenge was evolving the existing product into a scalable SaaS offering while continuing to meet enterprise customer requirements.

I helped drive the cloud transformation, align technical requirements with the product roadmap, and establish Agile delivery practices.

The modernization supported broader customer adoption and new revenue opportunities.

This experience strengthened my understanding of enterprise SaaS architecture, product modernization, and technical delivery leadership."